ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



ĐỒ ÁN TỐT NGHIỆP
BÁO CÁO

PHÁT TRIỂN HỆ THỐNG GAME CHIẾN THUẬT GIẢI TRÍ NHẸ

"FORTRESS OF THE FALLEN"

Ngành: Khoa học Máy tính

Học kỳ 2 năm học 2025–2026 (HK252)

Mã đồ án: HK252-DATN-409

Hội đồng:	Hội đồng 4L Khoa học Máy tính
Đợt:	GD2 – DATN/LVTN (Đồ án tốt nghiệp/Luận văn tốt nghiệp) – HK252
Giảng viên hướng dẫn:	ThS. Vương Bá Thịnh

Sinh viên thực hiện	**MSSV**
Trần Minh Khang	2211472
Hà Thái Toàn	2213524

TP. Hồ Chí Minh, năm 2026

Lời cam đoan

Chúng tôi xin cam đoan rằng báo cáo đồ án "Phát triển hệ thống game chiến thuật giải trí nhẹ *Fortress of the Fallen*" là kết quả của quá trình nghiên cứu, phân tích, thiết kế và hiện thực do nhóm tự thực hiện dưới sự hướng dẫn của giảng viên hướng dẫn. Toàn bộ nội dung được trình bày trong báo cáo phản ánh trung thực các công việc mà nhóm đã thực hiện trong phạm vi đồ án, bao gồm việc xác định mục tiêu và phạm vi hệ thống, phân tích yêu cầu, thiết kế kiến trúc tổng thể client–server, thiết kế mô-đun backend, thiết kế dữ liệu và giao tiếp, cũng như mô tả hiện thực, kiểm thử và đánh giá kết quả.

Chúng tôi cam kết rằng các phần nội dung mang tính tham khảo, kế thừa hoặc tổng hợp từ các nguồn tài liệu bên ngoài (sách, bài báo, tài liệu kỹ thuật, tài liệu hướng dẫn, hình minh hoạ tham khảo, và các nguồn công khai khác) đều được trích dẫn hoặc ghi nguồn phù hợp theo quy định của báo cáo. Các ý tưởng và cách tiếp cận được tham khảo chỉ đóng vai trò làm cơ sở lý thuyết hoặc định hướng triển khai, còn việc lựa chọn giải pháp, điều chỉnh theo phạm vi đề tài, tổ chức cấu trúc hệ thống, thiết kế dữ liệu, quy ước API, và hiện thực các chức năng cụ thể là do nhóm tự xây dựng. Chúng tôi không sao chép nguyên văn hoặc sử dụng sản phẩm nghiên cứu/hiện thực của bất kỳ cá nhân hay tổ chức nào như là kết quả của nhóm khi chưa được ghi nhận nguồn trích dẫn đúng quy định.

Chúng tôi cũng cam kết rằng các số liệu, bảng biểu, sơ đồ, hình ảnh và nội dung mô tả trong báo cáo được sử dụng nhằm mục đích học thuật và phục vụ việc trình bày kết quả nghiên cứu. Những nội dung này được thể hiện theo đúng hiểu biết và kết quả thực tế của nhóm trong quá trình thực hiện. Các đánh giá về mức độ hoàn thiện, ưu điểm, hạn chế và hướng phát triển tiếp theo được trình bày khách quan theo phạm vi đã triển khai; nhóm không cố tình làm sai lệch kết quả, không thổi phồng năng lực hệ thống, và không đưa ra những nhận định vượt quá bằng chứng/khả năng kiểm chứng trong đồ án.

Chúng tôi hiểu rằng tính trung thực và chuẩn mực học thuật là yêu cầu bắt buộc đối với một báo cáo đồ án. Do đó, chúng tôi xin hoàn toàn chịu trách nhiệm trước Khoa và Trường về nội dung của báo cáo này. Trong trường hợp phát hiện có sai sót liên quan đến trích dẫn, bản quyền hoặc tính chính xác của thông tin, nhóm cam kết sẽ nghiêm túc tiếp thu, chỉnh sửa và bổ sung theo yêu cầu của giảng viên hướng dẫn và Hội đồng, đồng thời chịu mọi hình thức xử lý (nếu có) theo quy định hiện hành.

Bằng lời cam đoan này, chúng tôi xác nhận rằng các nội dung trong báo cáo là trung thực, có nguồn gốc rõ ràng và được thực hiện đúng tinh thần học thuật.

TP. Hồ Chí Minh, năm 2026

Lời cảm ơn

Trong quá trình thực hiện đồ án "Phát triển hệ thống game chiến thuật giải trí nhẹ *Fortress of the Fallen*", nhóm đã nhận được sự hỗ trợ, hướng dẫn và tạo điều kiện từ nhiều cá nhân và tập thể. Với tinh thần trân trọng, nhóm xin gửi lời cảm ơn chân thành đến những người đã đồng hành và đóng góp để nhóm có thể hoàn thành đồ án cũng như báo cáo theo đúng mục tiêu và phạm vi đề ra.

Trước hết, nhóm xin bày tỏ lòng biết ơn sâu sắc đến **ThS. Vương Bá Thịnh** — giảng viên hướng dẫn trực tiếp — người đã dành nhiều thời gian theo sát tiến độ, định hướng phương pháp tiếp cận và liên tục góp ý trong từng giai đoạn. Từ việc chốt phạm vi bài toán, thống nhất mô hình client–server, tổ chức mô-đun backend theo hướng rõ ràng, cho đến việc hoàn thiện thiết kế dữ liệu và luồng giao tiếp REST, những nhận xét của thầy đã giúp nhóm tránh được các sai lệch phổ biến khi phát triển một hệ thống game online. Không chỉ hỗ trợ về mặt kỹ thuật, thầy còn góp ý cách trình bày báo cáo theo hướng chặt chẽ, có luận điểm và có dẫn chứng, giúp nhóm cải thiện đáng kể chất lượng nội dung và tính nhất quán giữa các chương.

Nhóm xin chân thành cảm ơn **quý thầy/cô trong Khoa Khoa học và Kỹ thuật Máy tính** đã giảng dạy và trang bị nền tảng kiến thức xuyên suốt quá trình học tập. Các học phần liên quan đến công nghệ phần mềm, cơ sở dữ liệu, mạng máy tính, hệ thống phân tán, cũng như các kỹ năng phân tích và thiết kế hệ thống đã tạo tiền đề quan trọng để nhóm có thể tiếp cận đề tài theo hướng có phương pháp. Bên cạnh đó, môi trường học tập của Khoa giúp nhóm rèn luyện tư duy kỷ luật, cách làm việc theo quy trình, và ý thức về chất lượng sản phẩm, đặc biệt quan trọng với những đề tài có tính tích hợp cao giữa gameplay, backend, dữ liệu và vận hành.

Nhóm cũng xin gửi lời cảm ơn đến **Hội đồng chấm bảo vệ** đã dành thời gian đọc, đánh giá và góp ý cho đồ án. Các nhận xét phản biện và gợi ý cải thiện là cơ sở để nhóm nhìn nhận rõ hơn những điểm còn hạn chế về kiến trúc, dữ liệu và trải nghiệm người dùng, đồng thời định hướng các hạng mục cần ưu tiên trong các giai đoạn phát triển tiếp theo. Nhóm xin tiếp thu nghiêm túc các góp ý và sẽ chủ động điều chỉnh để hoàn thiện sản phẩm ở mức tốt hơn.

Trong suốt quá trình triển khai, nhóm cũng nhận được nhiều hỗ trợ từ **bạn bè, các anh/chị khóa trên và các nhóm đồ án khác** thông qua việc trao đổi kinh nghiệm và chia sẻ tài liệu tham khảo. Những trao đổi về cách tổ chức mã nguồn, cách quản lý tài nguyên (asset), cách thiết kế cấu hình theo hướng data-driven, cũng như các lưu ý khi kiểm thử luồng nghiệp vụ và xử lý lỗi thực tế đã giúp nhóm rút ngắn đáng kể thời gian thử–sai. Đây là nguồn tham khảo hữu ích để nhóm cân bằng giữa mục tiêu kỹ thuật và giới hạn thời gian của một đồ án học thuật.

Cuối cùng, nhóm xin gửi lời cảm ơn sâu sắc đến **gia đình** đã luôn động viên, tạo điều kiện và hỗ trợ tinh thần để nhóm có thể tập trung thực hiện đồ án trong giai đoạn cao điểm. Sự cảm thông và đồng hành của gia đình là động lực quan trọng để nhóm duy trì kỷ luật làm việc, hoàn thành các mốc tiến độ và vượt qua áp lực về thời gian.

Nhóm ý thức rằng, dù đã nỗ lực hoàn thiện trong phạm vi cho phép, báo cáo và sản phẩm hiện thực vẫn có thể còn thiếu sót do giới hạn nguồn lực và thời lượng thực hiện. Nhóm kính mong tiếp tục nhận được sự góp ý từ quý thầy/cô để có thể hoàn thiện hơn trong các bước

phát triển kế tiếp. Nhóm xin trân trọng cảm ơn.

TP. Hồ Chí Minh, năm 2026

Tóm tắt

Đồ án tập trung phát triển hệ thống game *Fortress of the Fallen* theo định hướng *casual strategy*. Mục tiêu của đề tài là xây dựng nền tảng client–server có khả năng vận hành ổn định cho các chức năng cốt lõi: xác thực người dùng, quản lý player state, quest, battle session, tutorial progress, nâng cấp hero/skill, nâng cấp castle, leaderboard và shop. Vòng chơi chính của bản hiện tại bám theo luồng: tìm trận → roulette → battle → chốt thắng/thua → trả thưởng và đồng bộ hậu trận.

Hệ thống được tổ chức theo mô hình server-authoritative nhằm đảm bảo tính nhất quán dữ liệu và giảm rủi ro gian lận. Client (Unity) giao tiếp với backend (NestJS) qua REST cho toàn bộ luồng đang vận hành: auth, `/me`, quest sync/claim, battle start/finish, tutorial sync, inventory upgrade, castle upgrade và shop purchase. Cơ sở dữ liệu MongoDB lưu trữ dữ liệu tiến trình, trong khi dữ liệu cấu hình gameplay được quản trị theo hướng data-driven thông qua quy trình Google Sheets → TSV → import database → nạp Config Manager ở mức áp dụng hiện tại cho các nhóm cấu hình chính.

Kết quả đồ án là một kiến trúc mô-đun rõ ràng, có khả năng mở rộng theo từng phân hệ và có bộ kiểm thử tích hợp ban đầu cho các luồng nghiệp vụ chính.

Từ khóa: casual strategy, client–server, server-authoritative, NestJS, MongoDB, REST API, data-driven configuration.

Danh mục thuật ngữ

Bảng dưới đây liệt kê các thuật ngữ và viết tắt thường được sử dụng trong báo cáo, đặc biệt liên quan tới phát triển game casual strategy, kiến trúc client–server và cơ sở dữ liệu. Các thuật ngữ này sẽ được dùng xuyên suốt các chương sau.

Thuật ngữ	**Tên đầy đủ**	**Giải thích**
Nhóm thuật ngữ về game và gameplay		
Casual strategy	Casual strategy game	Thể loại game chiến thuật giải trí nhẹ, nhịp chơi ngắn, luật chơi dễ tiếp cận và tập trung vào quyết định chiến thuật cơ bản theo từng phiên.
Battle session	Battle session	Phiên chơi chiến đấu được backend khởi tạo và kết thúc theo contract; dùng để chốt reward, tiến trình và đồng bộ trạng thái người chơi.
PvE	Player vs Environment	Chế độ người chơi đối đầu với quái, boss hoặc môi trường do hệ thống điều khiển.
Boss	Boss	Kẻ địch đặc biệt có chỉ số cao, cơ chế tấn công phức tạp, thường là trọng tâm của một trận đánh thử thách.
Hitbox	Hitbox	Vùng hình học trong game đại diện cho phạm vi đòn tấn công có hiệu lực.
Hurtbox	Hurtbox	Vùng hình học đại diện cho khu vực trên nhân vật có thể nhận sát thương.
Cooldown (CD)	Cooldown	Thời gian chờ sau khi sử dụng kỹ năng trước khi kỹ năng đó có thể được sử dụng lại.
Animation	Animation	Chuỗi hình ảnh mô phỏng chuyển động trong game.
Nhóm thuật ngữ về networking và backend		
Latency	Network Latency	Độ trễ mạng giữa client và server, thường đo bằng mili-giây (ms).
Tick rate	Server Tick Rate	Số lần server cập nhật trạng thái game trong một giây.

(còn tiếp trang sau)

Thuật ngữ	**Tên đầy đủ**	**Giải thích**
Snapshot	State Snapshot	Bản chụp trạng thái game tại một thời điểm để đồng bộ client.
Client-side Prediction	Client-side Prediction	Client dự đoán kết quả hành động trước khi server phản hồi.
Reconciliation	Reconciliation	Client điều chỉnh trạng thái dự đoán cho khớp với server.
Server Authoritative	Server Authoritative Model	Server quyết định cuối cùng mọi logic game.
Instance	Instance	Phiên bản riêng của một khu vực game cho nhóm người chơi.
Zone / Shard	Zone / Shard	Phân vùng logic thế giới game để chia tải.
Interest Management	Interest Management	Chỉ gửi dữ liệu cần thiết cho mỗi client để giảm tải hệ thống.

Mục lục

Lời cam đoan **i**

Lời cảm ơn **ii**

Tóm tắt **iv**

Danh mục thuật ngữ **v**

1 Giới thiệu **1**
1.1 Động lực . . . 1
1.2 Thể loại game và luật chơi cốt lõi . . . 1
1.3 Mục tiêu . . . 2
1.4 Phạm vi . . . 3
1.4.1 Phạm vi hiện tại . . . 3
1.4.2 Ngoài phạm vi hiện tại . . . 4
1.5 Ý nghĩa của đề tài . . . 4
1.5.1 Ý nghĩa thực tiễn . . . 4
1.5.2 Ý nghĩa khoa học và học thuật . . . 4
1.6 Cấu trúc báo cáo . . . 4

2 Kiến thức nền tảng **5**
2.1 Luật chơi cốt lõi trong bản hiện tại . . . 5
2.2 Kiến thức nền tảng về casual strategy và môi trường theo phiên . . . 6
2.2.1 Đặc điểm của casual strategy . . . 6
2.2.2 Hệ thống nhiều phiên trong casual strategy . . . 7
2.3 Kiến thức nền tảng về Game Design . . . 7
2.3.1 Core Loop và Meta Loop . . . 8
2.3.2 Progression System (Hệ thống phát triển nhân vật) . . . 8
2.3.3 Combat Design (Thiết kế chiến đấu) . . . 12
2.3.4 World System và Level Design . . . 13
2.3.5 Hệ thống kinh tế (Game Economy) . . . 13
2.3.6 UX/UI Design trong game và định hướng phong cách Pixel Art . . . 13
2.3.7 Balancing Fundamentals (Cân bằng game) . . . 14
2.4 Tổng kết chương . . . 14

3 Công nghệ sử dụng **15**
3.1 Công nghệ phía client . . . 15
3.1.1 Game engine: Unity . . . 15
3.1.2 Ngôn ngữ lập trình: C# . . . 15
3.1.3 Nguyên tắc xử lý logic ở client . . . 16
3.2 Công nghệ phía server . . . 16
3.2.1 Nền tảng runtime: Node.js . . . 16
3.2.2 Framework: NestJS và TypeScript . . . 16
3.2.3 Tác vụ nền (background jobs) . . . 16
3.3 Công nghệ giao tiếp mạng . . . 17
3.3.1 HTTP/REST . . . 17
3.4 Công nghệ lưu trữ dữ liệu . . . 17
3.4.1 MongoDB . . . 17

3.4.2 Redis . . . 17
3.4.3 Object Storage (tuỳ chọn mở rộng) . . . 17
3.5 Công cụ quản trị game configuration theo hướng data-driven . . . 18
3.6 Công cụ hỗ trợ phát triển và tài liệu hoá . . . 18
3.6.1 Quản lý mã nguồn và phối hợp nhóm . . . 18
3.6.2 Công cụ mô hình hoá và tài liệu . . . 18
3.7 Tổng kết . . . 18

4 Các công trình liên quan **19**
4.1 Cách dùng nguồn tham khảo ở thời điểm hiện tại . . . 19
4.2 Đối chiếu nguồn cảm hứng và mức tương đồng hiện tại . . . 19
4.3 Nguồn tham khảo có ảnh hưởng trực tiếp đến triển khai . . . 20
4.4 Kết luận chương . . . 20

5 Phân tích yêu cầu **21**
5.1 Phạm vi và giả định . . . 21
5.1.1 Phạm vi hiện tại . . . 21
5.1.2 Giả định . . . 21
5.2 Tác nhân và ranh giới hệ thống . . . 21
5.2.1 Tác nhân . . . 21
5.2.2 Ranh giới hệ thống . . . 22
5.3 Yêu cầu chức năng . . . 22
5.3.1 Danh sách yêu cầu chức năng hiện tại . . . 22
5.4 Yêu cầu phi chức năng . . . 25
5.5 Yêu cầu dữ liệu . . . 25
5.5.1 Phân loại dữ liệu . . . 25
5.5.2 Yêu cầu mô hình dữ liệu mức logic . . . 25
5.6 Tiêu chí chấp nhận theo trạng thái hiện tại . . . 26

6 Phân tích hệ thống **26**
6.1 Tổng quan phân hệ và ranh giới trách nhiệm . . . 26
6.2 Tác nhân hệ thống . . . 27
6.3 Phân tích use case theo phạm vi hiện tại . . . 27
6.3.1 Nhóm xác thực và phiên . . . 27
6.3.2 Nhóm đồng bộ trạng thái người chơi . . . 30
6.3.3 Nhóm quest . . . 30
6.3.4 Nhóm battle session . . . 30
6.3.5 Nhóm tutorial progress . . . 30
6.3.6 Nhóm progression thao tác trực tiếp . . . 31
6.4 Đặc tả use case trọng yếu . . . 32
6.4.1 UC-01 – Đăng nhập và thiết lập phiên . . . 32
6.4.2 UC-02 – Đồng bộ player state . . . 33
6.4.3 UC-03 – Đồng bộ và claim quest . . . 33
6.4.4 UC-04 – Khởi tạo battle session . . . 35
6.4.5 UC-05 – Kết thúc battle và đồng bộ hậu trận . . . 36
6.4.6 UC-06 – Đồng bộ tutorial progress . . . 38
6.4.7 UC-07 – Nâng cấp hero và skill . . . 38
6.4.8 UC-08 – Mua shop offer và hiển thị phần thưởng . . . 38
6.4.9 UC-09 – Nâng cấp castle . . . 39
6.5 Hướng dẫn chơi & flow gameplay hiện tại . . . 39

6.5.1 Hướng dẫn chơi nhanh . . . 39
6.5.2 Flow chơi theo battle-first . . . 41
6.5.3 Luật battle . . . 42
6.5.4 Luật thắng/thua . . . 42
6.5.5 Luật reward và đồng bộ hậu trận . . . 42
6.5.6 Mở rộng sang các tính năng khác . . . 42
6.6 Quy tắc nghiệp vụ và ràng buộc hệ thống . . . 42
6.6.1 Server-authoritative state . . . 42
6.6.2 Idempotency và retry . . . 43
6.6.3 Nhất quán bất đồng bộ . . . 43
6.6.4 Tương thích cấu hình . . . 43
6.7 Phân tích dữ liệu ở mức logic . . . 43
6.7.1 Dữ liệu tiến trình . . . 43
6.7.2 Thực thể lõi theo luồng hiện tại . . . 43
6.7.3 Quan hệ dữ liệu quan trọng . . . 44
6.8 Tổng kết chương . . . 44

7 Thiết kế hệ thống 44
7.1 Mục tiêu thiết kế . . . 44
7.2 Kiến trúc tổng thể và triển khai . . . 45
7.2.1 Tổng quan triển khai . . . 45
7.2.2 Nguyên tắc server-authoritative . . . 46
7.2.3 Phân tách giao tiếp: REST API vận hành . . . 46
7.3 Thiết kế mô-đun backend . . . 46
7.3.1 Các thành phần dùng chung (cross-cutting) . . . 47
7.4 Thiết kế dữ liệu . . . 48
7.4.1 Nguyên tắc thiết kế dữ liệu . . . 48
7.4.2 Mô hình thực thể chính (ERD mức logic) . . . 48
7.4.3 Phân tích quan hệ và phụ thuộc thuộc tính . . . 49
7.4.4 Từ điển dữ liệu (Data Dictionary) ở mức collection . . . 51
7.4.5 Chỉ mục (indexes) và khoá truy vấn . . . 57
7.5 Thiết kế hệ thống chỉ số và tăng trưởng nhân vật . . . 57
7.5.1 Thuộc tính nền và chỉ số suy diễn . . . 57
7.5.2 Tính toán chỉ số theo hướng data-driven . . . 58
7.6 Thiết kế quản trị cấu hình game (Config Pipeline) . . . 59
7.6.1 Luồng dữ liệu cấu hình . . . 59
7.6.2 Thiết kế Config Manager . . . 59
7.7 Thiết kế giao tiếp client–server . . . 59
7.7.1 REST API: nhóm endpoint và quy ước . . . 59
7.8 Thiết kế giao diện người dùng (UI/UX) mức tham chiếu . . . 60
7.8.1 Luồng UI xác thực: Login/Register . . . 61
7.8.2 UI lobby và điều hướng phân hệ . . . 62
7.8.3 UI quest và milestone reward . . . 63
7.8.4 UI shop và popup phần thưởng . . . 64
7.8.5 UI hero inventory, upgrade và evolve . . . 65
7.8.6 UI tutorial và onboarding . . . 66
7.8.7 UI vào trận và gameplay battle . . . 67
7.8.8 UI kết quả trận và reward . . . 68
7.9 Bảo mật, nhất quán và quan sát hệ thống . . . 68

7.9.1 Bảo mật . . . 68
7.9.2 Nhất quán dữ liệu và giao dịch nghiệp vụ . . . 69
7.9.3 Quan sát hệ thống (observability) . . . 69
7.10 Tổng kết chương . . . 69

8 Hiện thực và tích hợp hệ thống 69
8.1 Mục tiêu chương . . . 69
8.2 Baseline triển khai hiện tại . . . 69
8.3 Hiện thực phía client (Unity) . . . 70
8.3.1 Tổ chức lớp giao tiếp backend . . . 70
8.3.2 Nguyên tắc đồng bộ dữ liệu . . . 70
8.4 Hiện thực phía backend theo contract . . . 71
8.4.1 Nhóm endpoint đã tích hợp . . . 71
8.4.2 Quy tắc dữ liệu quan trọng . . . 71
8.5 Luồng tích hợp đã kiểm chứng . . . 71
8.5.1 Luồng xác thực và vào lobby . . . 71
8.5.2 Luồng quest và reward . . . 71
8.5.3 Luồng battle và hậu battle . . . 72
8.5.4 Luồng level-up, tutorial và castle . . . 72
8.5.5 Luồng shop purchase và popup reward . . . 72
8.6 Claim và Evidence . . . 73
8.7 Các hạng mục mở rộng chưa hoàn tất . . . 73
8.8 Tổng kết chương . . . 73

9 Đánh giá hệ thống 74
9.1 Mục tiêu và phương pháp đánh giá . . . 74
9.2 Bao phủ yêu cầu chức năng . . . 74
9.3 Đánh giá nhất quán dữ liệu . . . 74
9.3.1 Điểm đã đạt . . . 74
9.3.2 Điểm cần kiểm soát . . . 75
9.4 Đánh giá phi chức năng . . . 75
9.5 Automation Testing với Unity Test Framework . . . 75
9.5.1 Cài đặt package kiểm thử . . . 76
9.5.2 Bộ EditMode tests . . . 76
9.5.3 Bộ PlayMode tests . . . 77
9.5.4 Kết quả chạy test trên player/test runner . . . 77
9.6 Rủi ro còn lại và hướng giảm thiểu . . . 78
9.7 Tổng kết chương . . . 78

10 Kết luận và hướng phát triển 78
10.1 Tổng kết kết quả hiện tại . . . 78
10.2 Đóng góp . . . 79
10.3 Hạn chế hiện tại . . . 79
10.4 Hướng phát triển tiếp theo . . . 79
10.5 Kết luận . . . 79

Phân công nhiệm vụ và đóng góp của các thành viên 80

Roadmap giai đoạn tiếp theo 81

Tài liệu tham khảo **82**

Danh sách hình vẽ

1 Ví dụ mô hình hoá Core Loop trong casual strategy 8
2 Mô hình hoá thuộc tính nền, chỉ số suy diễn và nhóm trait đặc biệt 9
3 Use case cho phân hệ xác thực và phiên 28
4 Activity flow cho phân hệ xác thực 29
5 Use case cho phân hệ battle session 30
6 Use case cho phân hệ progression người chơi 31
7 Use case cho phân hệ character và formation 32
8 Sequence đăng nhập và thiết lập phiên 33
9 Sequence làm mới access token 33
10 Sequence đồng bộ và claim quest 34
11 Activity flow cho luồng claim quest 35
12 Sequence khởi tạo và kết thúc battle session 36
13 Activity flow cho vòng đời battle session 37
14 Flow đầu vào trận theo runtime hiện tại: matching → roulette. 40
15 Flow core gameplay: battle → chốt kết quả → reward. 41
16 Sơ đồ triển khai tổng quan hệ thống (bản cập nhật) 45
17 Bộ sơ đồ ERD mức logic theo phân hệ (trình bày bằng hình) 49
18 Mô hình thuộc tính nền và chỉ số suy diễn 58
19 Luồng UI xác thực và tải dữ liệu ban đầu 61
20 UI lobby, nâng cấp castle và leaderboard 62
21 UI nhiệm vụ ngày, tuần và thành tựu 63
22 UI shop: catalog offer và popup reward 64
23 UI quản lý hero/unit: danh sách, UIUnit, chi tiết, nâng cấp và tiến hoá 65
24 UI tutorial điều hướng người chơi mới 66
25 UI chuẩn bị trận và gameplay battle 67
26 UI kết quả trận, reward và level-up 68
27 Unity Package Manager với Test Framework và UI Test Framework được cài trong dự án 76
28 Kết quả chạy EditMode tests cho các ca kiểm thử logic UI cục bộ 76
29 Kết quả chạy PlayMode tests trên scene runtime để kiểm tra tích hợp thành phần 77
30 Kết quả test runner ở chế độ player dùng để đối chiếu hành vi kiểm thử ngoài editor 77

Danh sách bảng

1 Vai trò của 6 thuộc tính nền trong thiết kế build 10
2 Ví dụ quan hệ phụ thuộc Secondary Stats từ Primary Attributes 10
3 Ví dụ mốc level và phần thưởng mở slot kỹ năng 11
4 Mức tương đồng hiện tại giữa nguồn cảm hứng và sản phẩm 20
5 Tác nhân và vai trò trong phạm vi triển khai hiện tại 27
6 Bản đồ mô-đun backend và dữ liệu liên quan 47
7 Tóm tắt Entity–Relation–Constraint theo nhóm ERD 51
8 Từ điển dữ liệu – Collection USER 51
9 Từ điển dữ liệu – Collection GAME_PROFILE 52
10 Từ điển dữ liệu – Collection SESSION_RUNTIME 53
11 Từ điển dữ liệu – Collection CHARACTER 53
12 Từ điển dữ liệu – Collection INVENTORY 54
13 Từ điển dữ liệu – Collection CHARACTER_EQUIPMENT 55
14 Từ điển dữ liệu – Collection CHARACTER_SKILLS 56
15 Từ điển dữ liệu – Collection CONFIG_ITEM 56
16 Từ điển dữ liệu – Collection CONFIG_SKILL 57
17 Bảng traceability claim kỹ thuật và bằng chứng triển khai 73
18 Đối chiếu mức bao phủ FR theo trạng thái triển khai hiện tại 74
19 Rủi ro hiện tại và biện pháp giảm thiểu 78
20 Phân công nhiệm vụ và đóng góp của các thành viên 80
21 Roadmap sau baseline triển khai (mốc thời gian tính từ 05/2026) 81

1 Giới thiệu

1.1 Động lực

Trong những năm gần đây, thị trường game trực tuyến tiếp tục tăng trưởng và xu hướng người chơi chuyển dịch mạnh sang các sản phẩm có vòng chơi ngắn, dễ tiếp cận và cập nhật liên tục[1]. Với định hướng casual strategy, yêu cầu trọng tâm là hệ thống vận hành ổn định, đồng bộ trạng thái nhất quán và duy trì tiến trình dài hạn (meta-progression) theo từng phiên chơi.

Để đáp ứng các kỳ vọng đó, hệ thống phía server không chỉ xử lý xác thực và lưu trữ dữ liệu, mà còn phải đảm nhiệm các bài toán đặc thù của game online như:

- Quản lý phiên chơi và xử lý nhiều kết nối đồng thời.
- Đồng bộ trạng thái theo mô hình phù hợp để giảm sai lệch giữa client và server, đồng thời hạn chế gian lận ở mức kiến trúc[2].
- Thiết kế dữ liệu tiến trình người chơi (profile/statistics/tutorial/inventory/formation/quest/battle history) sao cho có thể mở rộng và truy vết.
- Tổ chức các phân hệ nghiệp vụ cốt lõi (auth, player state, quest, battle, reward, progression, castle, shop) trong một kiến trúc nhất quán, giảm phụ thuộc chéo và thuận lợi bảo trì[3].

Ngoài ra, một thách thức quan trọng trong thiết kế game là quản trị **game configuration** (dữ liệu tĩnh) như định nghĩa vật phẩm, công trình, tham số cân bằng, bảng tỉ lệ và cấu hình sự kiện. Nếu cấu hình bị gắn chặt vào mã nguồn, việc điều chỉnh nội dung và cân bằng sẽ phụ thuộc vào chu kỳ build và phát hành. Do đó, hướng tiếp cận *data-driven* thường được áp dụng để tách cấu hình khỏi code, giúp quy trình cập nhật linh hoạt hơn và giảm rủi ro "hard-code" tham số[4, 5].

Trong bối cảnh đó, đề tài *Fortress of the Fallen* được triển khai theo hướng **vừa thiết kế vừa hiện thực** hệ thống client–server. Báo cáo không chỉ mô tả kiến trúc và mô hình dữ liệu, mà còn phản ánh các luồng đã chạy thực tế trong bản hiện tại.

1.2 Thể loại game và luật chơi cốt lõi

Định vị thể loại: *Fortress of the Fallen* được định vị là **casual strategy**. Trọng tâm trải nghiệm là vòng chơi ngắn, thao tác dễ tiếp cận, nhưng vẫn giữ quyết định chiến thuật ở từng phiên trận và progression dài hạn qua nhiều phiên.

Flow chơi chính (battle-first):

1. Người chơi vào **matching/tìm trận** từ lobby.
2. Hệ thống chuyển qua **roulette** để xác nhận vào trận.
3. Người chơi vào **battle session** và thực hiện gameplay của màn.

4. Trận được **chốt thắng/thua** theo điều kiện mục tiêu của phiên chơi.

5. Sau khi chốt kết quả, hệ thống trả **reward** và đồng bộ hậu trận.

6. Người chơi quay về lobby và rẽ sang các tính năng khác như **quest, nâng cấp, shop**.

Luật trận đấu cụ thể trong bản hiện tại:

- **Thời lượng trận:** trận thường có thời gian **180 giây**; luồng onboarding dùng **90 giây**. Khi hết giờ, trận kết thúc theo điều kiện `Timeout`.
- **Cơ chế chiếm ô đất:** khi unit của phe Player/Enemy di chuyển trên bản đồ, hệ thống Territory sẽ cập nhật owner của ô theo phe đó (visual đổi màu theo phe kiểm soát). Nói ngắn gọn: **unit phe nào dẫm lên thì ô chuyển màu phe đó**.
- **Điều kiện kết thúc sớm:** khi **một trong hai nhà chính (main hall/base fortress) bị phá huỷ**, trận kết thúc ngay theo điều kiện `ReachCastle`.
- **Luật thắng/thua theo tỷ lệ kiểm soát:** ở bước chốt kết quả, hệ thống so sánh **tỷ lệ ô đất kiểm soát** của hai phe để xác định Win/Lose/Draw (đây là tỷ lệ ô Player trên tổng ô đã bị chiếm bởi Player+Enemy).
- **Tính điểm và hậu trận:** sau khi chốt kết quả, hệ thống trả reward, cập nhật progression/ quest và đồng bộ state hậu trận về backend.

Nguyên tắc vận hành: trạng thái tiến trình và phần thưởng được xác nhận theo backend response; client hiển thị theo dữ liệu đã chốt để đảm bảo nhất quán xuyên suốt vòng chơi.

1.3 Mục tiêu

Mục tiêu tổng quát của đề tài là xây dựng và vận hành **nền tảng hệ thống client–server** cho trò chơi casual strategy *Fortress of the Fallen*, bao gồm kiến trúc, mô hình dữ liệu, luồng nghiệp vụ, hợp đồng giao tiếp và các màn hình UI trọng yếu.

Trong phạm vi hiện tại, đề tài tập trung vào các mục tiêu cụ thể:

- Chuẩn hoá và vận hành các phân hệ cốt lõi: Authentication, Player State, Tutorial Progress, Quest, Battle Session (start/finish), Reward, Progression, Castle Upgrade và Shop.
- Đồng bộ contract giữa Unity client và backend API cho các luồng chính: đăng nhập/đăng ký/refresh/logout, tải `/me`, đồng bộ/claim quest, vào trận, kết trận, cập nhật tutorial progress, nâng cấp hero/skill, nâng cấp castle, tải catalog shop và purchase offer.
- Thiết kế kiến trúc triển khai tổng thể theo mô hình client–server, bao gồm các thành phần backend và hạ tầng dữ liệu; xác định cơ chế giao tiếp phù hợp cho các tác vụ request–response[2, 3].

- Thiết kế mô hình dữ liệu mức logic cho các thực thể chính: user, session, profile, statistics, tutorial progress, inventory, formation, quest state và battle history.
- Thiết kế pipeline quản trị **game configuration (tĩnh)** theo hướng data-driven:

  Google Sheets → TSV → Import vào DB (collections cấu hình chính) → Load vào *Config Manager* khi khởi động.

  Pipeline này **chỉ áp dụng cho game configuration**
  Ví dụ: CONFIG_ITEM, CONFIG_SKILL, CONFIG_ENEMY và các bảng cấu hình mở rộng, không áp dụng cho dữ liệu tiến trình người chơi.

1.4 Phạm vi

1.4.1 Phạm vi hiện tại

Phạm vi hiện tại bao gồm **phân tích, thiết kế và triển khai vận hành** cho các luồng gameplay/lobby cốt lõi:

- **Luồng xác thực và state người chơi:** đăng nhập/đăng ký/refresh token, tải player state từ `/me`, áp dụng profile/statistics/currency/inventory/formation/tutorial progress.
- **Luồng quest và reward:** đồng bộ daily/weekly/achievement, claim quest, claim milestone reward và cập nhật tiến trình sau battle.
- **Luồng battle và hậu battle:** tạo battle session, xử lý kết trận, nhận reward/exp/currency/quest updates, đồng bộ lại UI lobby.
- **Luồng progression và unit management:** nâng cấp/evolve hero, cập nhật chỉ số và tutorial step liên quan đến nâng cấp.
- **Luồng shop và reward popup:** tải shop catalog, kiểm tra currency đủ mua, gửi purchase và hiển thị popup phần thưởng theo response backend.
- **Kiến trúc triển khai đang sử dụng:** phân tách vai trò client và server; backend vận hành theo API server mô-đun hoá và hạ tầng dữ liệu chính; các thành phần mở rộng quy mô được giữ ở roadmap.
- **Thiết kế kiến trúc phần mềm theo mô-đun:** phân rã module theo phân hệ, xác định dịch vụ dùng chung (validation, resource, RNG, inventory rules, sync/audit) nhằm giảm trùng lặp và đảm bảo nhất quán nghiệp vụ.
- **Thiết kế dữ liệu mức logic:** xây dựng mô hình dữ liệu cho tiến trình người chơi (dynamic) và mô hình cho cấu hình game (static), làm rõ điểm tham chiếu giữa instance gameplay và definition trong config.

- **Hợp đồng giao tiếp:** REST API được dùng xuyên suốt cho các luồng cốt lõi đang vận hành.
- **Thiết kế pipeline cấu hình tĩnh:** mô tả quy trình export/import TSV và cơ chế nạp cấu hình vào Config Manager khi khởi động để phục vụ truy xuất thống nhất.

1.4.2 Ngoài phạm vi hiện tại

Các nội dung sau **chưa thuộc phạm vi hiện tại**:

- Hoàn thiện gameplay ở mức vận hành cao hơn (độ sâu content, animation/FX và tuning cảm giác điều khiển trên nhiều thiết bị).
- Mở rộng các mode đối kháng khi cần (matchmaking nâng cao, cơ chế giảm gian lận, mùa giải và phần thưởng dài hạn).
- Live-ops thương mại và cân bằng kinh tế ở quy mô lớn (event vận hành định kỳ, monetization, A/B tuning).
- Benchmark tải lớn, tối ưu vận hành và triển khai multi-region ở mức SRE/DevOps.

1.5 Ý nghĩa của đề tài

1.5.1 Ý nghĩa thực tiễn

Đề tài cung cấp một hệ thống client–server đã tích hợp các luồng gameplay/lobby cốt lõi cho game casual strategy, đồng thời làm rõ cách tổ chức các phân hệ và quản trị dữ liệu nhất quán trong sản phẩm. Đặc biệt, cơ chế quản trị *game configuration* theo hướng data-driven tạo nền tảng cho mở rộng nội dung và cân bằng ở các giai đoạn sau mà không phụ thuộc hoàn toàn vào chỉnh sửa mã nguồn[4, 3].

1.5.2 Ý nghĩa khoa học và học thuật

Đề tài tạo môi trường để vận dụng tổng hợp kiến thức về kiến trúc phần mềm, hệ thống phân tán, cơ sở dữ liệu, và nguyên lý đồng bộ trạng thái trong game online. Việc mô hình hoá nghiệp vụ và dữ liệu ở mức logic, sau đó đối chiếu với luồng đã chạy thực tế, giúp tăng tính hệ thống và khả năng đánh giá rủi ro kỹ thuật[2, 3].

1.6 Cấu trúc báo cáo

Báo cáo được tổ chức như sau:

- **Chương 2 – Kiến thức nền tảng:** Trình bày các khái niệm nền tảng về casual strategy, core loop/meta loop, nguyên tắc client–server và hướng tiếp cận data-driven cho cấu hình[4, 5].

- **Chương 3 – Công nghệ sử dụng:** Giới thiệu các công nghệ đang sử dụng trong bản hiện tại và các hạng mục mở rộng đã tách riêng thành roadmap[6, 7, 8, 9].
- **Chương 4 – Các công trình liên quan:** Tổng hợp tham khảo theo nhóm phân hệ và bảng đối chiếu để rút ra các điểm áp dụng vào thiết kế.
- **Chương 5 – Phân tích yêu cầu:** Xác định yêu cầu chức năng, phi chức năng, dữ liệu và phạm vi hiện tại của hệ thống.
- **Chương 6 – Phân tích hệ thống:** Phân tích actors, use case, luồng nghiệp vụ và dữ liệu mức logic phục vụ thiết kế.
- **Chương 7 – Thiết kế hệ thống:** Trình bày kiến trúc triển khai, kiến trúc module, mô hình dữ liệu và hợp đồng giao tiếp; mô tả pipeline quản trị game configuration.
- **Chương 8 – Hiện thực hệ thống:** Trình bày phần đã tích hợp thực tế giữa client và backend, cùng các hạng mục còn lại trong backlog kỹ thuật.
- **Chương 9 – Đánh giá hệ thống:** Đánh giá mức độ bao phủ yêu cầu dựa trên trạng thái triển khai hiện tại, tính nhất quán dữ liệu và các rủi ro còn tồn tại.
- **Chương 10 – Kết luận:** Tổng kết kết quả đã triển khai và định hướng phát triển cho các giai đoạn tiếp theo.

2 Kiến thức nền tảng

Chương này hệ thống hoá các kiến thức cơ sở phục vụ cho việc thiết kế một trò chơi casual strategy trong môi trường trực tuyến. Nội dung tập trung vào hai nhóm chính: (i) đặc trưng của casual strategy và các nguyên lý đồng bộ state theo mô hình client–server; (ii) các khái niệm nền tảng của Game Design gồm vòng lặp chơi (loop), hệ thống tiến trình (progression), thiết kế chiến đấu, thiết kế thế giới, kinh tế trong game, UX/UI và cân bằng thông số[4, 5]. Các khái niệm ở chương này đóng vai trò "khung lý thuyết" để liên kết xuyên suốt sang phần phân tích yêu cầu (Chương 5) và phần thiết kế hệ thống (Chương 7).

Trong bản hiện tại của đề tài, một số cơ chế trong chương này đã được áp dụng trực tiếp trong runtime (xác thực, quest, battle session, reward, shop purchase, đồng bộ player state). Một số cơ chế còn lại được giữ ở mức tham chiếu thiết kế để định hướng mở rộng dài hạn (ví dụ các nhánh build chuyên sâu).

2.1 Luật chơi cốt lõi trong bản hiện tại

Định vị thể loại: *Fortress of the Fallen* được triển khai theo hướng **casual strategy**, tập trung vào vòng trận ngắn và quyết định chiến thuật dễ tiếp cận.

Flow gameplay chính (battle-first):

1. Từ lobby, người chơi bắt đầu **tìm trận/matching**.
2. Hệ thống chuyển qua bước **roulette** trước khi vào trận.
3. Người chơi vào **battle** và thực hiện thao tác chiến đấu trong phiên.
4. Hệ thống **chốt kết quả thắng/thua** theo điều kiện của phiên trận.
5. Kết quả được dùng để **trả thưởng và đồng bộ hậu trận** (currency, progression, quest progress).
6. Người chơi quay về lobby và tiếp tục các nhánh tính năng như **quest, upgrade, shop**.

Nguyên tắc dữ liệu: trạng thái sau trận được xác nhận theo backend response; client chỉ hiển thị và cập nhật theo dữ liệu đã chốt để giữ nhất quán.

2.2 Kiến thức nền tảng về casual strategy và môi trường theo phiên

2.2.1 Đặc điểm của casual strategy

Casual strategy trong phạm vi đề tài kết hợp vòng trận ngắn, quyết định chiến thuật đơn giản và tiến trình nâng cấp dài hạn. Các đặc điểm cốt lõi thường gặp[4, 5]:

- **Điều khiển trực tiếp và phản hồi tức thời**: hành vi chiến đấu (tấn công, né tránh, di chuyển, dùng kỹ năng) cần phản hồi nhanh để tạo cảm giác "đã tay".
- **Chiến đấu gắn với animation**: logic đòn đánh phụ thuộc vào các mốc thời gian của animation (khi nào hitbox bật/tắt, khi nào có i-frame, thời gian hồi chiêu/khôi phục).
- **Kết hợp kỹ năng người chơi và chỉ số nhân vật**: thao tác (timing, vị trí, né đòn) quyết định hiệu quả ngắn hạn; hệ thống chỉ số, trang bị và kỹ năng quyết định "tiềm năng" dài hạn.
- **Tính công bằng trong va chạm và sát thương**: cần quy ước rõ về hitbox/hurtbox, phạm vi hiệu lực và điều kiện trúng đòn nhằm giảm cảm giác "bị oan".

Một hành động tấn công điển hình thường được chia thành ba pha[4]:

1. **Startup**: thời gian chuẩn bị, người chơi có thể bị ngắt hoặc bị trừng phạt nếu ra đòn sai thời điểm.
2. **Active**: thời gian hitbox có hiệu lực, đòn có thể gây sát thương/hiệu ứng.
3. **Recovery**: thời gian hồi lại, tạo "cửa sổ" để đối thủ phản công.

Việc xác định hợp lý độ dài ba pha giúp kiểm soát nhịp độ combat (pacing), tạo khác biệt giữa vũ khí/kỹ năng, và là cơ sở để cân bằng.

2.2.2 Hệ thống nhiều phiên trong casual strategy

Với casual strategy online, bài toán kỹ thuật trung tâm là đồng bộ trạng thái trong điều kiện mạng không ổn định: độ trễ (latency), jitter, mất gói, chênh lệch tốc độ khung hình giữa các máy. Thực tế triển khai thường dựa trên mô hình **server authoritative**[2, 10].

Mô hình client–server và tính thẩm quyền (authoritative) Trong mô hình phổ biến:

- **Client** thu thập input, dự đoán (prediction) ở mức hiển thị và render hình ảnh/âm thanh.
- **Server** giữ trạng thái "chuẩn", xử lý các luật gameplay quan trọng (xác nhận vị trí, tính sát thương, trạng thái kỹ năng, kiểm tra điều kiện) và chống gian lận ở mức hệ thống[2].

Ưu điểm của server authoritative là giảm sai lệch giữa người chơi và hạn chế cheat; đánh đổi là cần cơ chế che giấu độ trễ để giữ cảm giác điều khiển mượt.

Tick-rate và vòng lặp mô phỏng Server chạy theo chu kỳ mô phỏng (tick). Ở mỗi tick, server có thể thực hiện: nhận input, cập nhật trạng thái, xử lý va chạm/chiến đấu, sau đó gửi snapshot cho client[2]. Tick-rate cao cho phản hồi tốt hơn nhưng tăng chi phí CPU/băng thông; tick-rate thấp tiết kiệm hơn nhưng đòi hỏi client xử lý nội suy tốt để tránh giật.

Các kỹ thuật che giấu độ trễ Để gameplay cảm giác "theo phiên" dù có latency, các kỹ thuật nền tảng thường phối hợp[2, 10]:

- **Snapshot interpolation**: client nhận snapshot định kỳ và nội suy để hiển thị chuyển động mượt.
- **Client-side prediction**: client tạm dự đoán kết quả dựa trên input thay vì chờ phản hồi server.
- **Reconciliation**: khi server gửi trạng thái chuẩn, client điều chỉnh sai lệch một cách "êm" để tránh giật.

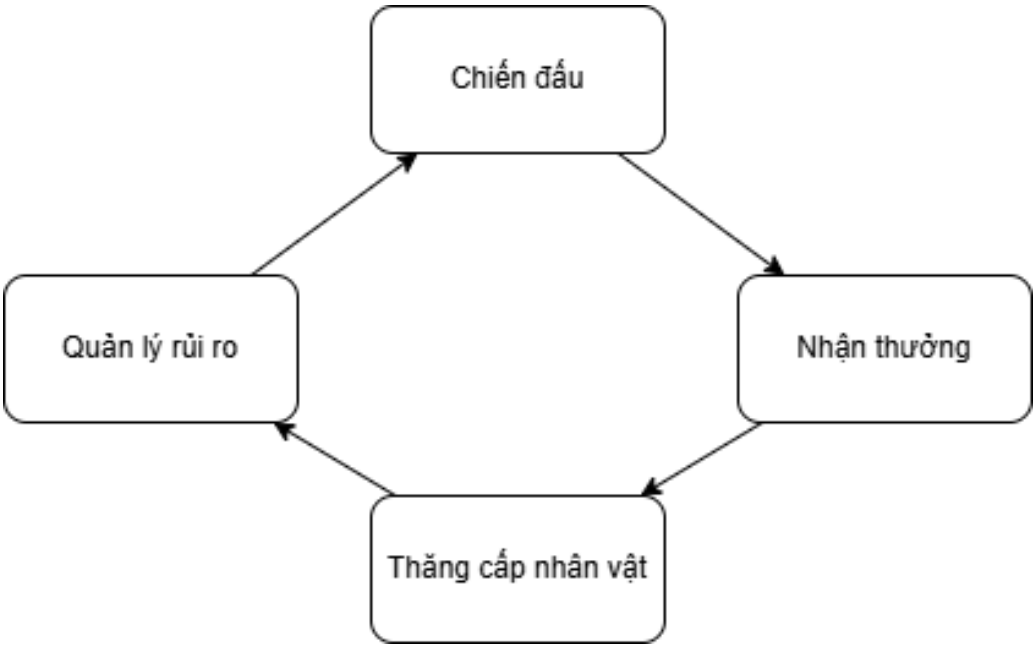
Các kỹ thuật này là nền tảng khi thiết kế combat có yêu cầu timing chính xác (né, dash, cast skill) và là tiền đề cho các quyết định ở phần thiết kế hệ thống.

2.3 Kiến thức nền tảng về Game Design

Game Design mô tả cách trò chơi vận hành: người chơi làm gì, vì sao họ muốn làm điều đó, và trò chơi phản hồi như thế nào[4, 5]. Một casual strategy online thường được phân tích theo các lớp: vòng lặp chơi, tiến trình nhân vật, chiến đấu, nội dung/không gian, kinh tế phần thưởng và UX/UI.

2.3.1 Core Loop và Meta Loop

Core Loop là vòng lặp hành vi ngắn hạn lặp lại liên tục. Với casual strategy, core loop thường xoay quanh: (i) tham gia giao tranh; (ii) nhận phần thưởng; (iii) dùng phần thưởng để mạnh lên; (iv) quay lại nội dung khó hơn[4].



Hình 1: Ví dụ mô hình hoá Core Loop trong casual strategy

Một core loop tốt cần dễ hiểu, có nhịp độ hợp lý và cung cấp "phản hồi thoả mãn" sau mỗi vòng lặp (âm thanh, hiệu ứng, tiến bộ nhìn thấy được)[5].

Meta Loop là lớp động lực dài hạn (giữ chân người chơi): tăng cấp, mở khoá hệ thống, sưu tầm trang bị/kỹ năng, tối ưu hoá build, tham gia nội dung endgame[4]. Meta loop cần có các **mốc rõ ràng** để tạo cảm giác tiến bộ, đồng thời đủ không gian cho người chơi cá nhân hoá (nhiều hướng build khác nhau).

2.3.2 Progression System (Hệ thống phát triển nhân vật)

Progression là "xương sống" của casual strategy: quyết định tốc độ mạnh lên, cách người chơi ra quyết định build và cách nội dung được mở khoá theo thời gian[4, 5]. Trong phạm vi kiến thức nền tảng của đề tài, hệ thống tiến trình được trình bày theo ba lớp: (i) chỉ số nền (attributes/stats); (ii) tăng cấp và mở slot kỹ năng; (iii) tiến trình lớp nghề nghiệp (class) và điều kiện mở khoá.

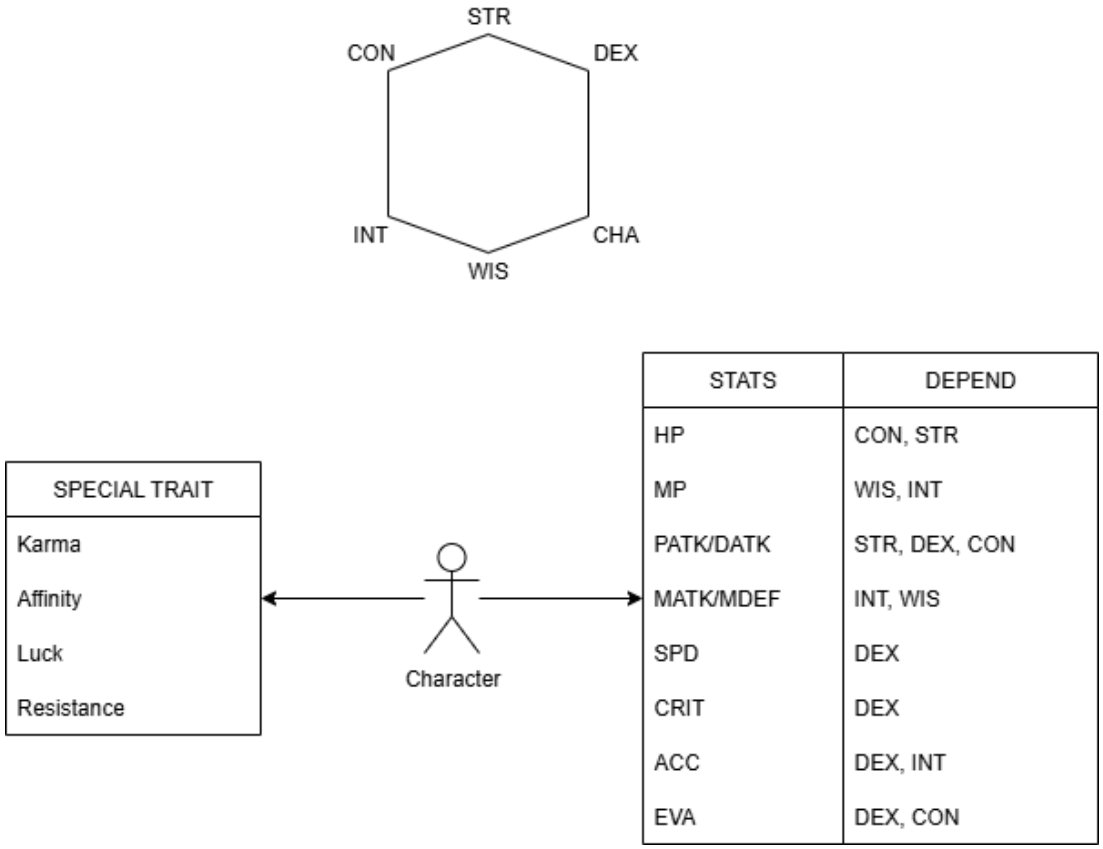
Phân loại chỉ số: Primary vs. Secondary Một thiết kế phổ biến là chia chỉ số thành:

- **Primary Attributes**: người chơi phân bổ trực tiếp (ví dụ STR/DEX/CON/INT/WIS/CHA).

- **Secondary Stats**: suy ra từ primary (HP, MP, tốc độ, chí mạng, chính xác, né tránh, tấn công/phòng thủ, . . .).

Cách phân lớp này giúp hệ thống dễ mở rộng và thuận lợi cho cân bằng: thay vì chỉnh trực tiếp mọi thứ, có thể điều chỉnh hệ số chuyển đổi giữa primary và secondary.

Hệ thống chỉ số 6 thuộc tính và quan hệ phụ thuộc Trong thiết kế tham chiếu của đề tài, nhân vật có 6 thuộc tính nền và các chỉ số chiến đấu suy diễn theo quan hệ phụ thuộc như Hình 2. Mục tiêu là đảm bảo *mỗi thuộc tính đều có giá trị* và hỗ trợ nhiều hướng build (tấn công, sống sót, cơ động, hỗ trợ).



Hình 2: Mô hình hoá thuộc tính nền, chỉ số suy diễn và nhóm trait đặc biệt

Bảng 1: Vai trò của 6 thuộc tính nền trong thiết kế build

Thuộc tính	**Ý nghĩa thiết kế**
STR	Sức mạnh thể chất; hỗ trợ sát thương cận chiến, tăng hiệu quả các hành động thiên về "đè lực".
DEX	Sự nhanh nhẹn; hỗ trợ tốc độ, độ linh hoạt (dash/dodge), chí mạng và các hướng build cơ động.
CON	Thể lực; tăng sống sót (HP), giảm rủi ro bị "bốc hơi", giúp cân bằng giữa tấn công và phòng thủ.
INT	Trí tuệ; hỗ trợ sát thương phép/khả năng kiểm soát, tương tác với cơ chế kỹ năng và hiệu ứng.
WIS	Minh triết; hỗ trợ tài nguyên (MP), hồi phục/kháng hiệu ứng và các vai trò hỗ trợ.
CHA	Sức hút/khả năng dẫn dắt; dùng làm biến điều kiện cho mở khoá, phối hợp đội hình và một số hướng build đặc thù.

Bảng 2: Ví dụ quan hệ phụ thuộc Secondary Stats từ Primary Attributes

Chỉ số suy diễn	**Phụ thuộc chính**
HP (máu tối đa)	CON, STR
MP (năng lượng kỹ năng)	WIS, INT
PATK/DATK (tấn công vật lý)	STR, DEX, CON
MATK/MDEF (tấn công/phòng thủ phép)	INT, WIS
SPD (tốc độ)	DEX
CRIT (tỉ lệ chí mạng)	DEX
ACC (độ chính xác)	DEX, INT
EVA (né tránh)	DEX, CON

Từ góc độ triển khai, các công thức nên được cấu hình hoá (hệ số có thể thay đổi), ví dụ:

$$HP_{\max} = HP_0 + a \cdot CON + b \cdot STR, \quad MP_{\max} = MP_0 + c \cdot WIS + d \cdot INT$$

$$PATK = p_1 \cdot STR + p_2 \cdot DEX + p_3 \cdot CON, \quad SPD = s_1 \cdot DEX$$

Việc tách *hệ số* khỏi logic cho phép cân bằng mà không cần thay đổi mã nguồn, đồng thời hỗ trợ thử nghiệm nhiều "đường cong sức mạnh" (linear/exponential) ở giai đoạn tinh chỉnh[4].

Nhóm trait đặc biệt (Special Traits) và ý nghĩa thiết kế Bên cạnh hệ 6 thuộc tính, thiết kế tham chiếu còn đưa vào nhóm trait đặc biệt nhằm tăng chiều sâu meta-progression:

- **Karma**: trục lựa chọn/định hướng hành vi, có thể ảnh hưởng điều kiện mở khoá class và nội dung.

- **Affinity**: liên kết nguyên tố/hệ (Light/Dark/Nature/...), dùng cho phân nhánh class/kỹ năng và kháng.
- **Luck**: tăng biến thiên theo hướng tích cực (drop, tỉ lệ chí mạng, các cơ chế RNG), nhưng cần giới hạn để tránh phá cân bằng.
- **Resistance**: sức đề kháng/kháng hiệu ứng, giảm cảm giác bị "khống chế liên hoàn" trong combat.

Nhóm trait này giúp hệ thống mở rộng theo hướng "điều kiện mở khoá" thay vì chỉ tăng số, tạo thêm mục tiêu dài hạn cho người chơi.

Leveling và mở slot kỹ năng Một lựa chọn thiết kế quan trọng là **level không trực tiếp cấp kỹ năng**, mà cấp **tài nguyên build** (điểm cộng chỉ số và slot để trang bị kỹ năng). Cách tiếp cận này tạo ra khác biệt giữa:

- **Học kỹ năng** (thu thập từ sách, nhiệm vụ, phần thưởng trận, ...).
- **Trang bị kỹ năng** (bị giới hạn bởi slot, buộc người chơi ra quyết định).

Trong kịch bản tham chiếu mở rộng của đề tài: level tối đa 100; mỗi level cho +5 điểm chỉ số để phân bổ; các mốc level mở dần slot kỹ năng thường và slot kỹ năng kết hợp (combo).

Bảng 3: Ví dụ mốc level và phần thưởng mở slot kỹ năng

Mốc	**Phần thưởng tiến trình**
10, 20, 30	Mở dần các slot kỹ năng thường (slot 1–3), giúp người chơi hình thành bộ kỹ năng cơ bản.
40	Mở slot kỹ năng combo đầu tiên, bắt đầu giai đoạn "xây dựng lối chơi".
50	Mốc nội dung/boss theo tuyến truyện và nới giới hạn stat cap (định hướng power spike).
60, 80, 100	Mở thêm các slot combo (2–4), hướng tới các tổ hợp mạnh và "ultimate combo".
70, 90	Mở thêm slot thường (slot 4–5), tăng tính linh hoạt của bộ kỹ năng.

Kỹ năng kết hợp (Combo Skills) như cơ chế đa dạng hoá build Combo skill là kỹ năng hình thành từ việc kết hợp ít nhất 2 kỹ năng đã học (cơ bản hoặc theo class). Từ góc độ Game Design, combo skill tạo ra:

- **Động lực sưu tầm kỹ năng**: người chơi có lý do tìm thêm kỹ năng ngoài "meta" hiện tại.
- **Không gian thử nghiệm**: cùng một level nhưng khác bộ kỹ năng vẫn tạo trải nghiệm khác biệt.

- **Đòn bẩy endgame**: ở level cao, số slot combo tăng, cho phép build chuyên sâu.

Tuy nhiên, combo skill cũng là rủi ro cân bằng: cần giới hạn điều kiện trang bị (slot), chi phí tài nguyên (MP/stamina), và ràng buộc cooldown để tránh "một combo thống trị".

Class và cơ chế mở khoá theo điều kiện Hệ class là lớp meta-progression giúp người chơi định hình vai trò và phong cách chơi. Một mô hình thường gặp:

- **Class** cung cấp kỹ năng nghề nghiệp, cơ chế chiến đấu đặc thù và nhánh thăng tiến (định hình lối chơi).

Trong thiết kế tham chiếu, class được phân tầng (Basic/Intermediate/Advanced/Legendary/Hidden) với điều kiện mở khoá dạng **ngưỡng chỉ số** (STR/DEX/INT/...), **trait đặc biệt** (Karma/Affinity/Luck/Resistance) và **thành tựu hành vi** (đối kháng, mốc tiến trình, ...). Cách thiết kế này biến "mở class" thành mục tiêu gameplay thay vì chỉ là lựa chọn menu, đồng thời cho phép hệ thống nội dung gắn với điều kiện mở khoá.

2.3.3 Combat Design (Thiết kế chiến đấu)

Combat là trung tâm trải nghiệm của casual strategy; mọi hệ thống tiến trình cuối cùng đều "đổ" vào cảm giác chiến đấu[4, 5]. Vì vậy, thiết kế combat cần đồng thời kiểm soát nhịp độ, độ rõ ràng và độ công bằng.

Nhịp độ (Pacing) và cửa sổ rủi ro Pacing được tạo bởi tốc độ animation, thời gian cooldown, khoảng cách hiệu lực và mức "trừng phạt" khi người chơi mắc lỗi. Một thiết kế tốt thường đảm bảo:

- Có hành động nhanh (đánh thường/dash) để duy trì nhịp.
- Có hành động mạnh nhưng rủi ro (kỹ năng nặng, hồi chiêu dài) để tạo quyết định.
- Có cơ chế phòng thủ/cơ động (dodge/i-frame) nhưng bị giới hạn tài nguyên để tránh lạm dụng.

Tương tác giữa chỉ số và cảm giác điều khiển Khi chỉ số ảnh hưởng đến combat, cần tránh để "chỉ số thay thế kỹ năng người chơi". Ví dụ:

- Tốc độ (SPD) và né tránh (EVA) nên tăng theo ngưỡng hợp lý để không biến combat thành "không thể trúng".
- Chính xác (ACC) cần có vai trò đối trọng với EVA để cân bằng PvE/đối kháng.
- HP/Resistance tăng sống sót nhưng cần đi kèm trade-off (mất sát thương hoặc giảm cơ động).

Các nguyên tắc này liên quan trực tiếp tới thiết kế power budget và TTK trong phần cân bằng.

2.3.4 World System và Level Design

World/Level design xác định cấu trúc không gian và cách nội dung được phân phối: luồng di chuyển, mật độ giao tranh, checkpoint, nhịp nghỉ[5]. Với casual strategy online, world design còn liên quan tới:

- **Tổ chức phiên chơi**: nội dung dạng phòng (room) hoặc khu (zone); ảnh hưởng tới cách đồng bộ nhiều phiên.
- **Điểm neo tiến trình**: checkpoint và mốc nội dung để tạo cảm giác "đi được một đoạn" thay vì cày vô hạn.
- **Phân tầng độ khó**: đảm bảo nội dung "vừa tầm" theo tiến trình, giảm cảm giác bế tắc.

Các nguyên lý này thường được kết hợp với progression (mốc level, mở slot, mở class) để tạo ra đường cong trải nghiệm liền mạch.

2.3.5 Hệ thống kinh tế (Game Economy)

Kinh tế trong game mô tả luồng tạo và tiêu thụ tài nguyên[4]. Về nguyên tắc, luôn cần đồng thời:

- **Nguồn (sources)**: quà nhiệm vụ, rơi vật phẩm, phần thưởng theo phiên, sự kiện.
- **Hút (sinks)**: nâng cấp trang bị, chế tạo, tiêu hao vật phẩm, phí sửa chữa/duy trì, các cơ chế RNG.

Thiết kế kinh tế tốt đảm bảo người chơi luôn có mục tiêu chi tiêu hợp lý, tránh tích luỹ vô hạn hoặc cạn kiệt quá nhanh.

2.3.6 UX/UI Design trong game và định hướng phong cách Pixel Art

UX/UI trong game có nhiệm vụ: truyền tải trạng thái, hỗ trợ ra quyết định nhanh và tạo phản hồi nhất quán[5]. Với định hướng đồ hoạ **pixel art**, các nguyên tắc UI cần nhấn mạnh tính **đọc được** (readability) và **rõ trạng thái**:

- **Lưới pixel và căn chỉnh**: UI nên bám lưới, viền/thanh máu dùng bội số pixel để tránh mờ/nhòe.
- **Tương phản cao**: chữ/icon cần tương phản nền đủ lớn; hạn chế gradient mịn gây giảm độ sắc.
- **Phân lớp thông tin**: trong combat chỉ hiển thị thông tin thiết yếu (HP/MP/cooldown/ buff-debuff), thông tin chi tiết chuyển sang màn hình nhân vật.
- **Phản hồi hành động**: tấn công trúng, bị trúng, nhặt vật phẩm, nâng cấp thành công/ thất bại cần có phản hồi hình–âm rõ ràng, nhất quán.

Ngoài ra, một bộ UI tối thiểu cho casual strategy online thường bao gồm:

- HUD chiến đấu: HP/MP, thanh kỹ năng theo slot, trạng thái cooldown, buff/debuff.
- Màn hình nhân vật: chỉ số, phân bổ điểm, class và trait đặc biệt.
- Inventory/Equipment: quản lý slot, trang bị, vật phẩm tiêu hao.
- Menu xã hội/cơ bản: bạn bè, party (nếu có), thiết lập (âm lượng, ngôn ngữ).

2.3.7 Balancing Fundamentals (Cân bằng game)

Cân bằng là quá trình điều chỉnh thông số để đảm bảo trải nghiệm hợp lý[4, 5]. Với casual strategy có hệ chỉ số và slot kỹ năng, các trục cân bằng thường gặp:

- **Tăng trưởng tuyến tính vs. luỹ thừa**: quyết định tốc độ mạnh lên theo thời gian và "độ chênh" giữa người chơi.
- **Time-to-Kill (TTK)**: thời gian hạ mục tiêu; quá thấp gây "bốc hơi", quá cao gây lê thê.
- **Risk–Reward**: nội dung rủi ro cao phải thưởng tương xứng; nếu không sẽ bị bỏ qua.
- **Power Budget**: tổng sức mạnh được phân bổ qua chỉ số, trang bị, kỹ năng; cần kiểm soát để combo/skill không vượt ngân sách.

Trong bối cảnh có combo skill và điều kiện mở class theo trait, cân bằng cần chú ý thêm:

- Giới hạn khả năng "stack" (xếp chồng) các lợi thế: chí mạng + tốc độ + né tránh + kháng hiệu ứng.
- Đảm bảo nhiều hướng build đều có "đất diễn": build cơ động mạnh nhưng mỏng; build trâu bò sống lâu nhưng thiếu bùng nổ; build phép mạnh nhưng phụ thuộc tài nguyên.

2.4 Tổng kết chương

Chương này đã trình bày các kiến thức nền tảng phục vụ thiết kế casual strategy online: đặc trưng theo phiên và đồng bộ nhiều phiên; các khái niệm game design như core loop, progression, combat, economy, UX/UI và balancing[4, 5, 2, 10]. Đồng thời, chương cũng mô hình hoá hệ chỉ số 6 thuộc tính, nhóm trait đặc biệt và cơ chế tiến trình thông qua mở slot kỹ năng/combo. Đây là cơ sở để chuyển sang phân tích yêu cầu (Chương 5) và đặc tả thiết kế hệ thống (Chương 7).

3 Công nghệ sử dụng

Chương này trình bày các công nghệ và nền tảng phần mềm **đang sử dụng** trong đồ án để xây dựng trò chơi casual strategy trực tuyến. Nội dung tập trung vào vai trò của từng công nghệ trong kiến trúc tổng thể, các ưu điểm/hạn chế và lý do phù hợp với bối cảnh hệ thống game client–server hiện tại[3, 2].

Về mặt chức năng, hệ thống có thể chia thành các lớp chính:

- **Client/Game Engine**: hiển thị đồ hoạ, nhận input, mô phỏng cục bộ (visual/animation), UI và giao tiếp mạng.
- **Backend/Application Server**: xử lý nghiệp vụ game, xác thực, quản lý phiên, đồng bộ trạng thái và cung cấp API.
- **Data & Storage**: lưu trữ lâu dài (progress người chơi), cache/pub-sub, và kho dữ liệu cấu hình/tài nguyên.
- **Tooling & Process**: quản lý mã nguồn, quản lý tác vụ, tài liệu hoá và công cụ thiết kế.

3.1 Công nghệ phía client

3.1.1 Game engine: Unity

Unity là lựa chọn phù hợp để hiện thực client trong bối cảnh đồ án nhờ hệ sinh thái hoàn chỉnh và tài liệu phong phú[6]. Các lý do chính:

- **Pipeline đồ hoạ và nội dung**: hỗ trợ sprite/animation, quản lý scene, prefab và asset import giúp tăng tốc phát triển nội dung.
- **UI framework**: cung cấp hệ thống UI phục vụ HUD chiến đấu, inventory, menu và các màn hình meta-progression.
- **Khả năng mở rộng**: cho phép tổ chức mã theo hướng component-based; có thể mở rộng sang hướng ECS tuỳ mức độ phức tạp[3].

Trong phạm vi hiện tại, Unity là nền tảng client để triển khai UI/gameplay và kiểm chứng contract giao tiếp với backend; tối ưu hiệu năng production sâu vẫn là mục tiêu ở giai đoạn sau.

3.1.2 Ngôn ngữ lập trình: C#

C# là ngôn ngữ chính khi phát triển với Unity[11]. Các đặc điểm phù hợp:

- Hỗ trợ tốt lập trình hướng đối tượng và tổ chức hệ thống gameplay theo mô-đun.
- Có thư viện tiêu chuẩn mạnh cho cấu trúc dữ liệu, toán học và xử lý bất đồng bộ ở mức client.
- Tích hợp trực tiếp với API Unity để điều khiển animation, UI và xử lý input[6].

3.1.3 Nguyên tắc xử lý logic ở client

Đối với game online, cần tách bạch rõ giữa:

- **Client logic (hiển thị)**: UI, animation, hiệu ứng, nội suy chuyển động và dự đoán phản hồi để giảm cảm giác trễ.
- **Server logic (quyết định)**: các kết quả quan trọng như tính sát thương, xác nhận loot, cập nhật tài nguyên và thay đổi trạng thái chính thức[2].

Nguyên tắc này giúp giảm rủi ro gian lận và đảm bảo tính nhất quán dữ liệu giữa các phiên chơi.

3.2 Công nghệ phía server

3.2.1 Nền tảng runtime: Node.js

Node.js phù hợp với backend game online nhờ mô hình I/O bất đồng bộ, thuận lợi khi phải xử lý đồng thời nhiều request và tác vụ nền[12]. Với đặc thù đồ án, Node.js hỗ trợ:

- Xây dựng API nhanh, dễ mở rộng theo mô-đun.
- Tích hợp tốt với hệ sinh thái thư viện cho xác thực, logging, validation và kết nối DB.

3.2.2 Framework: NestJS và TypeScript

NestJS cung cấp kiến trúc rõ ràng theo hướng module + dependency injection, phù hợp khi hệ thống cần phân rã theo phân hệ (auth, player state, tutorial progress, quests, battles, inventory, config, ...)[7]. TypeScript giúp tăng độ an toàn của mã nguồn nhờ kiểu tường minh, giảm lỗi trong phát triển trung hạn.

Các lý do lựa chọn:

- **Tổ chức mã nguồn**: module hoá tự nhiên, dễ quản lý ranh giới nghiệp vụ và tái sử dụng service dùng chung.
- **Hỗ trợ API rõ ràng**: REST đang vận hành ổn định cho các luồng gameplay/lobby cốt lõi[7].
- **Khả năng mở rộng**: có thể bắt đầu theo kiểu monolithic-module để giảm độ phức tạp, và có cơ sở để tách dịch vụ khi hệ thống tăng quy mô.

3.2.3 Tác vụ nền (background jobs)

Trong game online, một số xử lý không cần chạy trực tiếp trên request gameplay, ví dụ: đồng bộ dữ liệu định kỳ, tổng hợp log, xử lý mail hệ thống, hoặc quản lý sự kiện theo lịch. Trong bản hiện tại, phần job worker được tổ chức ở mức nền tảng và có thể kích hoạt dần theo nhu cầu vận hành để giảm tải cho API server.

3.3 Công nghệ giao tiếp mạng

3.3.1 HTTP/REST

HTTP phù hợp với các tác vụ không yêu cầu độ trễ thấp hoặc không cần kết nối hai chiều liên tục:

- Đăng ký/đăng nhập/refresh token, tải player state từ `/me`.
- Đồng bộ quest, tutorial progress, inventory hero upgrade/evolve.
- Khởi tạo/kết thúc battle session, nhận reward và cập nhật progression.
- Tải catalog shop, purchase offer và đồng bộ currency/reward popup.
- Tải dữ liệu cấu hình và truy vấn trạng thái phục vụ vận hành[7].

3.4 Công nghệ lưu trữ dữ liệu

3.4.1 MongoDB

MongoDB phù hợp cho dữ liệu game do đặc tính document linh hoạt, dễ biểu diễn các cấu trúc lồng nhau (inventory, quest tracks, formation, battle metadata) và thích nghi khi schema thay đổi trong quá trình phát triển[8]. Trong bối cảnh đồ án:

- **Dữ liệu tiến trình (dynamic)**: user/session/profile/statistics/tutorial/currency.
- inventory/formation/quests/battles lưu bền vững và truy vấn theo khoá định danh.
- **Dữ liệu cấu hình (static)**: hero/quest/rank/upgrade/account-level/campaign.
- các collection static được import từ pipeline TSV và nạp vào bộ nhớ khi chạy.

3.4.2 Redis

Redis được sử dụng trước hết cho hiệu năng (cache); các vai trò phối hợp giữa nhiều tiến trình được mở theo nhu cầu triển khai[9]:

- **Cache**: giảm tải đọc DB cho dữ liệu truy cập thường xuyên (ví dụ session mapping, một phần config hot, trạng thái online).
- **Pub/Sub**: chia sẻ sự kiện giữa nhiều tiến trình/instance backend khi mở rộng theo chiều ngang.

3.4.3 Object Storage (tuỳ chọn mở rộng)

Một số dữ liệu dạng file (asset pack, snapshot log, hoặc các tệp cấu hình đóng gói) có thể lưu trong object storage theo chuẩn S3-compatible. Trong phạm vi hiện tại, phần này được chuẩn bị ở mức kiến trúc để sẵn sàng mở rộng; chưa đặt trọng tâm vận hành production.

3.5 Công cụ quản trị game configuration theo hướng data-driven

Trong đồ án, **pipeline cấu hình** chỉ áp dụng cho **các bảng cấu hình/thông số game** (ví dụ: hero/quest/rank/upgrade và các hệ số cân bằng), không áp dụng cho toàn bộ dữ liệu tiến trình người chơi.

Quy trình đang áp dụng ở mức hiện tại:

- Thiết kế bảng cấu hình bằng **Google Sheets** để thuận lợi nhập liệu và review.
- Export sang **TSV** nhằm đảm bảo format đơn giản, dễ parse và thân thiện khi version control.
- Import TSV vào các **collections cấu hình** chính trong DB (tách biệt với collections progress).
- Khi khởi động hoặc khi tải game, hệ thống nạp config từ DB vào **Config Manager** (in-memory) để truy xuất thống nhất trong runtime.

Lợi ích chính của cách tiếp cận data-driven:

- Giảm phụ thuộc vào hard-code tham số, hỗ trợ điều chỉnh cân bằng nhanh.
- Tăng khả năng kiểm soát thay đổi: TSV có thể được quản lý theo commit để truy vết phiên bản cấu hình.
- Tạo tiền đề cho mở rộng nội dung và kiểm thử A/B cấu hình trong tương lai.

3.6 Công cụ hỗ trợ phát triển và tài liệu hoá

3.6.1 Quản lý mã nguồn và phối hợp nhóm

Git/GitHub được sử dụng để quản lý phiên bản mã nguồn và phối hợp theo pull request; GitHub Projects dùng để quản lý tác vụ theo bảng, gán trách nhiệm và theo dõi tiến độ[13, 14].

3.6.2 Công cụ mô hình hoá và tài liệu

Các sơ đồ use case/luồng nghiệp vụ/kiến trúc có thể được xây dựng bằng công cụ UML/ BPMN hoặc Mermaid để đảm bảo tính nhất quán khi trình bày[15]. Báo cáo và tài liệu hệ thống được soạn thảo bằng LaTeX nhằm đảm bảo định dạng học thuật và quản lý nội dung theo từng chương độc lập[16].

3.7 Tổng kết

Chương này đã trình bày các công nghệ chính đang sử dụng cho đồ án: Unity và C# ở phía client[6, 11]; Node.js, NestJS và TypeScript ở phía server[12, 7]; MongoDB và Redis cho lớp dữ liệu[8, 9]; cùng các công cụ hỗ trợ phát triển và tài liệu hoá[13, 14, 16]. Ngoài ra, chương cũng nêu pipeline quản trị **game configuration** theo hướng data-driven đang dùng để vận hành cấu hình nhất quán giữa backend và client.

4 Các công trình liên quan

Chương này được cập nhật theo trạng thái hiện tại của *Fortress of the Fallen*: các nguồn tham khảo bên ngoài chủ yếu còn vai trò **bối cảnh ý tưởng ban đầu**, không còn là khuôn mẫu bám sát như giai đoạn định hình concept.

Hiện tại, hướng phát triển sản phẩm được quyết định chủ yếu bởi:

- phạm vi tính năng đã triển khai thực tế (auth, `/me`, quest, battle session, tutorial sync, upgrade, castle, shop);
- ràng buộc tích hợp client–backend và tính nhất quán dữ liệu runtime;
- ưu tiên kỹ thuật để giữ ổn định vận hành trước khi mở rộng content.

4.1 Cách dùng nguồn tham khảo ở thời điểm hiện tại

Để tránh lệch giữa báo cáo và thực tế triển khai, các nguồn tham khảo được dùng theo 2 lớp:

- **Lớp định hướng ý tưởng (mức thấp):** tham khảo thematic về loop và fantasy setting.
- **Lớp quyết định triển khai (mức cao):** tài liệu kỹ thuật về kiến trúc game client–server, backend và dữ liệu [5, 4, 3, 10, 2, 6, 7, 12, 8, 9].

4.2 Đối chiếu nguồn cảm hứng và mức tương đồng hiện tại

Bảng 4 phản ánh mức tương đồng giữa các nguồn cảm hứng ban đầu và bản game đang chạy.

Nguồn	Mức tương đồng	Ghi chú ở trạng thái hiện tại	Vai trò trong báo cáo
SAO (Aincrad) [17]	Thấp	Không bám theo cấu trúc leo tầng đặc thù; chỉ còn ảnh hưởng thematic về tiến trình theo mốc	Ý tưởng ban đầu
Overlord [18]	Thấp	Chỉ còn ảnh hưởng ở bối cảnh fantasy, không còn bám vào hệ vận hành cốt lõi	Ý tưởng ban đầu
Pick Me Up! [19]	Thấp	Chỉ còn ảnh hưởng ở nhịp progression theo mốc ngắn hạn	Ý tưởng ban đầu
Arcane Odyssey [20]	Thấp–Trung bình	Chỉ còn ảnh hưởng ở cảm giác combat nhanh; không sao chép loop hay cấu trúc mode	Tham khảo định hướng UX combat
Soul Knight Prequel [21]	Trung bình	Gần nhất ở loop phiên ngắn: vào trận → kết trận → nhận thưởng	Tham khảo loop gameplay
Clash of Clans [22]	Thấp–Trung bình	Chỉ còn ảnh hưởng ở ý tưởng progression dài hạn (castle upgrade), không phải base-building đầy đủ	Tham khảo meta-progression

Bảng 4: Mức tương đồng hiện tại giữa nguồn cảm hứng và sản phẩm

4.3 Nguồn tham khảo có ảnh hưởng trực tiếp đến triển khai

Khác với nhóm cảm hứng nội dung, nhóm tài liệu kỹ thuật dưới đây đang ảnh hưởng trực tiếp đến thiết kế và hiện thực:

- **Kiến trúc hệ thống game online:** phân tách client–server, authoritative state, tổ chức vòng lặp xử lý backend [3, 2, 10].
- **Stack triển khai:** Unity/C#, Node.js/NestJS, MongoDB, Redis [6, 7, 12, 8, 9].
- **Nguyên lý game design để kiểm tra tính nhất quán:** loop, progression, economy và UX ở mức framework tư duy [5, 4].

4.4 Kết luận chương

Ở phiên bản báo cáo hiện tại, Chương 4 không còn trình bày các ref như "mẫu thiết kế" cho sản phẩm. Thay vào đó:

- nhóm ref cảm hứng chỉ được giữ ở mức bối cảnh lịch sử hình thành ý tưởng;
- các quyết định hệ thống hiện tại ưu tiên theo evidence triển khai thực tế và tài liệu kỹ thuật.

Cách trình bày này giúp giảm mâu thuẫn giữa phần tham khảo và trạng thái game đang vận hành.

5 Phân tích yêu cầu

Chương này xác định yêu cầu cho hệ thống game *Fortress of the Fallen* theo phạm vi **đã triển khai thực tế**. Nội dung tập trung vào các luồng runtime đang hoạt động giữa Unity client và backend API.

5.1 Phạm vi và giả định

5.1.1 Phạm vi hiện tại

Các yêu cầu trong chương này chỉ bao gồm các phân hệ đã có runtime/API:

- xác thực và vòng đời phiên (register/login/refresh/logout);
- đồng bộ player state qua `GET /me`;
- quest sync và claim reward;
- battle session start/finish;
- tutorial progress sync;
- nâng cấp hero/skill;
- nâng cấp castle;
- shop catalog, purchase và popup reward.

5.1.2 Giả định

- Hệ thống vận hành theo mô hình client–server; backend là nguồn sự thật cho progression và reward.
- Các thao tác nhạy cảm (finish battle, claim, upgrade) cần contract idempotency để tránh cấp trùng.
- Dữ liệu hiển thị ở UI được cập nhật từ payload backend, không chốt local-only.

5.2 Tác nhân và ranh giới hệ thống

5.2.1 Tác nhân

- **Player**: người chơi thao tác qua Unity client.
- **Backend services**: cung cấp API và xác nhận kết quả nghiệp vụ.

5.2.2 Ranh giới hệ thống

- **Client**: UI, input, điều hướng màn hình, gọi API.
- **Server**: auth, session, battle result, quest, tutorial, progression, castle, shop.
- **Data layer**: lưu player state, battle session, quest state, tutorial flags, progression state.

5.3 Yêu cầu chức năng

Mỗi yêu cầu được mô tả theo cùng một format: **Mục tiêu – Điều kiện vào – Kết quả mong đợi – Bằng chứng triển khai**.

5.3.1 Danh sách yêu cầu chức năng hiện tại

FR-01 – Đăng ký tài khoản

- **Mục tiêu:** tạo tài khoản mới hợp lệ.
- **Điều kiện vào:** người chơi nhập username/password/confirmPassword/displayName hợp lệ.
- **Kết quả mong đợi:** backend tạo tài khoản và trả access token + refresh token + thông tin player.
- **Bằng chứng triển khai:** `POST /api/v1/auth/register`.

FR-02 – Đăng nhập và quản lý phiên

- **Mục tiêu:** cho phép vào game an toàn theo vòng đời token.
- **Điều kiện vào:** tài khoản hợp lệ, token refresh còn hiệu lực khi cần làm mới phiên.
- **Kết quả mong đợi:** login nhận token; refresh đổi token mới; logout revoke refresh token.
- **Bằng chứng triển khai:** `POST /api/v1/auth/login`, `/auth/refresh`, `/auth/logout`.

FR-03 – Đồng bộ player state

- **Mục tiêu:** nạp trạng thái người chơi nhất quán khi vào game.
- **Điều kiện vào:** access token hợp lệ.
- **Kết quả mong đợi:** trả các trường chính gồm profile, statistics, tutorialProgress, currency, inventory, formation, quests, configVersion.
- **Bằng chứng triển khai:** `GET /api/v1/me`.

FR-04 – Đồng bộ quest và nhận thưởng quest

- **Mục tiêu:** theo dõi tiến độ daily/weekly/achievement và claim reward theo backend.
- **Điều kiện vào:** người chơi đã đăng nhập; quest tồn tại và đủ điều kiện claim.
- **Kết quả mong đợi:** danh sách quest/tiến độ/rewardClaimed đồng bộ đúng với backend; claim cập nhật ngay state.
- **Bằng chứng triển khai:** `GET /api/v1/quests` và các endpoint claim quest/progress reward đã tích hợp trong client.

FR-05 – Khởi tạo battle session

- **Mục tiêu:** tạo phiên battle hợp lệ trước khi vào trận.
- **Điều kiện vào:** người chơi chọn mode/formation/config version hợp lệ.
- **Kết quả mong đợi:** backend trả `battleId`, mode, seed, configVersion, expiresAt (và dữ liệu đối thủ theo mode hiện hành).
- **Bằng chứng triển khai:** `POST /api/v1/battles/start`.

FR-06 – Kết thúc battle và chốt hậu trận

- **Mục tiêu:** chốt kết quả battle theo cơ chế server-authoritative.
- **Điều kiện vào:** có `battleId` hợp lệ và payload finish hợp lệ, kèm `idempotencyKey`.
- **Kết quả mong đợi:** backend trả reward, playerDelta, statistics, tutorialProgress, currency, questUpdates; client cập nhật state theo payload này.
- **Bằng chứng triển khai:** `POST/api/v1/battles/{battleId}/finish`.

FR-07 – Đồng bộ tutorial progress

- **Mục tiêu:** đảm bảo cờ tutorial nhất quán giữa các phiên/chuyển scene.
- **Điều kiện vào:** có `updates` hợp lệ và thời điểm cập nhật client.
- **Kết quả mong đợi:** backend xác nhận và trả tutorialProgress mới để client apply ngay.
- **Bằng chứng triển khai:** `PATCH /api/v1/me/tutorial-progress`.

FR-08 – Nâng cấp hero

- **Mục tiêu:** cho phép tăng tiến hero theo tài nguyên và rule backend.
- **Điều kiện vào:** hero tồn tại, đủ currency/material theo cấu hình.
- **Kết quả mong đợi:** backend trả trạng thái hero/currency mới; client cập nhật UI Units theo dữ liệu trả về.
- **Bằng chứng triển khai:** luồng upgrade hero đã tích hợp API trong runtime hiện tại.

FR-09 – Nâng cấp và mua skill

- **Mục tiêu:** quản lý tiến trình kỹ năng theo rule backend.
- **Điều kiện vào:** skill hợp lệ và đủ điều kiện tài nguyên.
- **Kết quả mong đợi:** backend trả skill/currency cập nhật; client hiển thị đúng trạng thái mới.
- **Bằng chứng triển khai:** luồng purchase/upgrade skill đã tích hợp API trong runtime hiện tại.

FR-10 – Nâng cấp castle

- **Mục tiêu:** đồng bộ castle progression qua backend, không chốt local-only.
- **Điều kiện vào:** người chơi đủ tài nguyên và điều kiện nâng cấp.
- **Kết quả mong đợi:** backend trả castle state/currency mới; client cập nhật UI Upgrade Castle theo response.
- **Bằng chứng triển khai:** endpoint `/api/v1/lobby/castle/upgrade` đã được tích hợp theo spec.

FR-11 – Mua shop offer và nhận reward

- **Mục tiêu:** cho phép người chơi xem catalog shop, mua offer và nhận thưởng theo rule backend.
- **Điều kiện vào:** người chơi đã đăng nhập; offer khả dụng; đủ currency theo loại giá (GO/GE).
- **Kết quả mong đợi:** client tải catalog từ backend, gửi purchase với idempotency key, nhận currency mới và reward để hiển thị popup.
- **Bằng chứng triển khai:**
  `GET/api/v1/shop/catalog`
  `POST/api/v1/shop/offers/{offerId}/purchase`

5.4 Yêu cầu phi chức năng

- **NFR-01 – Security:** mật khẩu phải lưu dạng hash; access/refresh token có hạn dùng và hỗ trợ revoke.
- **NFR-02 – Authority boundary:** reward, currency, statistics, quest progress, tutorial flags, castle progression và shop currency/reward phải lấy theo backend response.
- **NFR-03 – Idempotency:** finish battle, claim reward, shop purchase và upgrade operations phải chống cấp trùng khi retry.
- **NFR-04 – Consistency:** client không được ghi đè state mới bằng response cũ trong các luồng bất đồng bộ.
- **NFR-05 – Observability:** phải có log request và log nghiệp vụ đủ để truy vết battle/quest/tutorial/upgrade.
- **NFR-06 – UX state clarity:** UI cần thể hiện rõ trạng thái loading/success/error/disabled ở các thao tác gọi API.
- **NFR-07 – Maintainability:** tách lớp giao tiếp API và lớp điều phối nghiệp vụ để giảm phụ thuộc chéo.
- **NFR-08 – Version compatibility:** client gửi/đọc `configVersion` để đảm bảo gameplay chạy trên cấu hình hợp lệ.

5.5 Yêu cầu dữ liệu

5.5.1 Phân loại dữ liệu

Dữ liệu động (player state) Profile, statistics, tutorial progress, currency, inventory (heroes/skills), formation, quest state, battle session state, castle progression, shop state.

Dữ liệu cấu hình Manifest version và các bảng cấu hình gameplay mà backend dùng để validate/compute reward và progression.

5.5.2 Yêu cầu mô hình dữ liệu mức logic

- **DR-01** – Mỗi account có định danh player duy nhất và trạng thái auth/session tương ứng.
- **DR-02** – Snapshot player state từ `/me` phải gom nhất quán profile, statistics, tutorial, currency, inventory, formation, quests.
- **DR-03** – Quest state phải lưu được track daily/weekly/achievement, progress, required, rewardClaimed, progressRewards.

- **DR-04** – Battle session phải lưu được `battleId`, mode, seed, expiresAt và trạng thái finish theo idempotency key.
- **DR-05** – Progression data phải phản ánh được hero/skill upgrade, castle upgrade, shop purchase cùng biến động currency.
- **DR-06** – Các object có mốc thời gian cần trường cập nhật để giải quyết xung đột state (ví dụ: `updatedAt`, `serverTime`).

5.6 Tiêu chí chấp nhận theo trạng thái hiện tại

- Toàn bộ FR-01 đến FR-11 có thể đối chiếu trực tiếp với endpoint hoặc luồng runtime đã chạy.
- Không xuất hiện yêu cầu cho tính năng chưa có bằng chứng runtime/API trong bản hiện tại.
- NFR-01 đến NFR-08 phản ánh đúng mô hình server-authoritative và các ràng buộc nhất quán dữ liệu đang áp dụng.
- Dữ liệu yêu cầu ở DR-01 đến DR-06 đủ để truy vết các luồng auth/quest/battle/tutorial/upgrade/castle/shop.

6 Phân tích hệ thống

Chương này phân tích hệ thống ở mức logic cho phạm vi đã triển khai thực tế, nhằm chuyển hoá yêu cầu ở Chương 5 thành mô tả vận hành rõ ràng cho thiết kế và kiểm thử tích hợp.

6.1 Tổng quan phân hệ và ranh giới trách nhiệm

Hệ thống vận hành theo mô hình client–server:

- **Client**: hiển thị UI, thu thập input, gửi request và áp dụng state trả về.
- **Server**: xác thực, quản lý phiên, xử lý nghiệp vụ và chốt state authoritative.

Các phân hệ đang có runtime/API trong baseline:

- Auth/Session.
- Player State Snapshot (`/me`).
- Quest Sync/Claim.
- Battle Session Start/Finish.
- Tutorial Progress Sync.

- Hero/Skill Progression.
- Castle Progression.
- Shop Catalog/Purchase.

6.2 Tác nhân hệ thống

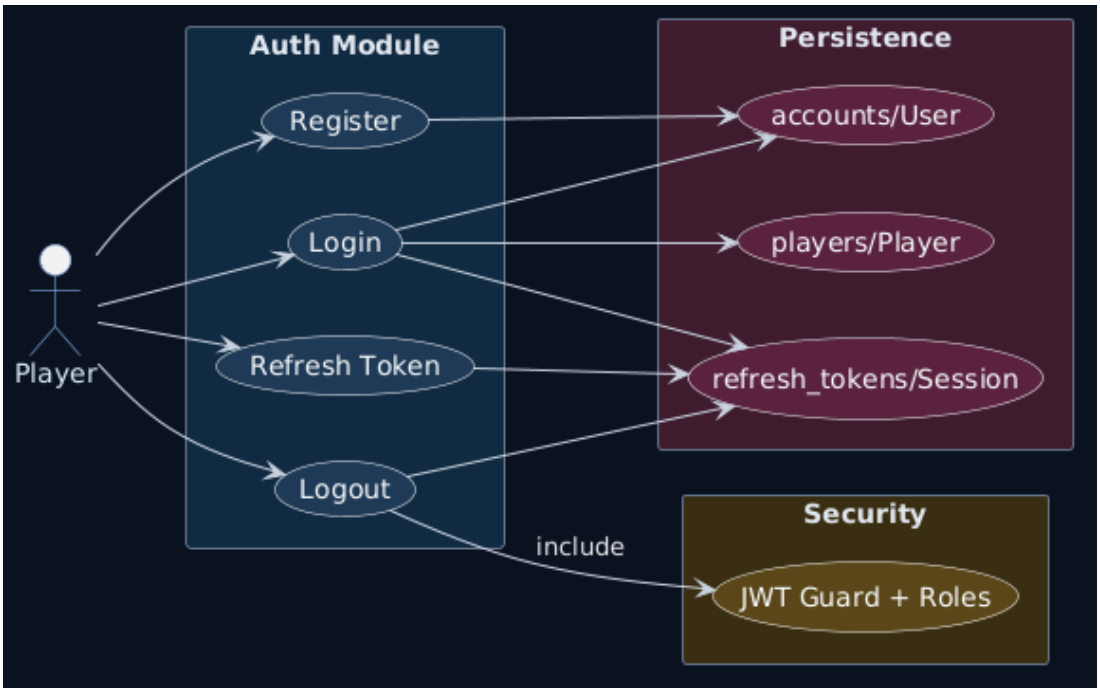
Tác nhân	**Mô tả**
Player	Người chơi thao tác trên game client để đăng nhập, chơi trận, làm quest và nâng cấp tiến trình.
Game Client	Ứng dụng Unity gửi HTTP request, nhận response và đồng bộ trạng thái hiển thị.
Backend Server	Thành phần xử lý auth, nghiệp vụ và cập nhật dữ liệu tiến trình theo server-authoritative.
Database	Lưu trạng thái người chơi và dữ liệu phục vụ nghiệp vụ runtime.

Bảng 5: Tác nhân và vai trò trong phạm vi triển khai hiện tại

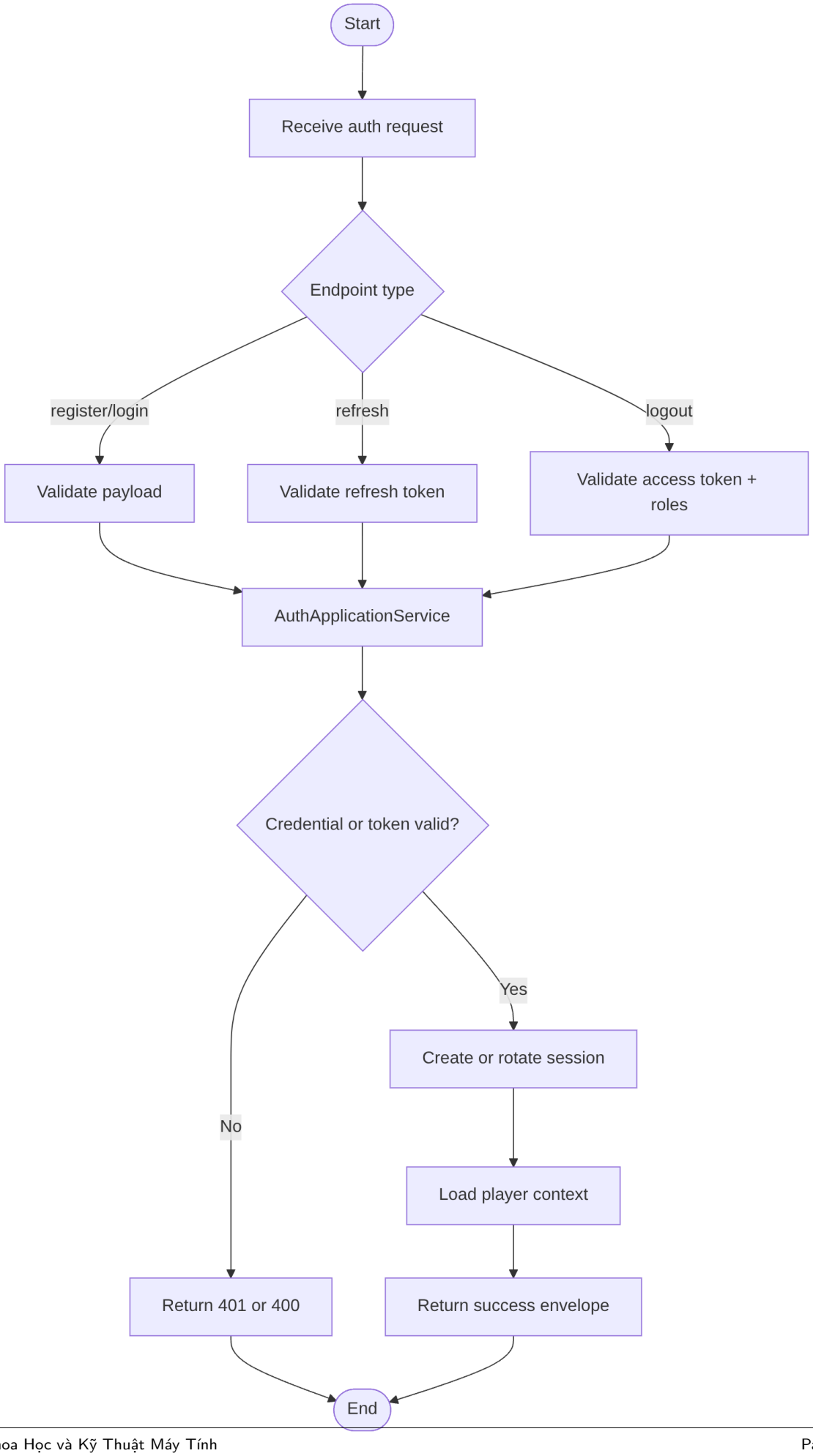
6.3 Phân tích use case theo phạm vi hiện tại

6.3.1 Nhóm xác thực và phiên

- Đăng ký tài khoản.
- Đăng nhập.
- Làm mới access token bằng refresh token.
- Đăng xuất và revoke refresh token.



Hình 3: Use case cho phân hệ xác thực và phiên


Start
Receive auth request
Endpoint type
register/login
refresh
logout
Validate payload
Validate refresh token
Validate access token + roles
AuthApplicationService
Credential or token valid?
Yes
No
Create or rotate session
Load player context
Return 401 or 400
Return success envelope
End

6.3.2 Nhóm đồng bộ trạng thái người chơi

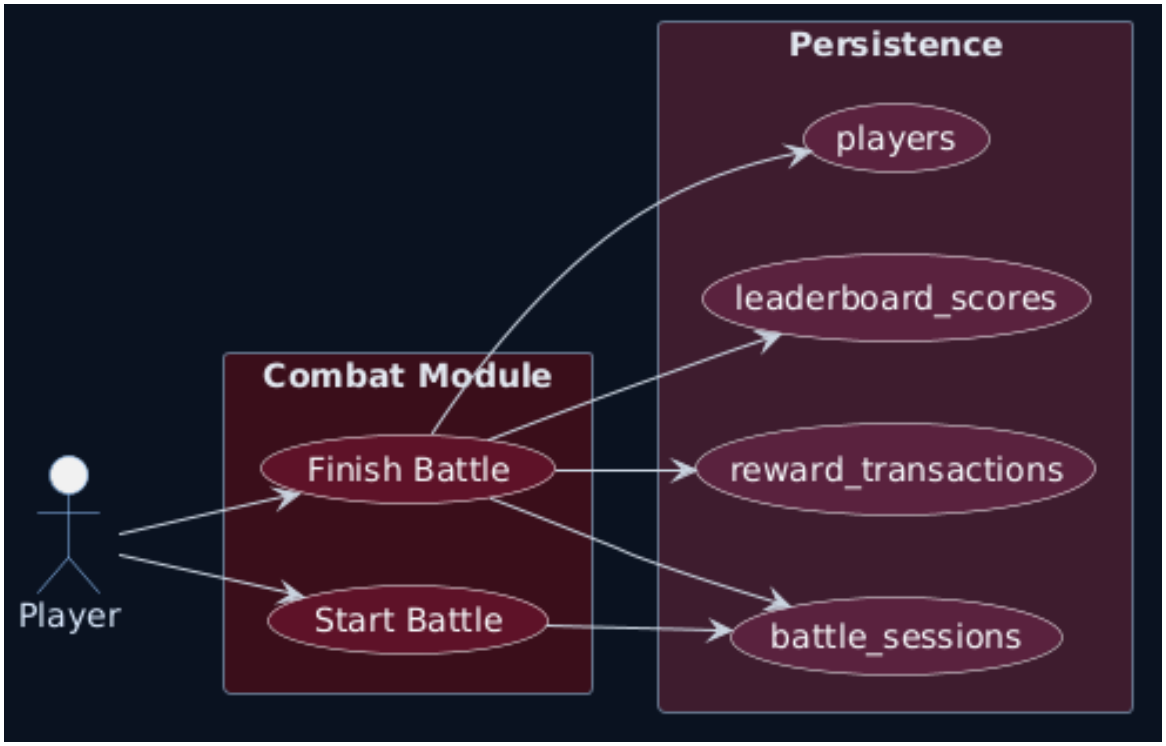
- Tải snapshot player state qua `/me`.
- Áp dụng profile, statistics, tutorialProgress, currency, inventory, formation, quests vào state client.

6.3.3 Nhóm quest

- Đồng bộ quest tracks (daily/weekly/achievement).
- Claim quest reward và progress reward theo điều kiện backend.

6.3.4 Nhóm battle session

- Khởi tạo phiên battle (`battleId`, seed, configVersion, expiresAt).
- Kết thúc phiên battle với idempotency key.
- Đồng bộ hậu trận: reward, statistics, currency, tutorialProgress, questUpdates.



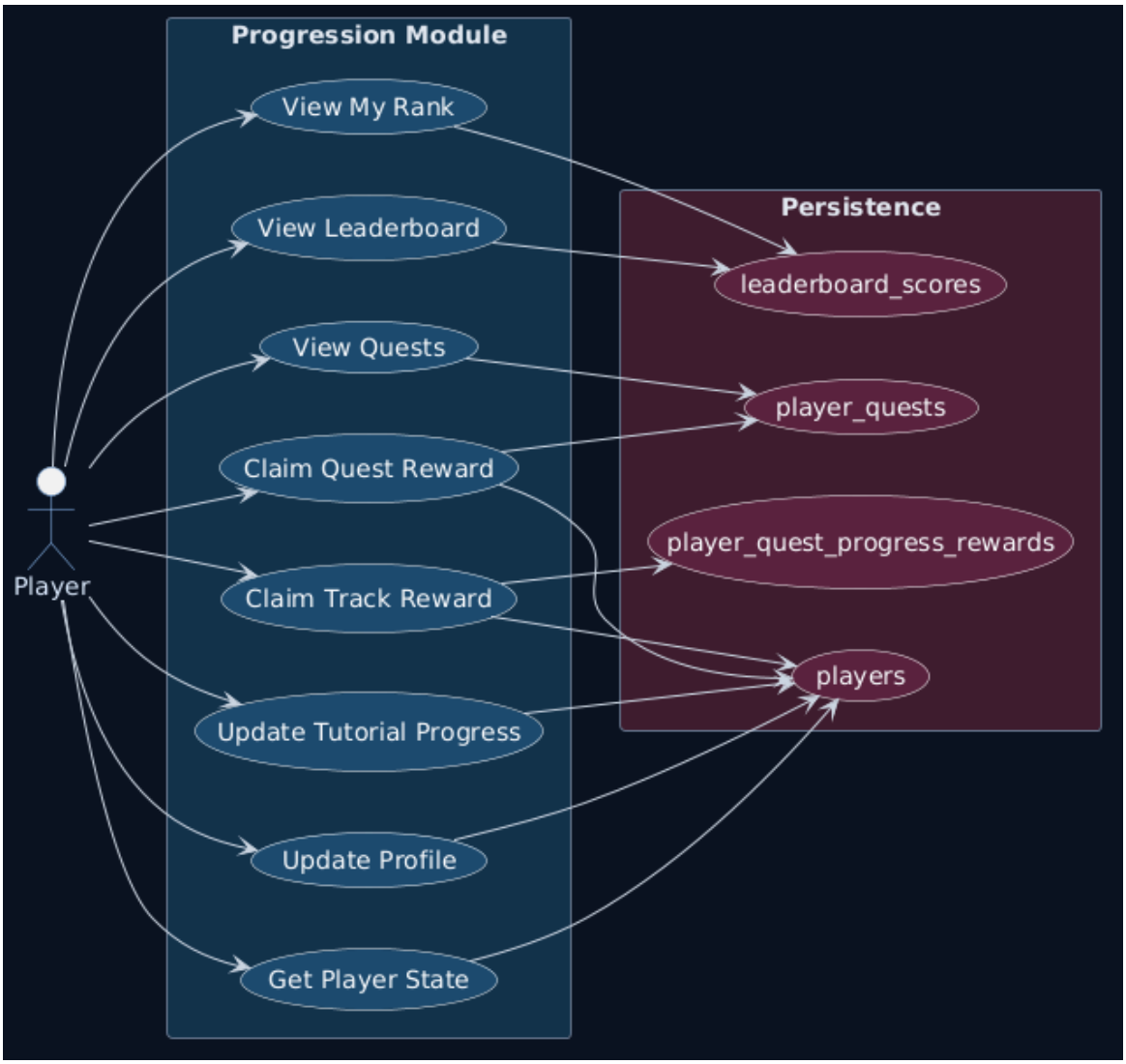
Hình 5: Use case cho phân hệ battle session

6.3.5 Nhóm tutorial progress

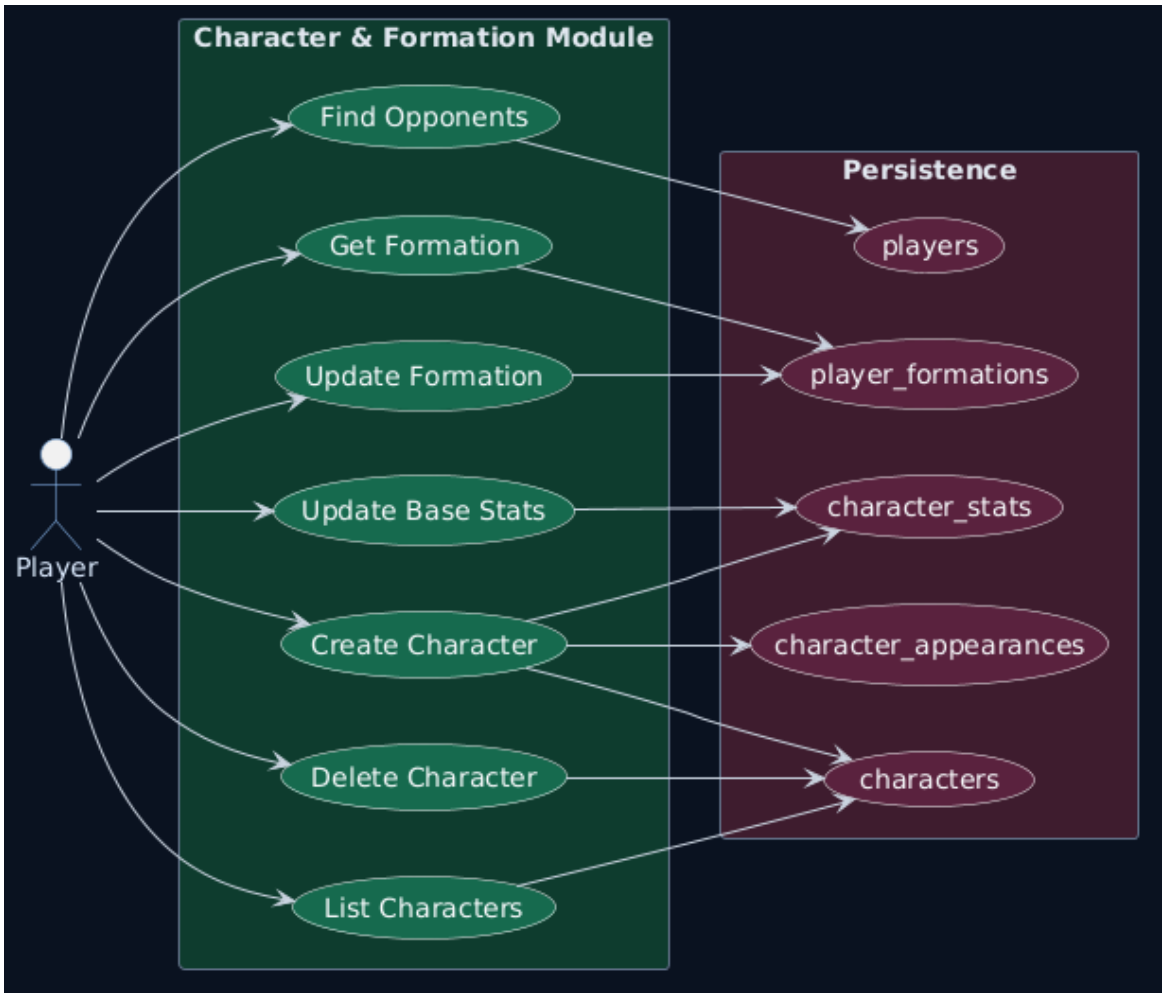
- Cập nhật cờ tutorial qua endpoint patch.
- Hợp nhất cờ tutorial từ nhiều nguồn response backend theo trạng thái mới nhất.

6.3.6 Nhóm progression thao tác trực tiếp

- Nâng cấp hero.
- Mua và nâng cấp skill.
- Nâng cấp castle qua endpoint backend chuyên biệt.
- Mua shop offer và nhận reward popup.



Hình 6: Use case cho phân hệ progression người chơi



Hình 7: Use case cho phân hệ character và formation

6.4 Đặc tả use case trọng yếu

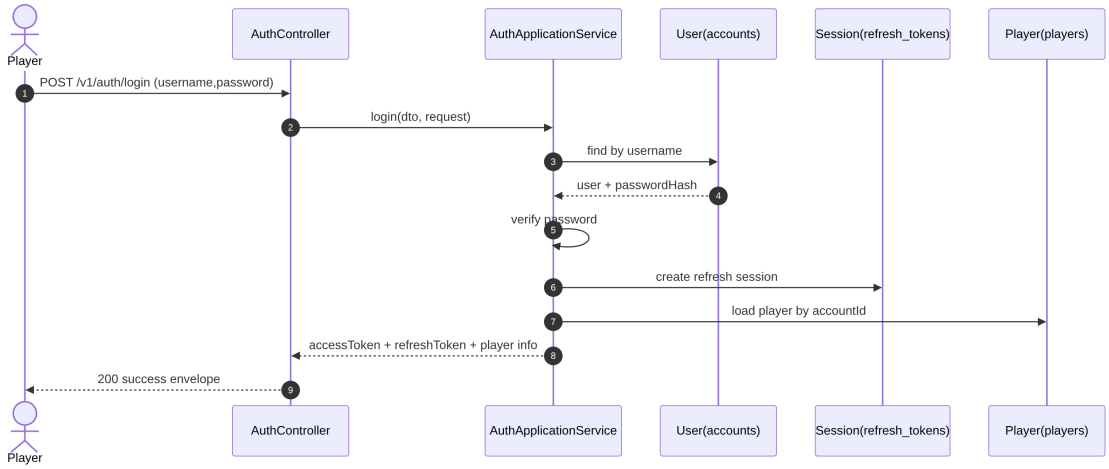
6.4.1 UC-01 – Đăng nhập và thiết lập phiên

Mục tiêu: xác thực người chơi và cấp token hợp lệ để truy cập API.
Tác nhân chính: Player.
Tiền điều kiện: tài khoản tồn tại và không bị khoá.
Hậu điều kiện: client có access/refresh token; có thể gọi `/me`.
Luồng chính:

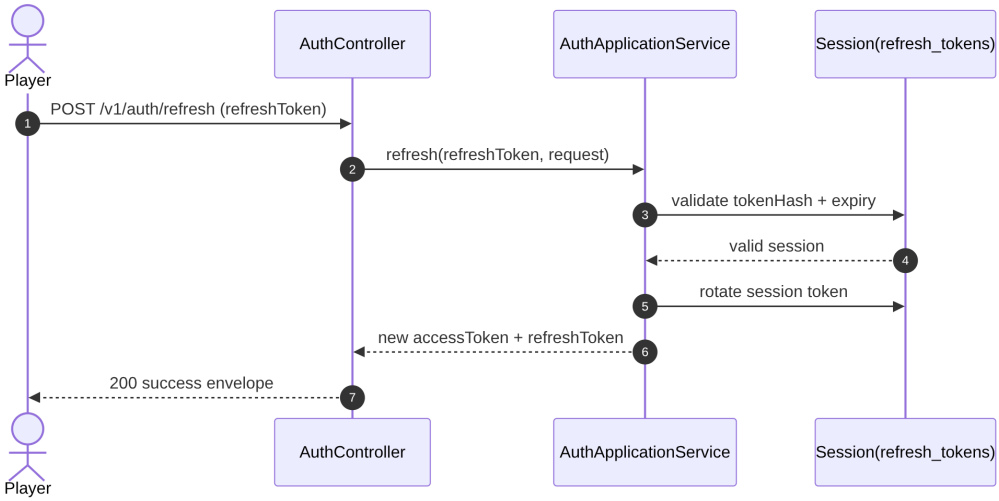
1. Player nhập thông tin đăng nhập.
2. Client gửi request login.
3. Server xác thực và trả token.
4. Client lưu token và chuyển sang đồng bộ state.

Ngoại lệ:

- Sai thông tin đăng nhập → trả lỗi xác thực.



Hình 8: Sequence đăng nhập và thiết lập phiên



Hình 9: Sequence làm mới access token

6.4.2 UC-02 – Đồng bộ player state

Mục tiêu: nạp trạng thái người chơi nhất quán sau khi xác thực.
Tác nhân chính: Game Client.
Tiền điều kiện: access token hợp lệ.
Hậu điều kiện: state local phản ánh đúng snapshot backend.
Luồng chính:

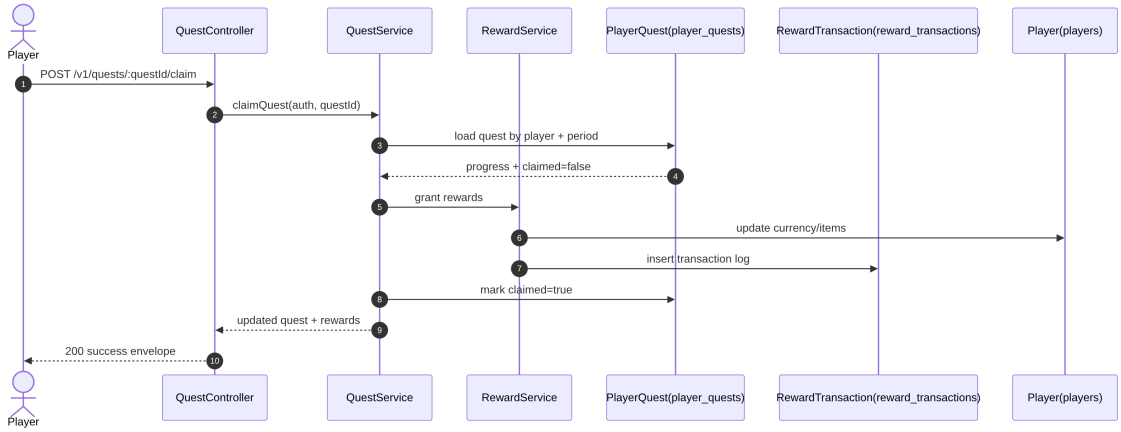
1. Client gọi `GET /me`.
2. Server trả profile, statistics, tutorialProgress, currency, inventory, formation, quests.
3. Client apply state theo thứ tự ưu tiên backend response.

6.4.3 UC-03 – Đồng bộ và claim quest

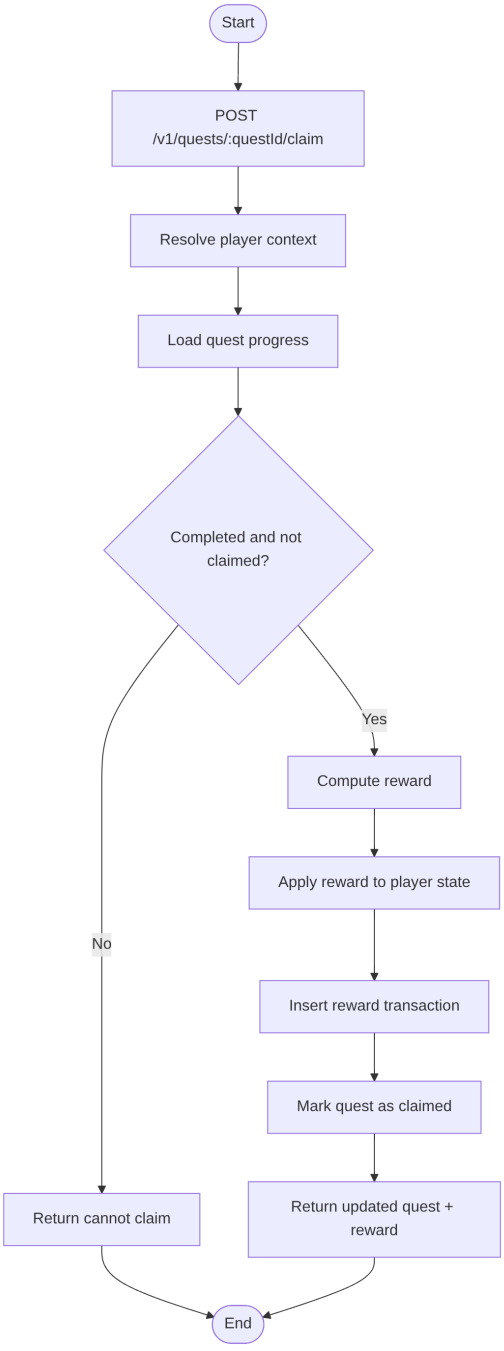
Mục tiêu: cập nhật tiến độ quest và nhận thưởng theo điều kiện backend.

Tác nhân chính: Player.
Tiền điều kiện: đã có snapshot state, quest tồn tại.
Hậu điều kiện: progress/rewardClaimed được cập nhật nhất quán.
Luồng chính:

1. Client gọi endpoint lấy quest.
2. Player thực hiện claim khi đủ điều kiện.
3. Server validate, cập nhật dữ liệu và trả trạng thái mới.
4. Client cập nhật UI quest theo payload trả về.



Hình 10: Sequence đồng bộ và claim quest



Hình 11: Activity flow cho luồng claim quest

6.4.4 UC-04 – Khởi tạo battle session

Mục tiêu: tạo phiên battle hợp lệ trước khi vào trận.
Tác nhân chính: Player.
Tiền điều kiện: đã đăng nhập, có formation hợp lệ.
Hậu điều kiện: nhận `battleId` và thông tin phiên để bắt đầu gameplay.
Luồng chính:

1. Client gửi request `battles/start` với mode, formationName, configVersion.

2. Server tạo battle session và trả dữ liệu phiên.

3. Client lưu `battleId` để dùng cho bước finish.

6.4.5 UC-05 – Kết thúc battle và đồng bộ hậu trận

Mục tiêu: chốt kết quả trận đấu theo backend source-of-truth.
Tác nhân chính: Player.
Tiền điều kiện: có `battleId` còn hiệu lực.
Hậu điều kiện: reward, statistics, currency, tutorialProgress, questUpdates được cập nhật.
Luồng chính:

1. Client gửi `battles/{battleId}/finish` kèm `idempotencyKey`.

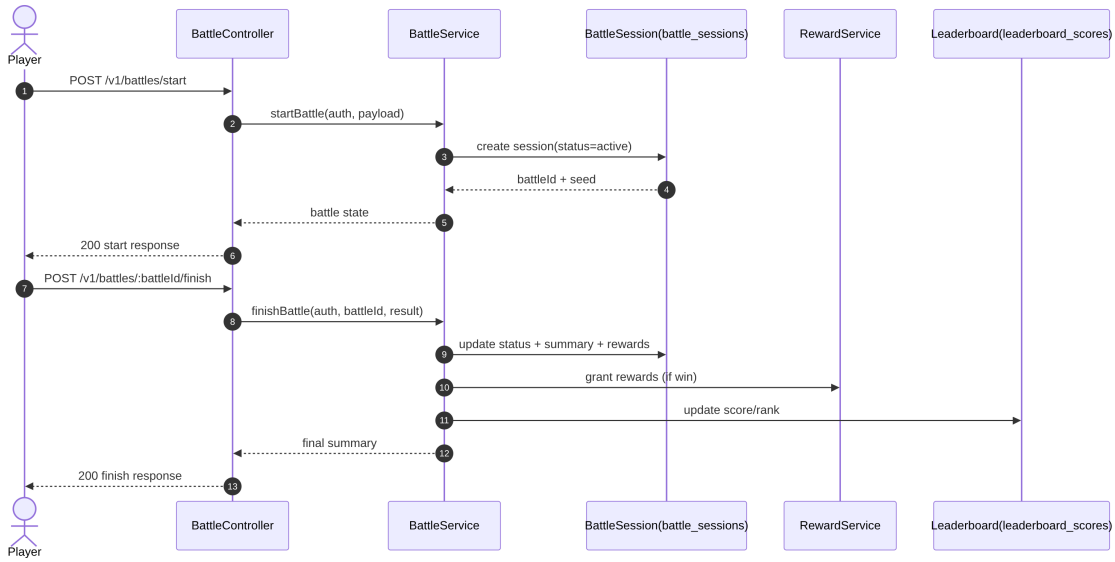
2. Server validate phiên và chốt kết quả.

3. Server trả payload hậu trận.

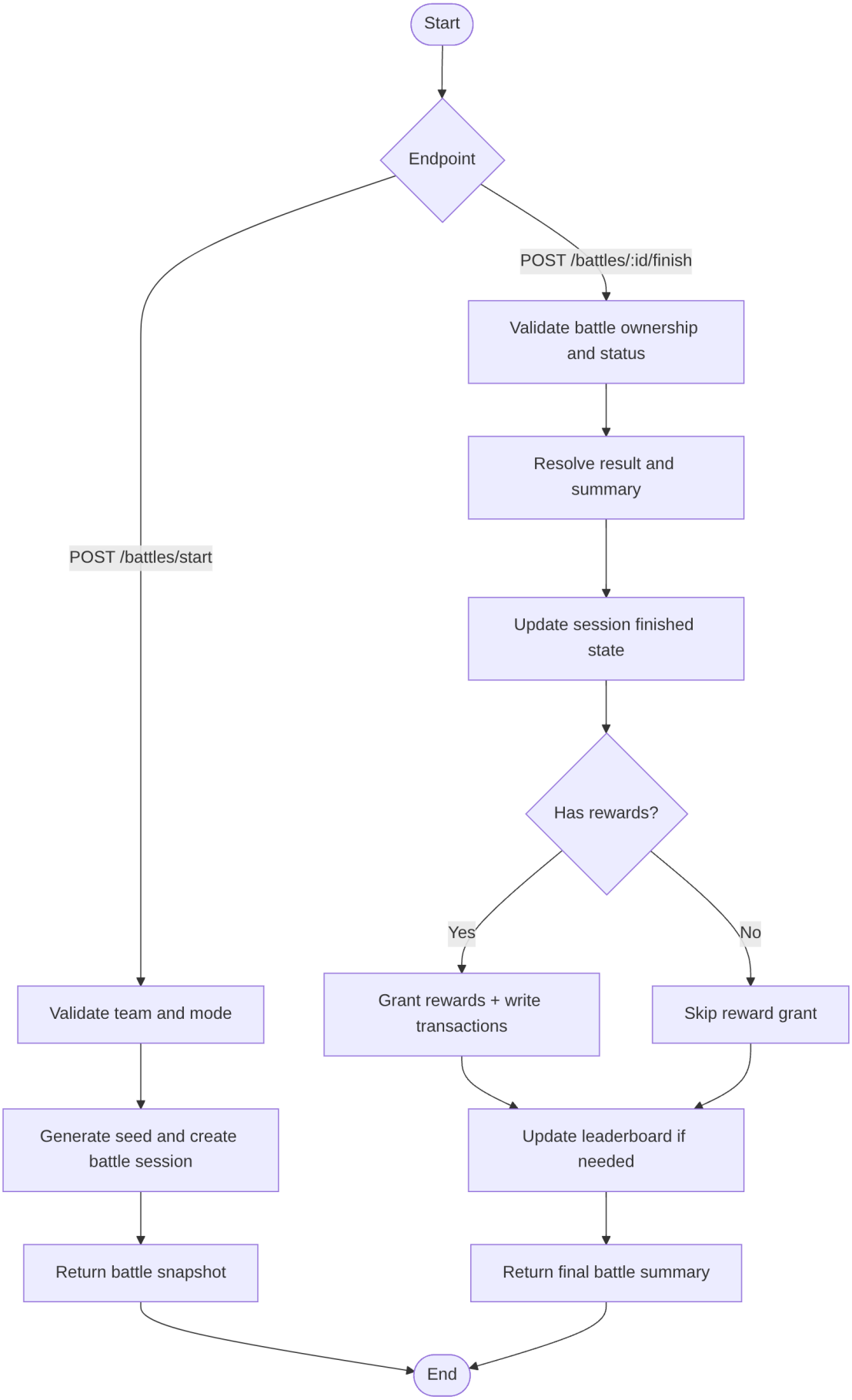
4. Client cập nhật state và UI theo payload này.

Ngoại lệ:

- Retry cùng idempotency key → backend không cấp trùng kết quả.



Hình 12: Sequence khởi tạo và kết thúc battle session



Hình 13: Activity flow cho vòng đời battle session

6.4.6 UC-06 – Đồng bộ tutorial progress

Mục tiêu: đồng bộ các cờ tutorial liên phiên.
Tác nhân chính: Game Client.
Tiền điều kiện: có thay đổi cờ tutorial từ gameplay.
Hậu điều kiện: backend lưu và trả tutorialProgress mới.
Luồng chính:

1. Client gửi `PATCH /me/tutorial-progress` với các trường cập nhật.
2. Server ghi nhận và trả trạng thái mới.
3. Client apply lại cờ tutorial theo response.

6.4.7 UC-07 – Nâng cấp hero và skill

Mục tiêu: cập nhật progression hero/skill theo rule backend.
Tác nhân chính: Player.
Tiền điều kiện: đủ tài nguyên và điều kiện nâng cấp.
Hậu điều kiện: dữ liệu hero/skill/currency được cập nhật đồng bộ.
Luồng chính:

1. Player thao tác nâng cấp trên UI Units hoặc UI kỹ năng.
2. Client gọi endpoint nâng cấp tương ứng.
3. Server validate điều kiện và trả trạng thái mới.
4. Client cập nhật lại dữ liệu hiển thị.

6.4.8 UC-08 – Mua shop offer và hiển thị phần thưởng

Mục tiêu: thực hiện giao dịch shop theo backend source-of-truth và hiển thị reward đúng payload trả về.
Tác nhân chính: Player.
Tiền điều kiện: người chơi đã đăng nhập; catalog shop tải thành công; offer còn khả dụng.
Hậu điều kiện: currency và trạng thái mua được cập nhật; popup reward hiển thị đúng danh sách item nhận được.
Luồng chính:

1. Client gọi `GET /shop/catalog` khi mở UIShop.
2. Người chơi chọn offer và bấm mua.
3. Client gửi `POST /shop/offers/{offerId}/purchase` kèm idempotency key.
4. Server validate giá/giới hạn mua, trừ currency và trả reward.
5. Client cập nhật UI shop theo catalog mới và mở popup reward.

6.4.9 UC-09 – Nâng cấp castle

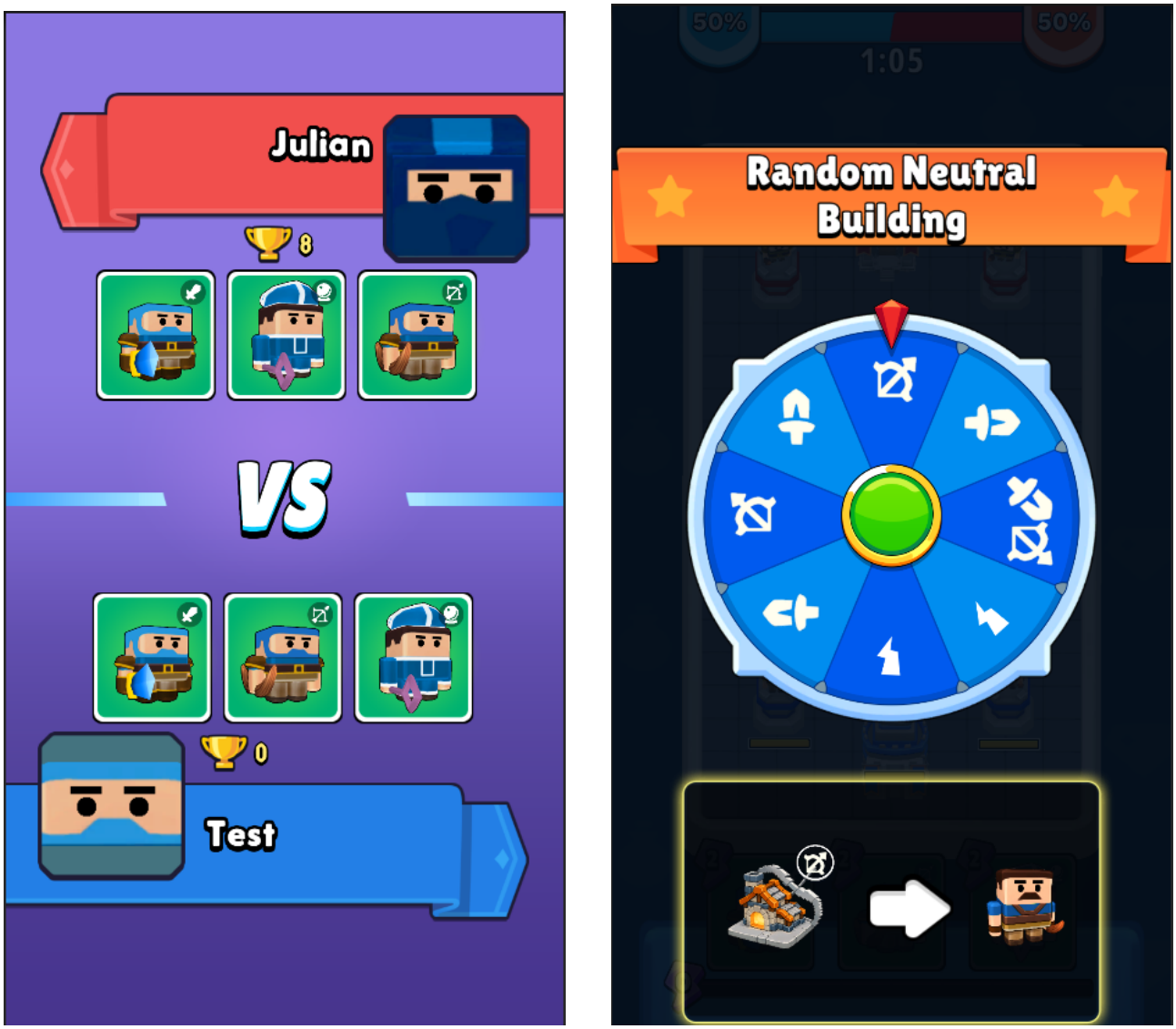
Mục tiêu: thực hiện nâng cấp castle bằng backend contract riêng.
Tác nhân chính: Player.
Tiền điều kiện: đủ tài nguyên và điều kiện nâng cấp castle.
Hậu điều kiện: castle progression và currency mới được trả về cho client.
Luồng chính:

1. Player bấm nâng cấp castle.
2. Client gọi endpoint nâng cấp castle.
3. Server validate và ghi nhận nâng cấp.
4. Client cập nhật UI theo response backend.

6.5 Hướng dẫn chơi & flow gameplay hiện tại

6.5.1 Hướng dẫn chơi nhanh

1. Đăng nhập/đăng ký, đồng bộ state người chơi qua `/me` và config cần thiết qua `/configs/manifest`.
2. Từ lobby, vào matching để tìm trận phù hợp với trạng thái hiện tại.
3. Đi qua màn roulette để xác nhận vào battle.
4. Vào battle session và thực hiện gameplay của trận.
5. Chốt trạng thái thắng/thua, sau đó nhận reward và đồng bộ hậu trận.
6. Quay về lobby và tiếp tục với các tính năng khác như quest/upgrade/shop.



(a) Màn tìm trận

(b) Màn roulette trước trận

Hình 14: Flow đầu vào trận theo runtime hiện tại: matching → roulette.



(a) Màn battle

(b) Reward hậu trận

Hình 15: Flow core gameplay: battle → chốt kết quả → reward.

6.5.2 Flow chơi theo battle-first

Bước 1 – Matching/tìm trận:

1. Người chơi vào matching từ lobby.
2. Hệ thống ghép cặp theo cấu hình và trạng thái phiên hiện tại.

Bước 2 – Roulette trước trận:

1. Sau matching, người chơi đi qua màn roulette.
2. Kết quả roulette xác nhận chuyển tiếp vào battle session.

Bước 3 – Battle:

1. Khởi tạo phiên trận và nhận `battleId`.

2. Thực hiện gameplay chiến đấu trong màn.

3. Trận kết thúc khi đạt điều kiện thắng hoặc thua.

6.5.3 Luật battle

- Battle runtime đi qua đúng chuỗi matching → roulette → battle session.
- Mỗi trận hợp lệ phải có `battleId` được backend cấp ở bước start.
- Mọi dữ liệu của trận được chốt tại bước finish battle, không suy diễn cục bộ.

6.5.4 Luật thắng/thua

- **Thắng:** người chơi hoàn tất điều kiện mục tiêu của màn battle hiện tại.
- **Thua:** không đạt điều kiện mục tiêu theo logic runtime của battle session.
- Trạng thái thắng/thua là đầu vào trực tiếp cho luồng hậu trận và reward.

6.5.5 Luật reward và đồng bộ hậu trận

- Client gửi finish battle kèm `idempotencyKey` để chốt kết quả an toàn khi retry.
- Reward, currency, statistics và quest progress chỉ có hiệu lực theo payload backend trả về.
- UI hậu trận hiển thị dữ liệu theo response mới nhất sau khi backend xác nhận.

6.5.6 Mở rộng sang các tính năng khác

- Sau reward, người chơi quay về lobby để tiếp tục quest và các mốc progression.
- Tài nguyên hậu trận được dùng cho upgrade hero/skill/castle.
- Shop là nhánh economy hỗ trợ progression, không phải trục chính của flow battle.

Phần flow và luật gameplay này phản ánh đúng phạm vi implemented-only hiện tại: battle-first, backend-authoritative và đồng bộ hậu trận rõ ràng.

6.6 Quy tắc nghiệp vụ và ràng buộc hệ thống

6.6.1 Server-authoritative state

- Reward, currency, statistics, quest progress, tutorial flags, castle progression, shop purchase result chỉ lấy từ payload backend.
- Client không tự suy diễn kết quả hậu trận hoặc kết quả claim.

6.6.2 Idempotency và retry

- Các thao tác có khả năng retry phải đi kèm idempotency key.
- Backend phải chống cấp trùng dữ liệu khi nhận lại request cùng khoá.

6.6.3 Nhất quán bất đồng bộ

- Khi có nhiều response gần nhau, client chỉ áp dụng trạng thái mới nhất theo mốc thời gian/version.
- Luồng hậu battle phải apply state trước khi mở các UI kế tiếp.

6.6.4 Tương thích cấu hình

- Battle session phải gắn với `configVersion` hợp lệ.
- Client và backend cần cùng phiên bản config để tránh lệch tính toán.

6.7 Phân tích dữ liệu ở mức logic

6.7.1 Dữ liệu tiến trình

- Auth/session tokens.
- Player profile và statistics.
- Tutorial progress flags.
- Currency và inventory (heroes/skills).
- Formation state.
- Quest tracks và progress rewards.
- Battle session state và dữ liệu hậu trận.
- Castle progression state.

6.7.2 Thực thể lõi theo luồng hiện tại

- **Identity**: account, session/token state.
- **Player snapshot**: profile, statistics, tutorial, currency, inventory, formation, quests.
- **Battle**: phiên start/finish và reward/result state.
- **Progression**: hero/skill upgrade state, castle state, shop purchase state.

6.7.3 Quan hệ dữ liệu quan trọng

- Một account gắn với một định danh player dùng xuyên suốt các luồng auth, state, battle, quest.
- Snapshot `/me` là điểm hợp nhất dữ liệu cho nhiều phân hệ để client đồng bộ nhất quán.
- Battle finish, quest claim, tutorial patch và upgrade đều tạo ra thay đổi progression cần phản ánh lại về snapshot hiển thị.

6.8 Tổng kết chương

Chương này đã phân tích hệ thống theo đúng phạm vi implemented-only: xác thực phiên, đồng bộ trạng thái người chơi, quest, battle session, tutorial sync, nâng cấp hero/skill, nâng cấp castle và shop purchase. Kết quả phân tích là đầu vào trực tiếp cho quyết định thiết kế ở Chương 7 và bám chặt baseline vận hành thực tế.

7 Thiết kế hệ thống

7.1 Mục tiêu thiết kế

Mục tiêu của chương này là chuyển hoá kết quả phân tích ở Chương 6 thành các quyết định thiết kế cụ thể ở mức: (i) kiến trúc tổng thể client–server và triển khai; (ii) thiết kế mô-đun backend và ranh giới trách nhiệm; (iii) thiết kế dữ liệu (progression/state và game configuration); (iv) thiết kế giao tiếp REST và quy ước dữ liệu; (v) thiết kế các cơ chế đảm bảo nhất quán, bảo mật và quan sát hệ thống; (vi) thiết kế giao diện UI/UX ở mức tham chiếu để đối chiếu với luồng nghiệp vụ và API.

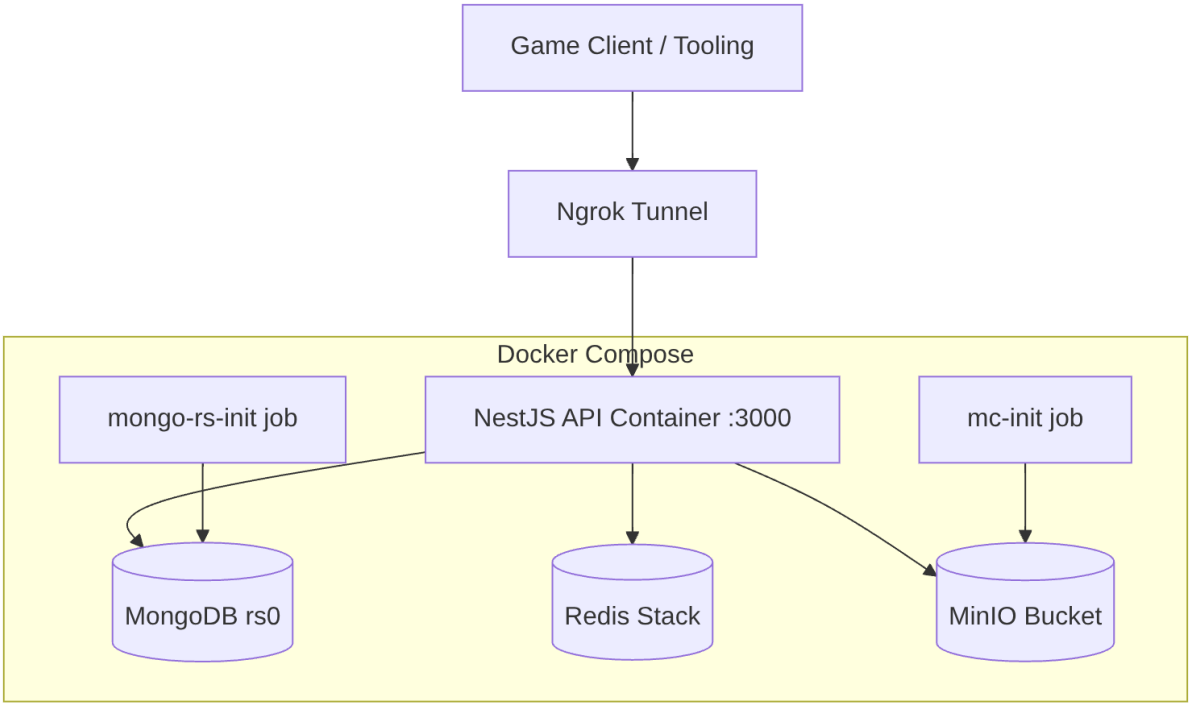
Trong baseline hiện tại, thiết kế ưu tiên:

- **Tính đúng đắn và nhất quán**: các thay đổi ảnh hưởng tiến trình (reward/currency/checkpoint, nâng cấp, tiêu hao vật phẩm) phải được server xác nhận.
- **Tính mô-đun**: phân tách rõ theo phân hệ (auth, player state, quest, battle, inventory, skills, castle, shop, leaderboard).
- **Tính data-driven cho cấu hình**: tách dữ liệu cấu hình gameplay khỏi mã nguồn; hỗ trợ quản trị và thay đổi nhanh.
- **Phù hợp phạm vi đồ án**: triển khai ở mức hợp lý (single deployment có thể chạy được), nhưng vẫn giữ được hướng mở rộng.

7.2 Kiến trúc tổng thể và triển khai

7.2.1 Tổng quan triển khai

Hệ thống được tổ chức thành 3 lớp chính: *Client Devices*, *Backend Infrastructure* và *Data & Storage*. Hình 16 mô tả các thành phần và đường giao tiếp chính.



Hình 16: Sơ đồ triển khai tổng quan hệ thống (bản cập nhật)

Các thành phần cốt lõi:

- **Game Client (Unity)**: hiển thị đồ hoạ, UI, thu thập input; hiện giao tiếp với server qua HTTPS cho các luồng cốt lõi.
- **API Server (NestJS)**: lớp ứng dụng xử lý REST API cho các phân hệ đang vận hành.
- **Job Worker**: thành phần nền theo lịch, hiện được chuẩn bị để xử lý các tác vụ như mail hệ thống, sự kiện, tổng hợp log và dọn dẹp dữ liệu runtime.
- **MongoDB (Primary DB)**: lưu dữ liệu tiến trình bền vững và các collections cấu hình.
- **Redis (Cache & Pub/Sub)**: dùng cho cache; phần phối hợp qua pub-sub được giữ theo nhu cầu mở rộng nhiều instance.
- **MinIO (Object Storage)**: lưu trữ tệp và asset bundle (nếu áp dụng); client tải qua presigned URL.

7.2.2 Nguyên tắc server-authoritative

Thiết kế áp dụng nguyên tắc **server-authoritative** cho các tác vụ ảnh hưởng tiến trình và công bằng:

- Client chỉ gửi **ý định** (intent), không gửi trạng thái "đã xảy ra" như "đã trúng đòn" hay "đã nhận item".
- Server kiểm tra luật (cooldown, tài nguyên, điều kiện tham gia, dung lượng inventory, điều kiện skill) và chỉ khi hợp lệ mới ghi dữ liệu.
- Các giao dịch reward/currency/checkpoint, nâng cấp trang bị/kỹ năng được coi là **giao dịch nghiệp vụ** và phải có log/audit để truy vết.

Các quyết định này giúp giảm rủi ro gian lận và tăng tính nhất quán trong môi trường online [10, 2].

7.2.3 Phân tách giao tiếp: REST API vận hành

REST được dùng cho:

- vòng đời xác thực: register/login/refresh/logout;
- đồng bộ state: `/me`, quest, tutorial progress;
- battle session start/finish và các cập nhật hậu trận;
- progression: nâng cấp hero/skill, castle upgrade;
- shop catalog/purchase và leaderboard.

7.3 Thiết kế mô-đun backend

Backend được tổ chức theo phong cách module hoá của NestJS [7]. Mỗi mô-đun tương ứng một phân hệ nghiệp vụ và sở hữu ranh giới dữ liệu rõ ràng. Bảng 6 tóm tắt các mô-đun chính.

Mô-đun	**Trách nhiệm chính**	**Dữ liệu/Collection liên quan**
Auth Module	Đăng ký/đăng nhập; cấp và xác thực token; chính sách role	USER, SESSION_ RUNTIME
Profile Module	Tạo/chọn profile; cập nhật last_played; thiết lập profile	GAME_PROFILE
Character Module	Tạo/chọn/xoá nhân vật; quản lý level/EXP; base stats	CHARACTER
Inventory Module	Quản lý slot, stack, currency; dùng vật phẩm; cập nhật reward	INVENTORY, CONFIG_ITEM
Equipment Module	Trang bị theo slot; đồng bộ durability/ enhance; tính chỉ số cộng thêm	CHARACTER_ EQUIPMENT, CONFIG_ITEM
Skill Module	Điểm kỹ năng; learned skills; hotbar; nâng cấp skill	CHARACTER_ SKILLS, CONFIG_ SKILL
Battle Module	Khởi tạo/kết thúc battle session; chốt reward/ statistics/quest updates	SESSION_ RUNTIME, battle result state
Quest Module	Đồng bộ quest tracks; claim reward và progress reward	quest state, progress reward state
Castle Module	Nâng cấp castle theo nguồn sự thật backend	statistics/castle progression state
Shop Module	Tải catalog, purchase offer, trả reward và currency mới	shop catalog state, purchase result state
Config Module	Nạp và cung cấp truy xuất cấu hình; versioning; cache	CONFIG_* (static)

Bảng 6: Bản đồ mô-đun backend và dữ liệu liên quan

7.3.1 Các thành phần dùng chung (cross-cutting)

Để giảm trùng lặp và tăng tính nhất quán, hệ thống dùng các thành phần dùng chung:

- **Validation**: kiểm tra input cho REST (schema/DTO) để chặn dữ liệu sai từ đầu.
- **Error handling**: chuẩn hoá mã lỗi và thông điệp lỗi; đảm bảo client có thể hiển thị đúng trạng thái UI (loading/error/disabled).
- **Logging & audit**: log request, log nghiệp vụ (reward/equip/skill-up/shop/castle), kèm correlation id.
- **Rate limit / anti-spam**: giới hạn tần suất với endpoint nhạy cảm (login, purchase, claim, battle finish).

7.4 Thiết kế dữ liệu

7.4.1 Nguyên tắc thiết kế dữ liệu

Hệ thống dữ liệu được chia thành 2 nhóm:

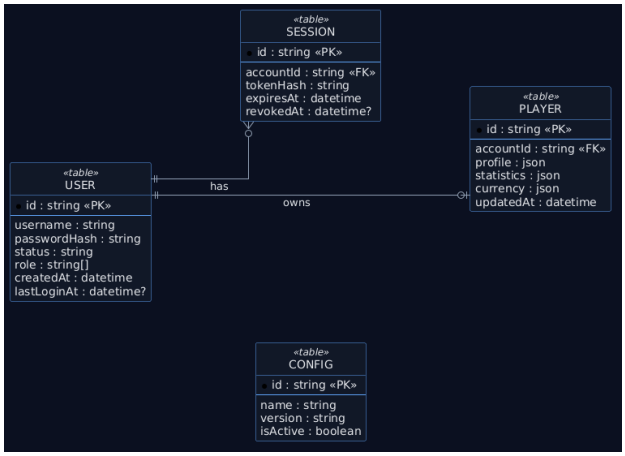
- **Progression/State (động)**: dữ liệu người chơi thay đổi theo thời gian (profile, character, inventory, equipment, skills, quest, battle result, castle, shop state).
- **Game Configuration (tĩnh)**: dữ liệu định nghĩa và thông số gameplay (hero/quest/rank/upgrade/account-level/campaign/shop/sprite resources).

Với MongoDB, thiết kế ưu tiên:

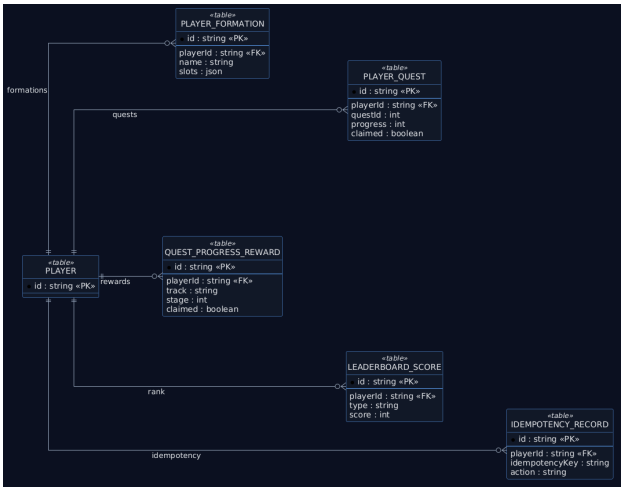
- **Embed** khi dữ liệu có vòng đời gắn chặt với thực thể cha và thường được tải cùng nhau (ví dụ: base_stats/resources/position nhúng trong CHARACTER; items nhúng trong INVENTORY; skill list/hotbar nhúng trong CHARACTER_SKILLS; từng slot trang bị nhúng trong CHARACTER_EQUIPMENT).
- **Reference** khi dữ liệu là định nghĩa tĩnh dùng chung (CONFIG collections) hoặc cần tái sử dụng giữa nhiều thực thể.

7.4.2 Mô hình thực thể chính (ERD mức logic)

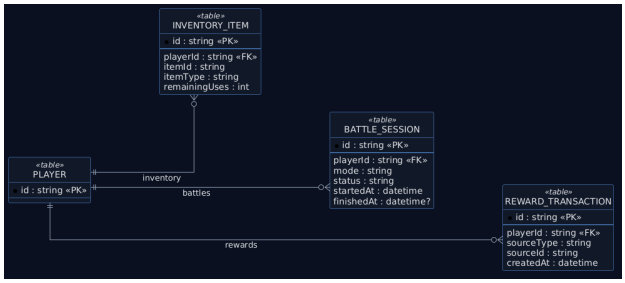
Trong phạm vi báo cáo, ERD được trình bày ở mức logic để làm rõ quan hệ 1:1, 1:N và các liên kết tham chiếu tới cấu hình. Các "STRUCT_*" trong ERD đại diện cho **embedded document** (struct nhúng), không phải collection độc lập.



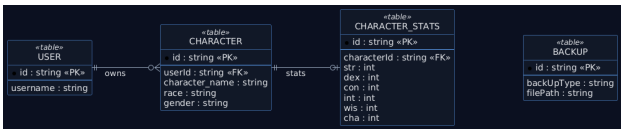
(a) ERD auth & player



(b) ERD character progression



(c) ERD inventory & battle



(d) ERD character customization

Hình 17: Bộ sơ đồ ERD mức logic theo phân hệ (trình bày bằng hình)

7.4.3 Phân tích quan hệ và phụ thuộc thuộc tính

Phần này phân tích ERD theo nhóm để làm rõ mục đích dữ liệu, quan hệ chính, khóa/tham chiếu và các ràng buộc cần giữ nhất quán.

Nhóm danh tính và phiên (USER, GAME_PROFILE, SESSION_RUNTIME) **Mục đích dữ liệu:** quản lý tài khoản, profile chơi và trạng thái phiên runtime.

- **Quan hệ chính:** USER (1:N) GAME_PROFILE; USER (1:1) OBJECT_USER_SETTINGS (nhúng); SESSION_RUNTIME tham chiếu GAME_PROFILE.
- **Khóa/Ref quan trọng:** `USER._id`, `GAME_PROFILE.user_id`.
- `SESSION_RUNTIME.profile_id`.
- **Ràng buộc nhất quán:** `username`/`email` cần unique. session chỉ hợp lệ khi profile còn tồn tại; `expires_at` dùng TTL để tự dọn phiên hết hạn.

Nhóm nhân vật và tiến trình (CHARACTER + stats/resources/position) **Mục đích dữ liệu:** lưu trạng thái phát triển nhân vật và dữ liệu hiển thị/vận hành trong gameplay.

- **Quan hệ chính:** GAME_PROFILE (1:N) CHARACTER; CHARACTER (1:1) với các struct nhúng `base_stats`, `resources`, `position`.
- **Khóa/Ref quan trọng:** `CHARACTER._id`, `CHARACTER.profile_id`.
- **Ràng buộc nhất quán:** các struct nhúng phải theo cùng vòng đời với CHARACTER; cập nhật level/EXP phải đồng bộ với các ngưỡng progression.

Nhóm inventory/equipment/skill (instance state vs config reference) **Mục đích dữ liệu:** lưu toàn bộ tài sản và năng lực tác chiến của nhân vật.

- **Quan hệ chính:** CHARACTER (1:1) INVENTORY.
- CHARACTER (1:1) CHARACTER_EQUIPMENT.
- CHARACTER (1:1) CHARACTER_SKILLS.
- **Khóa/Ref quan trọng:** `character_id` (unique trong từng collection 1:1).
- `item_ref_id` → CONFIG_ITEM.
- `skill_ref_id` → CONFIG_SKILL, `unique_uid` cho item instance.
- **Ràng buộc nhất quán:** item đã equip phải tồn tại hợp lệ trong inventory; hotbar chỉ tham chiếu skill đã học; dữ liệu động (durability, cooldown, skill level) không được lệch khỏi config tĩnh.

Nhóm battle/runtime linkage **Mục đích dữ liệu:** nối danh tính phiên với trạng thái chiến đấu và kết quả hậu trận.

- **Quan hệ chính:** SESSION_RUNTIME liên kết profile/character/room để điều phối runtime.
- **Khóa/Ref quan trọng:** `token`, `character_id`, `current_room_id`.
- **Ràng buộc nhất quán:** một phiên runtime chỉ ánh xạ một ngữ cảnh hoạt động tại một thời điểm; dữ liệu hậu trận phải commit về các thực thể progression theo thứ tự giao dịch.

Nhóm thực thể	Quan hệ chính	Ràng buộc cốt lõi
Identity & Session	USER (1:N) GAME_PROFILE; SESSION_ RUNTIME → GAME_PROFILE	Unique account identity; session TTL; không orphan profile trong phiên.
Character & Progression	GAME_PROFILE (1:N) CHARACTER; CHARACTER nhúng stats/ resources/position	Tiến trình level/EXP nhất quán; struct nhúng cùng vòng đời CHARACTER.
Inventory/ Equipment/Skill	CHARACTER (1:1) INVENTORY/ EQUIPMENT/ SKILLS; ref sang CONFIG_ITEM/ CONFIG_SKILL	Đồng bộ item equip và inventory; hotbar hợp lệ; state động không lệch config tĩnh.
Battle Runtime/ Linkage	SESSION_ RUNTIME nối profile/character/ room	Một phiên một ngữ cảnh; commit hậu trận theo giao dịch chuẩn.

Bảng 7: Tóm tắt Entity–Relation–Constraint theo nhóm ERD

7.4.4 Từ điển dữ liệu (Data Dictionary) ở mức collection

Phần này mô tả các collection chính ở mức thuộc tính, mục đích và ràng buộc quan trọng. Các struct nhúng (OBJECT_USER_SETTINGS, STRUCT_*, ...) được xem là "sub-document" nằm trong document cha.

Collection: USER

- **Mục đích:** lưu tài khoản đăng nhập và thiết lập người dùng.
- **Quan hệ:** USER (1) – (N) GAME_PROFILE; USER (1) nhúng (1) OBJECT_USER_ SETTINGS.

Bảng 8: Từ điển dữ liệu – Collection USER

Thuộc tính	Kiểu	Bắt buộc	Mô tả / Ràng buộc
`_id`	ObjectId	Yes	Khoá chính.
`username`	string	Yes	Unique; dùng đăng nhập (tuỳ chính sách).

(còn tiếp trang sau)

Thuộc tính	Kiểu	Bắt buộc	Mô tả / Ràng buộc
email	string	Yes	Unique; dùng đăng nhập/khôi phục.
password_hash	string	Yes	Hash mật khẩu; không lưu plaintext.
role	string	Yes	user; dùng phân quyền nghiệp vụ client/gameplay.
status	enum	Yes	active/banned/...; ban user sẽ thu hồi session.
created_at	date	Yes	Thời điểm tạo.
last_login	date	No	Cập nhật khi login thành công.
settings	object/embed	Yes	Nhúng OBJECT_USER_ SETTINGS.

Collection: GAME_PROFILE

- **Mục đích:** tách tiến trình theo profile và làm "root" cho character và các trạng thái gameplay.
- **Quan hệ:** tham chiếu USER; sở hữu CHARACTER (1:N) và liên kết các snapshot gameplay theo player.

Bảng 9: Từ điển dữ liệu – Collection GAME_PROFILE

Thuộc tính	Kiểu	Bắt buộc	Mô tả / Ràng buộc
_id	ObjectId	Yes	Khoá chính.
user_id	ObjectId	Yes	FK tới USER._id; index theo user_id.
profile_name	string	Yes	Tên profile; có thể unique theo (user_id, profile_name).
server_id	string	No/ Yes	Server/realm; có thể bắt buộc nếu game phân cụm theo khu vực.
created_at	date	Yes	Thời điểm tạo profile.
last_played	date	No	Cập nhật khi profile được chọn.
preferences	object/embed	No	Cài đặt local của profile (tuỳ thiết kế).

Collection: SESSION_RUNTIME

- **Mục đích:** lưu trạng thái phiên hoạt động (token, socket, bối cảnh profile/character/room).
- **Quan hệ:** tham chiếu GAME_PROFILE theo `profile_id`.

Bảng 10: Từ điển dữ liệu – Collection SESSION_RUNTIME

Thuộc tính	**Kiểu**	**Bắt buộc**	**Mô tả / Ràng buộc**
`token`	string	Yes	PK; token phiên.
`socket_id`	string	No	Trường dự phòng cho định danh kết nối phiên; cập nhật khi có cơ chế kết nối phiên.
`user_id`	string	Yes	Tham chiếu user; dùng kiểm tra nhanh.
`profile_id`	string	No	Profile đang active; ref GAME_PROFILE._id.
`character_id`	string	No	Nhân vật đang active; ref CHARACTER._id.
`current_room_id`	string	No	Room/instance đang tham gia.
`expires_at`	timestamp	Yes	TTL đề xuất để tự dọn session.

Collection: CHARACTER

- **Mục đích:** lưu tiến trình nhân vật; là trung tâm cho chiến đấu và meta.
- **Quan hệ:** tham chiếu GAME_PROFILE theo `profile_id`; liên kết INVENTORY/EQUIPMENT/SKILLS theo 1:1.

Bảng 11: Từ điển dữ liệu – Collection CHARACTER

Thuộc tính	**Kiểu**	**Bắt buộc**	**Mô tả / Ràng buộc**
`_id`	ObjectId	Yes	PK.
`profile_id`	ObjectId	Yes	FK tới GAME_PROFILE._id; index theo profile_id.
`name`	string	Yes	Tên nhân vật; có thể unique theo (profile_id, name).

(còn tiếp trang sau)

Thuộc tính	Kiểu	Bắt buộc	Mô tả / Ràng buộc
gender	enum	No	Thông tin hiển thị; không bắt buộc ảnh hưởng logic.
class	enum	Yes	Class hiện tại; ảnh hưởng bộ skill/role.
level	int	Yes	Level; giới hạn tối đa theo thiết kế (ví dụ 100).
current_exp	long	Yes	EXP tích luỹ; dùng tính level-up.
created_at	date	Yes	Thời điểm tạo.
base_stats	object/ embed	Yes	STRUCT_BASE_STATS (STR/DEX/ CON/INT/WIS/CHA hoặc mapping tương đương).
resources	object/ embed	Yes	STRUCT_CURRENT_RESOURCES (HP/MP/Stamina).
position	object/ embed	Yes	STRUCT_POSITION_DATA (map + toạ độ).

Collection: INVENTORY

- **Mục đích:** lưu currency và item instance của nhân vật.
- **Quan hệ:** INVENTORY (1:1) CHARACTER; item instance ref CONFIG_ITEM.

Bảng 12: Từ điển dữ liệu – Collection INVENTORY

Thuộc tính	Kiểu	Bắt buộc	Mô tả / Ràng buộc
_id	ObjectId	Yes	PK.
character_id	ObjectId	Yes	Unique FK tới CHARACTER._id.
max_slots	int	Yes	Số slot tối đa.
used_slots	int	Yes	Slot đang dùng; có thể suy ra nhưng lưu để tối ưu.
currency_gold	double	Yes	Vàng.
currency_gem	double	Yes	Gem.
items	array/embed	Yes	Danh sách STRUCT_INVENTORY_ITEM.

Ràng buộc item instance (STRUCT_INVENTORY_ITEM):

- item_ref_id phải tồn tại trong CONFIG_ITEM.id.

- `quantity` tuân thủ `CONFIG_ITEM.max_stack` (nếu là item stackable).
- `slot_index` nằm trong `[0, max_slots-1]` và không trùng trong cùng inventory.
- `unique_uid` định danh item instance (phục vụ enhance/durability/equip).

Collection: CHARACTER_EQUIPMENT

- **Mục đích:** lưu trang bị theo slot của nhân vật (1:1).
- **Quan hệ:** tham chiếu CHARACTER; từng slot tham chiếu CONFIG_ITEM qua `item_ref_id`.

Bảng 13: Từ điển dữ liệu – Collection CHARACTER_EQUIPMENT

Thuộc tính	**Kiểu**	**Bắt buộc**	**Mô tả / Ràng buộc**
`_id`	ObjectId	Yes	PK.
`character_id`	ObjectId	Yes	Unique FK tới CHARACTER._id.
`helmet`	object/embed	No	Slot mũ; STRUCT_EQUIPPED_ITEM (rỗng nếu chưa trang bị).
`chestplate`	object/embed	No	Slot giáp; STRUCT_EQUIPPED_ITEM.
`leggings`	object/embed	No	Slot quần; STRUCT_EQUIPPED_ITEM.
`boots`	object/embed	No	Slot giày; STRUCT_EQUIPPED_ITEM.
`necklace`	object/embed	No	Slot dây chuyền; STRUCT_EQUIPPED_ITEM.
`ring`	object/embed	No	Slot nhẫn; STRUCT_EQUIPPED_ITEM.
`weapon`	object/embed	No	Slot vũ khí; STRUCT_EQUIPPED_ITEM.

Ràng buộc slot trang bị (STRUCT_EQUIPPED_ITEM):

- `item_ref_id` tham chiếu CONFIG_ITEM.`id`.
- `unique_uid` phải trùng với instance đang tồn tại trong inventory theo chính sách nhất quán (equip/unequip).
- `enhancement_level/durability_current` phải đồng bộ với instance (nếu bạn chọn lưu ở cả inventory và equipment).

Collection: CHARACTER_SKILLS

- **Mục đích:** quản lý điểm kỹ năng, danh sách skill đã học và hotbar (1:1).
- **Quan hệ:** tham chiếu CHARACTER; skill ref CONFIG_SKILL.

Bảng 14: Từ điển dữ liệu – Collection CHARACTER_SKILLS

Thuộc tính	**Kiểu**	**Bắt buộc**	**Mô tả / Ràng buộc**
`_id`	ObjectId	Yes	PK.
`character_id`	ObjectId	Yes	Unique FK tới CHARACTER._id.
`skill_points_available`	int	Yes	Điểm kỹ năng còn lại để học/nâng.
`learned_skills`	array/embed	Yes	Danh sách STRUCT_LEARNED_SKILL.
`hotbar_loadout`	array/embed	Yes	Danh sách STRUCT_HOTBAR_SLOT.

Ràng buộc learned skill (STRUCT_LEARNED_SKILL) và hotbar (STRUCT_HOTBAR_SLOT):

- `skill_ref_id` phải tồn tại trong CONFIG_SKILL.`id`.
- `current_level` không vượt `CONFIG_SKILL.max_level`.
- `cooldown_end` là runtime; tránh cập nhật DB theo tick, chỉ lưu khi cần.
- `hotbar_slot.slot_index` không trùng; nằm trong miền slot UI định nghĩa (ví dụ 0..7).

Collection: CONFIG_ITEM / CONFIG_SKILL (tĩnh) Các collection config là dữ liệu tĩnh phục vụ gameplay (data-driven). Các trường có thể mở rộng dần theo nhu cầu cân bằng và nội dung.

Bảng 15: Từ điển dữ liệu – Collection CONFIG_ITEM

Thuộc tính	**Kiểu**	**Bắt buộc**	**Mô tả / Ràng buộc**
`id`	string	Yes	PK; khoá config.
`name`	string	Yes	Tên hiển thị.
`type`	string/enum	Yes	Loại item (mat/consumable/equip/...).

(còn tiếp trang sau)

Thuộc tính	Kiểu	Bắt buộc	Mô tả / Ràng buộc
max_stack	int	Yes	Số lượng stack tối đa (1 nếu không stack).

Bảng 16: Từ điển dữ liệu – Collection CONFIG_SKILL

Thuộc tính	Kiểu	Bắt buộc	Mô tả / Ràng buộc
id	string	Yes	PK; khoá config.
name	string	Yes	Tên skill.
type	enum	Yes	Active/Passive/Buff.
max_level	int	Yes	Level tối đa.
base_cooldown	float	Yes	Hồi chiêu cơ bản.
base_mana_cost	float	Yes	Mana cost cơ bản.
description	string	No	Mô tả hiển thị UI.

7.4.5 Chỉ mục (indexes) và khoá truy vấn

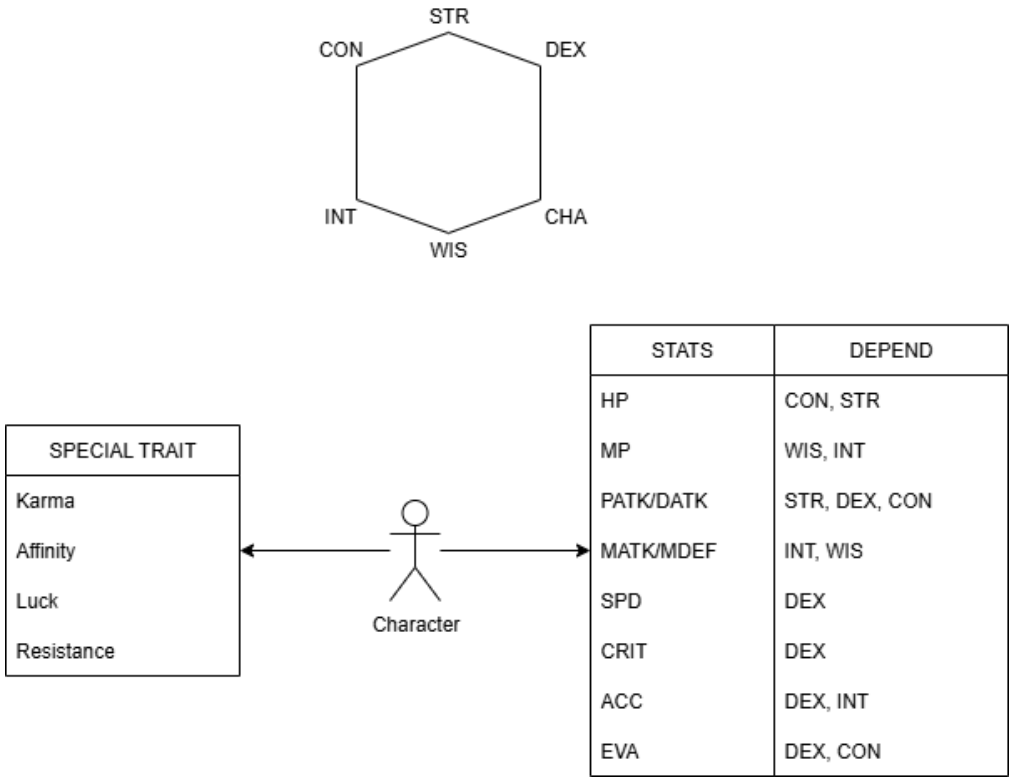
Để đảm bảo truy vấn ổn định, các chỉ mục đề xuất:

- USER: unique index cho `username`, `email`; index cho `status` (nếu thường lọc).
- GAME_PROFILE: index theo `user_id`; unique theo (`user_id`, `profile_name`) nếu cần.
- CHARACTER: index theo `profile_id`; unique theo (`profile_id`, `name`) nếu cần.
- INVENTORY: unique index theo `character_id`.
- CHARACTER_EQUIPMENT: unique index theo `character_id`.
- CHARACTER_SKILLS: unique index theo `character_id`.
- SESSION_RUNTIME: index theo `user_id`, `profile_id`; TTL index theo `expires_at`.
- CONFIG collections: unique index theo `id` (khoá cấu hình).

7.5 Thiết kế hệ thống chỉ số và tăng trưởng nhân vật

7.5.1 Thuộc tính nền và chỉ số suy diễn

Nhân vật có 6 thuộc tính nền: STR, DEX, CON, INT, WIS, CHA. Từ đó hệ thống suy diễn các chỉ số chiến đấu như HP/MP/ATK/DEF/CRIT/ACC/EVA. Hình 18 minh hoạ mối phụ thuộc giữa các nhóm chỉ số trong thiết kế dữ liệu.



Hình 18: Mô hình thuộc tính nền và chỉ số suy diễn

Thiết kế tách rõ:

- **Base Stats**: điểm phân bổ trực tiếp bởi người chơi theo level (+5 points/level, tối đa level 100).
- **Derived Stats**: chỉ số tính toán (HP/MP/ATK/...) dựa trên công thức và hệ số cân bằng.
- **Special Traits**: Karma, Affinity, Luck, Resistance là nhóm "đặc tính" dùng cho điều kiện mở khoá và/hoặc hiệu ứng meta.

7.5.2 Tính toán chỉ số theo hướng data-driven

Để tránh hard-code công thức, thiết kế cho phép cấu hình hoá hệ số (ví dụ: HP nhận trọng số từ CON và STR; MP nhận trọng số từ WIS và INT; SPD từ DEX). Cách tiếp cận:

- Công thức được lưu dưới dạng **config** (ví dụ bảng hệ số theo stat).
- Server tính toán derived stats khi:
  - player phân bổ điểm;
  - thay đổi trang bị;
  - thay đổi class có modifier;
  - nâng cấp kỹ năng passive/buff có modifier (nếu áp dụng).

Ở mức thiết kế, có thể biểu diễn công thức tuyến tính:

$$S_{derived} = \sum_i w_i \cdot A_i + b$$

trong đó A_i là thuộc tính nền (STR/DEX/CON/INT/WIS/CHA) và w_i là hệ số cấu hình.

7.6 Thiết kế quản trị cấu hình game (Config Pipeline)

Pipeline cấu hình chỉ áp dụng cho **các bảng cấu hình/thông số gameplay** (hero/quest/rank/upgrade/campaign/shop/sprite resources). Dữ liệu tiến trình người chơi vẫn được ghi trực tiếp bởi server trong các collections động.

7.6.1 Luồng dữ liệu cấu hình

Luồng quản trị cấu hình đề xuất:

1. **Soạn thảo trên Google Sheets**: mỗi bảng là một nhóm cấu hình (hero, quest, rank, upgrade, campaign, shop, sprite resources).
2. **Xuất TSV**: xuất theo template thống nhất; đảm bảo kiểu dữ liệu và khoá `id`.
3. **Import vào DB**: script import thực hiện *upsert* theo `id`; tạo bản ghi version/import log.
4. **Nạp vào Config Manager**: khi server khởi động (hoặc theo chính sách), server tải config từ DB vào bộ nhớ để truy xuất nhanh.

7.6.2 Thiết kế Config Manager

Config Manager là lớp truy xuất cấu hình thống nhất cho toàn bộ service:

- **In-memory cache**: lưu map `id` → `config object` cho từng nhóm cấu hình.
- **Versioning**: gắn `config_version` cho lần nạp, phục vụ debug và đảm bảo nhất quán khi tính toán.
- **Validation khi nạp**: kiểm tra dữ liệu sau import (schema, khoá trùng, tham chiếu hợp lệ).

Ở baseline hiện tại, cơ chế reload có thể ở mức đơn giản (restart server để nạp lại), kết hợp version manifest để client đồng bộ đúng phiên bản config.

7.7 Thiết kế giao tiếp client–server

7.7.1 REST API: nhóm endpoint và quy ước

REST API được nhóm theo phân hệ. Mục tiêu thiết kế API:

- tài nguyên rõ ràng (resource-oriented);
- payload ổn định cho UI;
- lỗi có mã và thông điệp nhất quán.

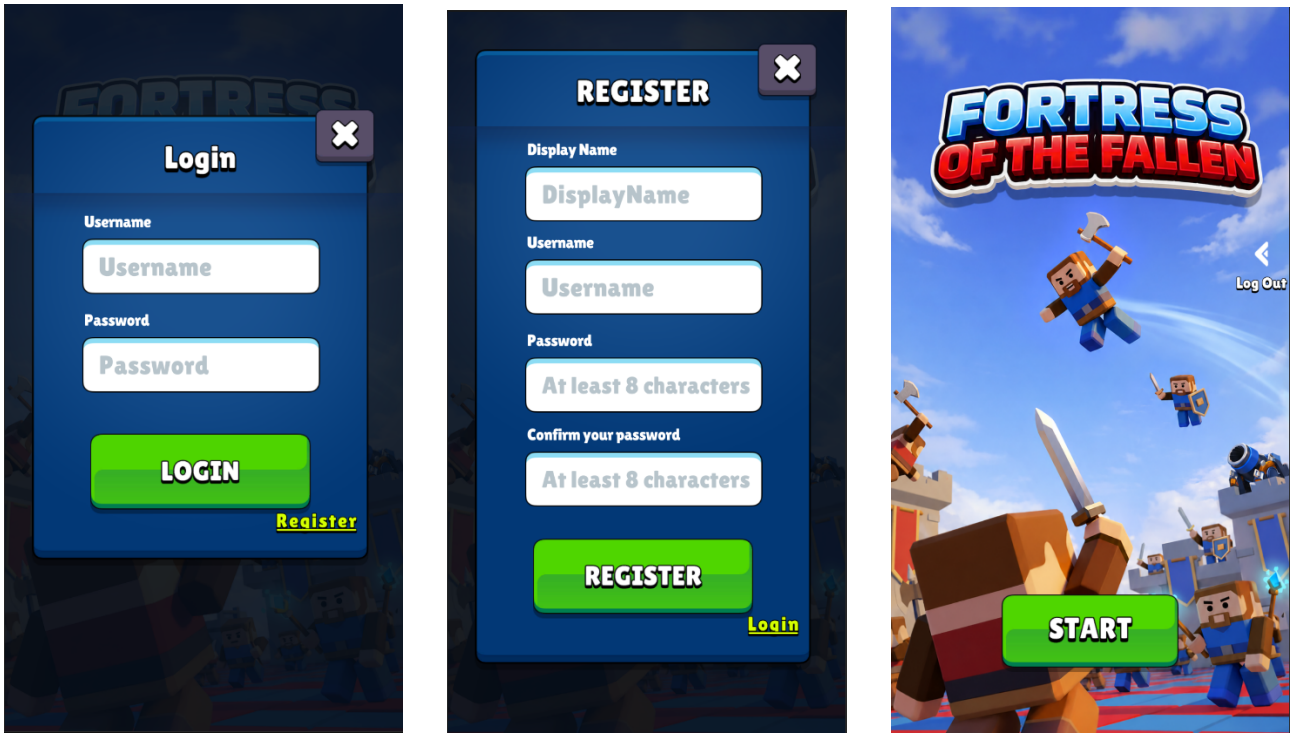
Nhóm endpoint vận hành trong baseline hiện tại:

- `/auth/register`, `/auth/login`, `/auth/refresh`, `/auth/logout`
- `/me`, `/me/profile`, `/me/tutorial-progress`
- `/quests`, `/quests/{questId}/claim`
- `/quests/progress-rewards/{track}/{stage}/claim`
- `/battles/start`, `/battles/{battleId}/finish`
- `/inventory/heroes/{instanceId}/upgrade`
- `/inventory/heroes/{instanceId}/evolve`
- `/skills/purchase`, `/skills/{skillId}/upgrade`
- `/lobby/castle/upgrade`
- `/shop/catalog`, `/shop/offers/{offerId}/purchase`
- `/leaderboard`, `/matchmaking/opponents`, `/configs/manifest`

7.8 Thiết kế giao diện người dùng (UI/UX) mức tham chiếu

Mục này tổng hợp các màn hình UI mới được chụp trực tiếp từ bản Unity hiện tại. Các hình được dùng để minh hoạ luồng người dùng thực tế (xác thực → lobby → nhiệm vụ/nâng cấp → vào trận → nhận thưởng), đồng thời đối chiếu trạng thái UI với dữ liệu backend đang đồng bộ qua REST API.

7.8.1 Luồng UI xác thực: Login/Register



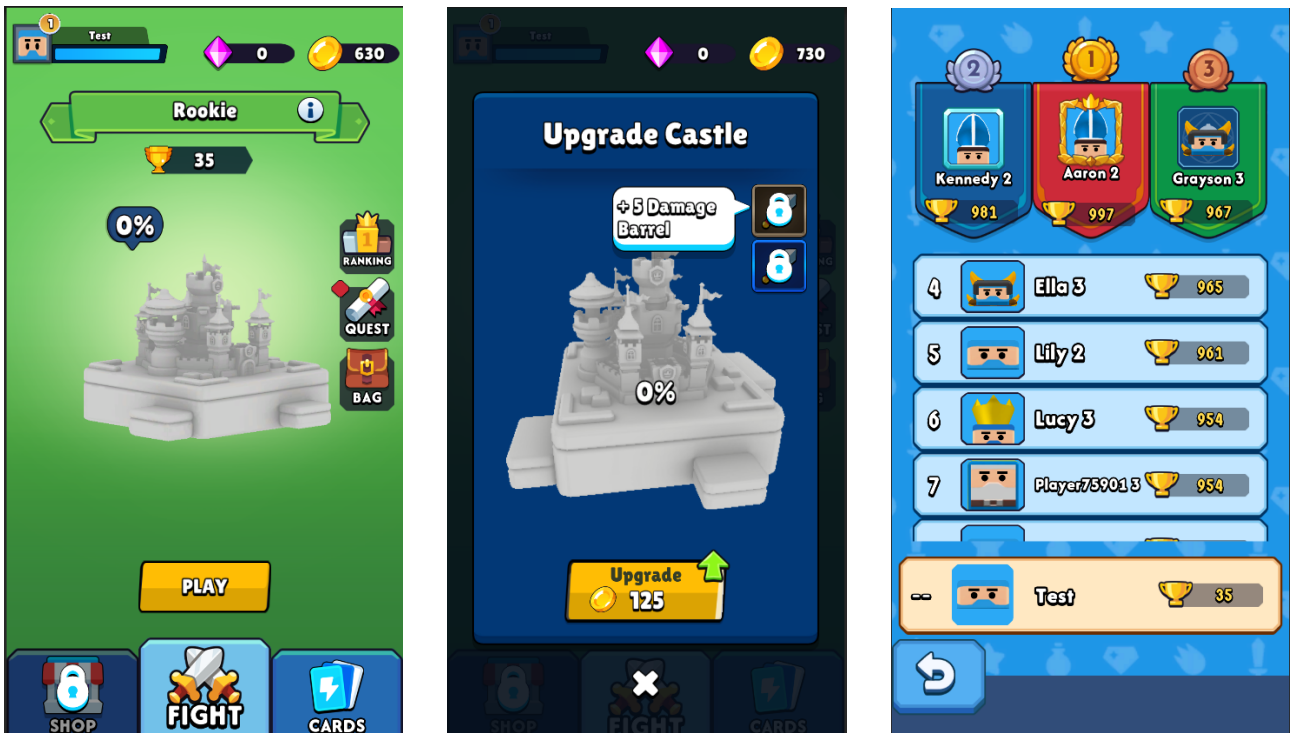
(a) Đăng nhập (b) Đăng ký (c) Loading

Hình 19: Luồng UI xác thực và tải dữ liệu ban đầu

Mapping nghiệp vụ và API:

- **Input validation**: username/password không rỗng; hiển thị lỗi theo mã lỗi chuẩn hoá.
- **Endpoint**: `/auth/login`, `/auth/register`, `/auth/refresh`, `/me`.
- **Trạng thái UI**: loading khi submit; disable nút khi request đang chạy; chuyển scene sau khi token và player state được tải thành công.

7.8.2 UI lobby và điều hướng phân hệ



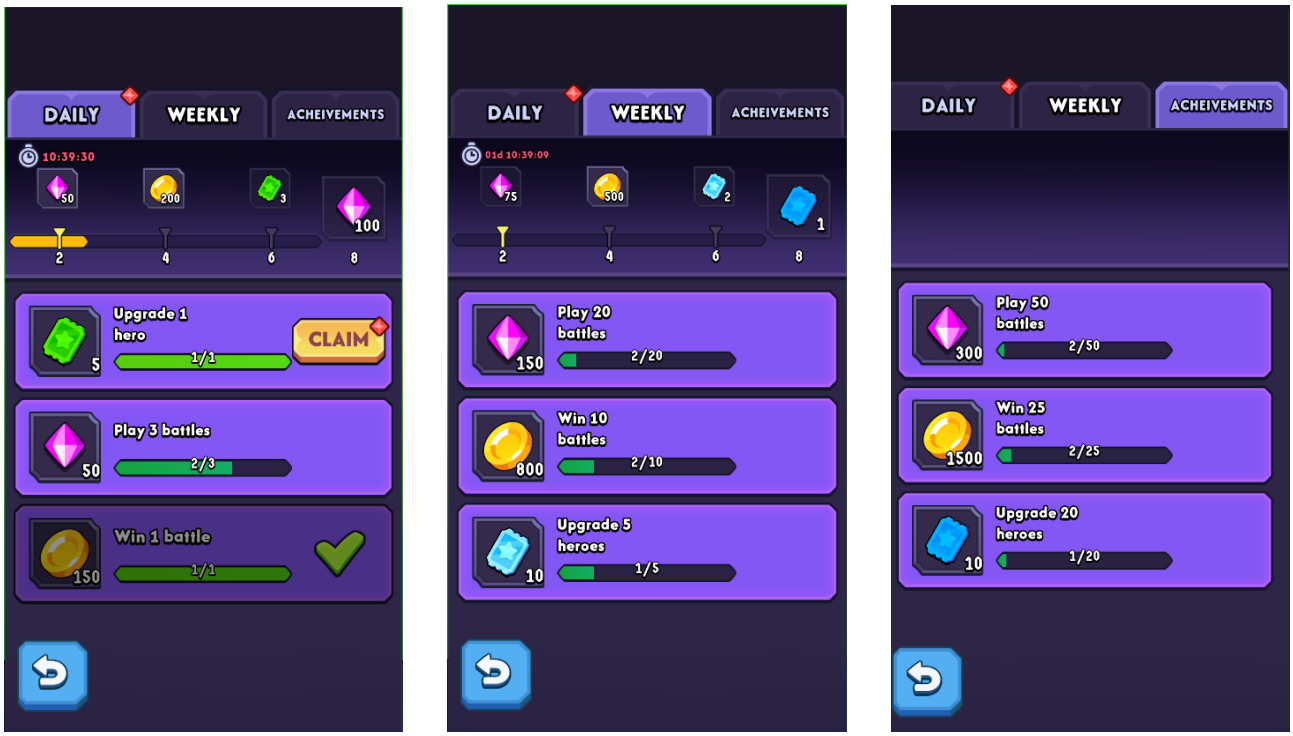
(a) Lobby chính (b) Nâng cấp castle (c) Bảng xếp hạng

Hình 20: UI lobby, nâng cấp castle và leaderboard

Mapping nghiệp vụ và API:

- **Player state**: lobby đọc `profile`, `statistics`, `currency` từ `/me`.
- cùng endpoint `/me` trả `formation` và `tutorialProgress`.
- **Castle upgrade**: UI dùng `levelCastle` và `lobbyUpgradeSpent`.
- thao tác nâng cấp đi qua endpoint backend riêng để tránh mất tiến trình khi đồng bộ lại.
- **Leaderboard**: hiển thị điểm/trophy và thứ hạng theo dữ liệu rank hoặc leaderboard từ backend.

7.8.3 UI quest và milestone reward



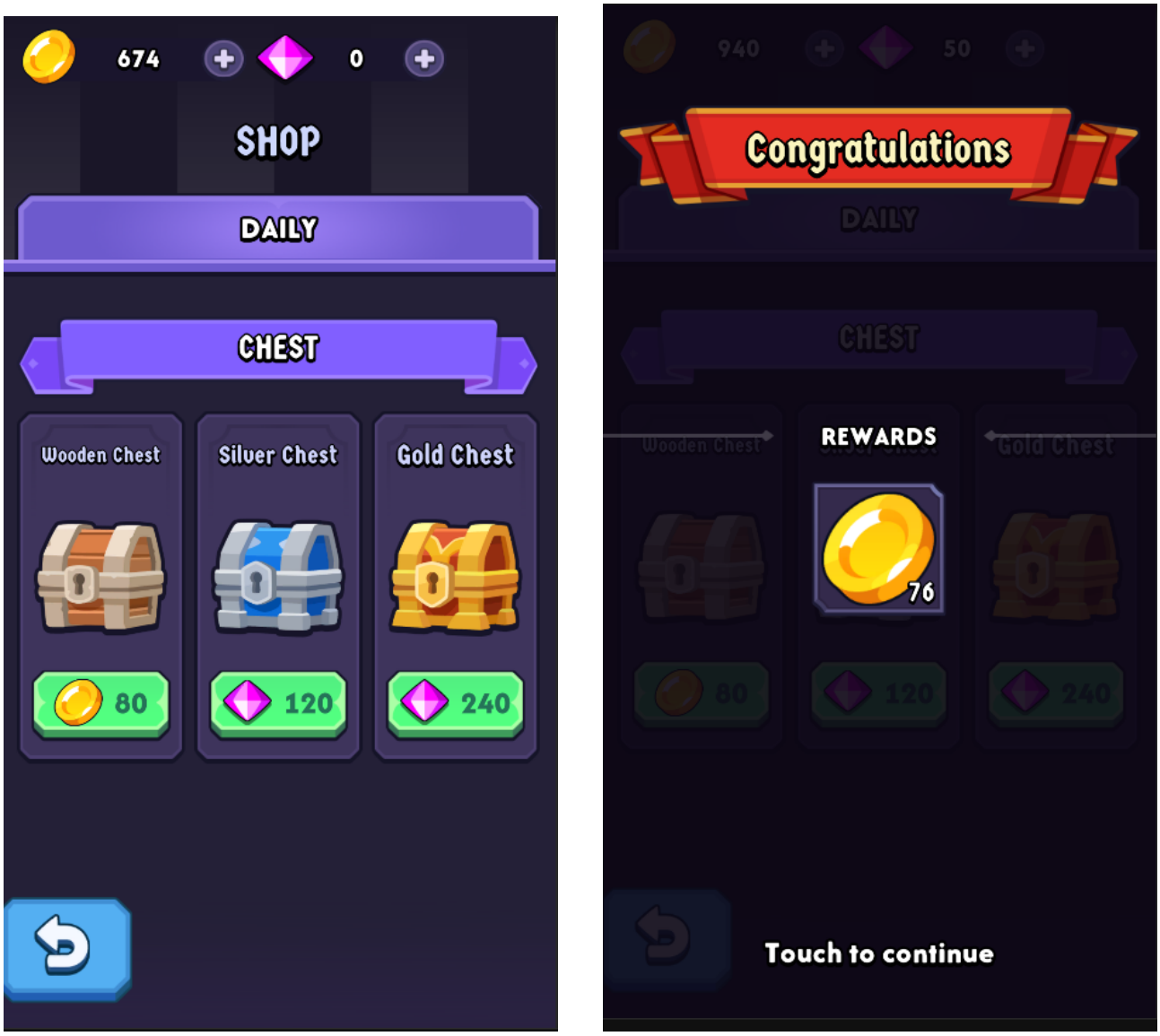
(a) Daily quest (b) Weekly quest (c) Achievement

Hình 21: UI nhiệm vụ ngày, tuần và thành tựu

Mapping nghiệp vụ và API:

- **Endpoint**: `/quests`.
- **Claim quest**: `/quests/{questId}/claim`.
- **Claim progress reward**: `/quests/progress-rewards/{track}/{stage}/claim`.
- **Dữ liệu**: daily/weekly/achievement gồm progress, required, trạng thái completed/claimed và reward.
- **Milestone**: thanh mốc phía trên đọc `points` và `progressRewards`; icon reward lấy từ item id do backend trả.

7.8.4 UI shop và popup phần thưởng



(a) Màn hình UIShop

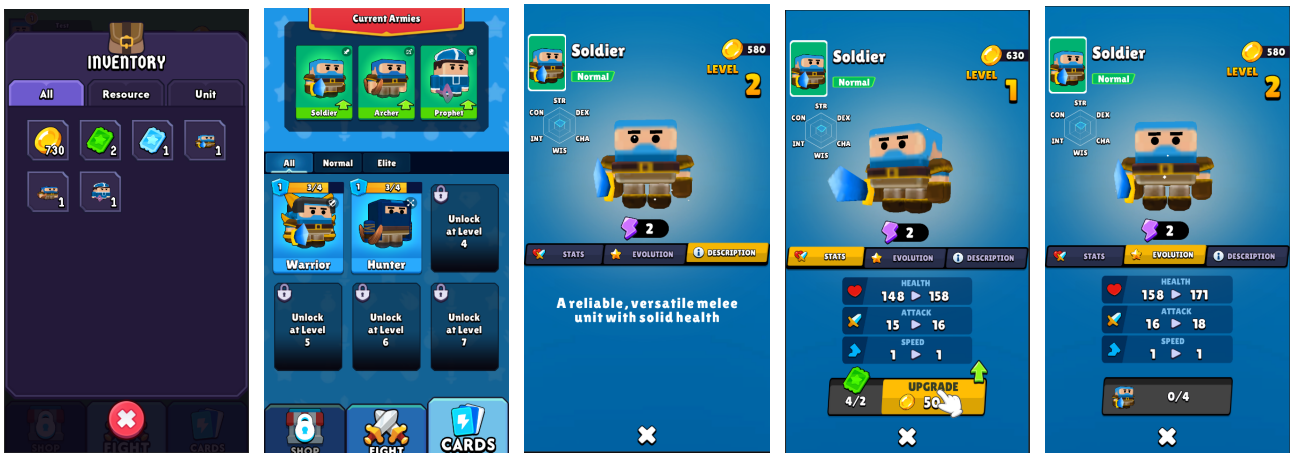
(b) Popup nhận thưởng sau khi mua

Hình 22: UI shop: catalog offer và popup reward

Mapping nghiệp vụ và API:

- **Điều hướng UI**: mở `UIShop` từ tab Shop trong `UISetup`; nút back quay lại `UISetup`.
- **Catalog endpoint**: `GET/shop/catalog`.
- **Purchase endpoint**: `POST/shop/offers/{offerId}/purchase`.
- **Dữ liệu**: catalog trả `offers`, `currency`, `resetAt`, `configVersion`; UI bind giá theo `priceCurrency`/`priceAmount`.
- **Popup reward**: hiển thị danh sách từ `grantedRewards`/`rewards` do backend trả về sau mua.

7.8.5 UI hero inventory, upgrade và evolve



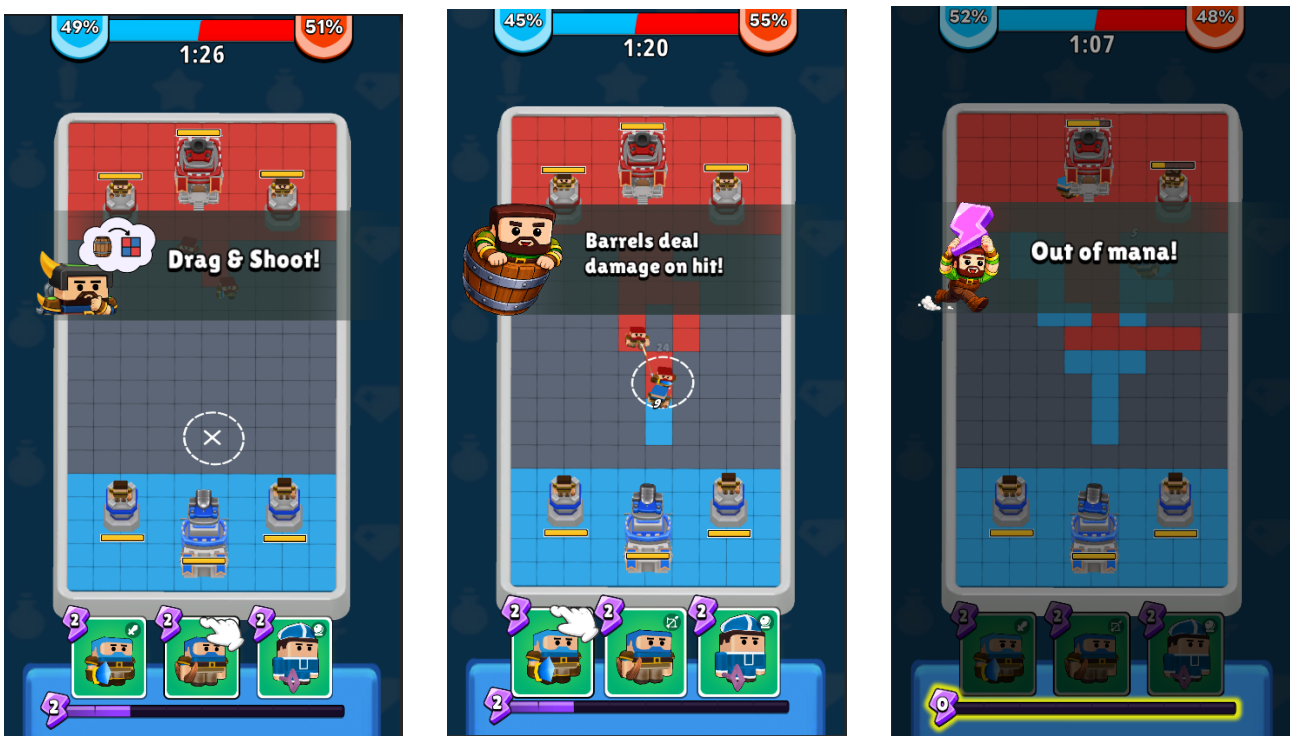
(a) Hero inventory (b) UIUnit (c) Chi tiết unit (d) Upgrade (e) Evolve

Hình 23: UI quản lý hero/unit: danh sách, UIUnit, chi tiết, nâng cấp và tiến hoá

Mapping nghiệp vụ và API:

- **Upgrade endpoint**: `/inventory/heroes/{instanceId}/upgrade`.
- **Evolve endpoint**: `/inventory/heroes/{instanceId}/evolve`.
- **Dữ liệu**: danh sách hero instance (màn UIUnit), `customData.lv`, `customData.evlove_lv`, shard và gold hiện có.
- **Ràng buộc**: backend kiểm tra chi phí, config version và idempotency; client chỉ cập nhật UI sau khi nhận state mới.

7.8.6 UI tutorial và onboarding



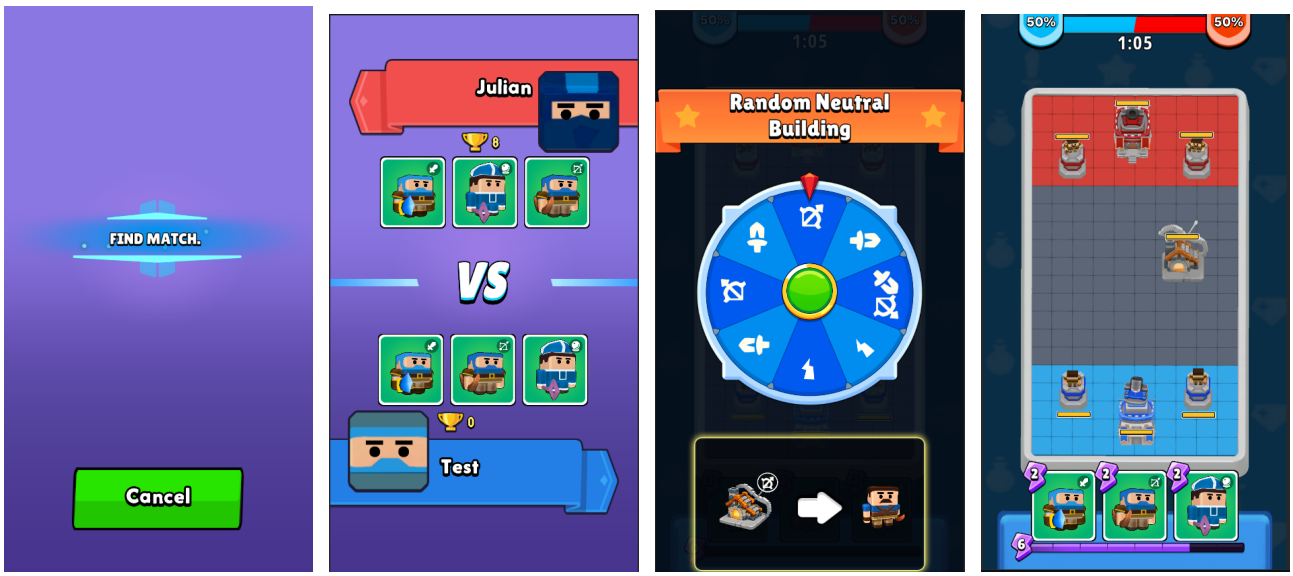
(a) Bước hướng dẫn 1 (b) Bước hướng dẫn 2 (c) Bước hướng dẫn 3

Hình 24: UI tutorial điều hướng người chơi mới

Mapping nghiệp vụ và API:

- **Tutorial progress**: `finishOnboarding`, `finishFirstDeploy`.
- `isDoneUpgradeUnitTutorial`.
- **Endpoint**: `PATCH /me/tutorial-progress`.
- battle finish cũng trả tutorial state mới để client đồng bộ.
- **Ràng buộc UI**: chỉ hiển thị hand/dim khi bước trước đã hoàn tất và không có popup reward hoặc level-up đang mở.

7.8.7 UI vào trận và gameplay battle



(a) Finding match (b) Matching (c) Roulette (d) Battle

Hình 25: UI chuẩn bị trận và gameplay battle

Mapping nghiệp vụ và API:

- **Start battle endpoint**: `POST/battles/start`.
- **Response chính**: trả `battleId`, seed, mode và config version.
- **Gameplay**: client render battle, territory, unit deployment và hiệu ứng; backend lưu session phục vụ reward/state authoritative.
- **Finish battle endpoint**: `POST/battles/{battleId}/finish`.
- **Response chính**: trả reward, statistics, currency, tutorial progress và quest updates.

7.8.8 UI kết quả trận và reward



(a) End match (b) Victory reward (c) Defeat reward (d) Level up

Hình 26: UI kết quả trận, reward và level-up

Mapping nghiệp vụ và API:

- **Reward**: client hiển thị đúng danh sách `grantedRewards`/`rewards` do backend trả.
- **Level-up**: `statistics.level` và `statistics.exp` là nguồn sự thật; client không tự suy diễn level nếu backend chưa tăng level.
- **Quest update**: battle finish trả `questUpdates` để cập nhật daily/weekly/achievement progress ngay sau trận.

7.9 Bảo mật, nhất quán và quan sát hệ thống

7.9.1 Bảo mật

Các nguyên tắc:

- mật khẩu lưu dạng hash; không lưu plaintext;
- token có thời hạn; khi ban user cần thu hồi session;
- phân quyền rõ theo vai trò người dùng và phạm vi endpoint;
- giới hạn tần suất login/purchase/claim để chống spam.

7.9.2 Nhất quán dữ liệu và giao dịch nghiệp vụ

Các thao tác như reward, claim, equip/unequip, nâng cấp và purchase cần đảm bảo:

- **atomic ở mức tài liệu** (MongoDB document update) khi cập nhật trong cùng thực thể;
- **idempotency** cho request dễ bị gửi lại (retry do mạng);
- **log nghiệp vụ** để có thể đối soát và khôi phục khi lỗi.

7.9.3 Quan sát hệ thống (observability)

Thiết kế log tối thiểu:

- request log: endpoint, latency, status code, user id (nếu có);
- business log: reward delivery, currency delta, equip/skill-up, castle upgrade, shop purchase;
- state apply log: `/me`, quest sync, battle finish và thời điểm apply state trên client.

7.10 Tổng kết chương

Chương này đã mô tả thiết kế hệ thống ở mức kiến trúc, mô-đun, dữ liệu, giao tiếp và UI/UX tham chiếu cho *Fortress of the Fallen*. Thiết kế nhấn mạnh ranh giới trách nhiệm client–server theo hướng server-authoritative, tổ chức backend theo mô-đun rõ ràng, tách dữ liệu tiến trình khỏi dữ liệu cấu hình, và xây dựng pipeline quản trị cấu hình theo hướng data-driven. Phần thiết kế dữ liệu đã cập nhật ERD (inventory/equipment/skills) và bổ sung data dictionary có caption để các bảng được đánh số và xuất hiện trong danh sách bảng. Các quyết định thiết kế này là nền tảng để mô tả hiện thực hệ thống và kiểm thử trong các chương tiếp theo [7, 8, 9, 10, 2].

8 Hiện thực và tích hợp hệ thống

8.1 Mục tiêu chương

Chương này mô tả trạng thái hiện thực đang vận hành của hệ thống *Fortress of the Fallen*, tập trung vào các luồng đã chạy thực tế giữa Unity client và backend API. Mục tiêu là đối chiếu trực tiếp giữa thiết kế ở Chương 7 và hành vi runtime hiện tại, đồng thời tách rõ phần mở rộng chưa triển khai.

8.2 Baseline triển khai hiện tại

Tại thời điểm viết báo cáo, các luồng cốt lõi đã tích hợp gồm:

- Authentication: register/login/refresh/logout.

- Đồng bộ player state qua `GET /me`.
- Quest: tải quest, claim quest, claim progress reward.
- Battle session: `battles/start` và `battles/{id}/finish`.
- Tutorial progress sync: `PATCH /me/tutorial-progress`.
- Progression thao tác trực tiếp: nâng cấp hero/skill, evolve hero.
- Castle progression: nâng cấp castle qua endpoint backend chuyên biệt.
- Shop flow: tải catalog, purchase offer và hiển thị popup reward.

Các luồng trên là nguồn sự thật chính cho dữ liệu người chơi (level, exp, currency, quest progress, tutorial flags, castle state, shop purchase state).

8.3 Hiện thực phía client (Unity)

8.3.1 Tổ chức lớp giao tiếp backend

Client tổ chức lớp API theo mô hình:

- **BackendApiClient**: gửi request HTTP, parse response chuẩn, xử lý lỗi mạng/timeout.
- **APIManager**: điều phối luồng nghiệp vụ mức game (auth success, load `/me`, sync quest, battle start/finish, tutorial update, upgrade actions).
- **UI layer**: các màn hình `UILoginAuth`, `UISetupBattle`, `UITask`, `UIUnits`, `UIReward`, `UILevelUp`, `UIShop`, `PopUpShop` đọc dữ liệu đã chuẩn hoá từ APIManager.

8.3.2 Nguyên tắc đồng bộ dữ liệu

Client áp dụng nguyên tắc:

- Backend là nguồn sự thật cho statistics/currency/quest/tutorial/castle/shop.
- UI chỉ hiển thị theo dữ liệu phản hồi; không tự "chốt" reward, level hay quest state nếu backend chưa xác nhận.
- Sau các thao tác nhạy cảm (battle finish, quest claim, upgrade), client thực hiện cập nhật lại state theo payload backend.

8.4 Hiện thực phía backend theo contract

8.4.1 Nhóm endpoint đã tích hợp

Các nhóm endpoint được client gọi trong runtime hiện tại:

- **Auth**: `/auth/register`, `/auth/login`, `/auth/refresh`, `/auth/logout`.
- **Player state**: `/me`.
- **Tutorial**: `/me/tutorial-progress`.
- **Quest**: `/quests`, claim quest, claim progress reward.
- **Battle**: `/battles/start`, `/battles/{battleId}/finish`.
- **Inventory progression**: upgrade/evolve hero, purchase/upgrade skill.
- **Castle**: `/lobby/castle/upgrade`.
- **Shop**: `/shop/catalog`, `/shop/offers/{offerId}/purchase`.

8.4.2 Quy tắc dữ liệu quan trọng

- Battle finish trả đồng thời reward, statistics, currency, tutorialProgress và questUpdates để giảm lệch state giữa màn battle và lobby.
- Tutorial flags được cập nhật qua hai đường: payload battle finish và endpoint patch riêng; client hợp nhất theo trạng thái mới nhất từ backend.
- Các thao tác có khả năng retry dùng idempotency key theo contract để tránh cấp trùng.
- Castle upgrade dùng endpoint riêng để không phụ thuộc vào biến cục bộ client.

8.5 Luồng tích hợp đã kiểm chứng

8.5.1 Luồng xác thực và vào lobby

1. Người chơi login/register thành công, nhận access/refresh token.
2. Client gọi `/me`, nhận các trường chính: profile, statistics, tutorial, currency, inventory, formation, quests.
3. APIManager áp dụng state vào dữ liệu local và mở lobby.

8.5.2 Luồng quest và reward

1. Client đồng bộ `/quests` khi vào game và khi cần refresh.
2. Người chơi claim quest/progress reward qua endpoint tương ứng.
3. UI quest cập nhật tiến độ và trạng thái claim theo dữ liệu backend.

8.5.3 Luồng battle và hậu battle

1. Trước trận: client gọi `/battles/start`, nhận `battleId`, seed và config version.
2. Sau trận: client gọi `/battles/{battleId}/finish` với idempotency key.
3. Backend trả reward/statistics/tutorial/quest updates; client cập nhật lobby UI theo payload mới.

8.5.4 Luồng level-up, tutorial và castle

- Màn `UILevelUp` hiển thị theo delta trước/sau do backend trả; level thực tế lấy từ `statistics.level`.
- `finishFirstDeploy` và các tutorial flag được sync về backend để tránh lệch khi đổi thiết bị/phiên.
- Nâng cấp castle đi qua API riêng, cập nhật lại state hiển thị sau phản hồi thành công.

8.5.5 Luồng shop purchase và popup reward

1. Người chơi mở tab Shop từ `UISetup`; client gọi `GET/shop/catalog`.
2. Client bind offer theo `offerId`/`priceCurrency`/`priceAmount` và currency hiện có từ backend.
3. Khi bấm mua, client gọi `POST/shop/offers/{offerId}/purchase` kèm `idempotencyKey`.
4. Backend trả currency mới và `grantedRewards`/`rewards`; client mở `PopUpShop` để hiển thị phần thưởng.

8.6 Claim và Evidence

Claim kỹ thuật	Evidence triển khai	Trạng thái
Auth vòng đời phiên	Client gọi register/login/refresh/logout, nhận và xoay token theo API contract	Đã triển khai
Lobby lấy state tập trung từ backend	`GET /me` trả profile, statistics, tutorial, inventory, formation, quest; client apply một lần vào state local	Đã triển khai
Quest sync và claim theo backend	`/quests` + claim endpoints; UI daily/ weekly/achievement cập nhật từ payload backend	Đã triển khai
Battle reward là server-authoritative	`battles/start` + `battles/{id}/finish`; reward/ currency/quest update trả từ backend	Đã triển khai
Tutorial progress đồng bộ liên phiên	`PATCH /me/tutorial-progress` + tutorial payload từ battle finish	Đã triển khai
Upgrade hero/skill theo API	Client gọi endpoint nâng cấp, sau đó đồng bộ lại trạng thái từ phản hồi backend	Đã triển khai
Castle upgrade có backend source-of-truth	Luồng nâng cấp castle dùng endpoint chuyên biệt, không xử lý local-only	Đã triển khai
Shop purchase và reward popup đồng bộ backend	Catalog/purchase gọi qua API shop; popup reward đọc trực tiếp từ `grantedRewards`/`rewards` response	Đã triển khai

Bảng 17: Bảng traceability claim kỹ thuật và bằng chứng triển khai

8.7 Các hạng mục mở rộng chưa hoàn tất

- Bổ sung bộ benchmark tải và dashboard observability cho các luồng chính.
- Tiếp tục mở rộng nội dung gameplay và cân bằng kinh tế ở mức live-ops.

8.8 Tổng kết chương

Hệ thống hiện đã vượt mức "prototype thiết kế" và có tích hợp runtime cho các luồng cốt lõi: auth, player state, quest, battle, tutorial sync, progression, castle upgrade và shop purchase. Phần còn lại được tách rõ thành roadmap mở rộng để tránh nhầm lẫn giữa tính năng đã chạy và định hướng tương lai.

9 Đánh giá hệ thống

9.1 Mục tiêu và phương pháp đánh giá

Mục tiêu của chương này là đánh giá mức độ đáp ứng các yêu cầu đã chuẩn hóa ở Chương 5 theo phạm vi triển khai thực tế. Phương pháp gồm:

- đối chiếu trực tiếp FR–NFR với luồng runtime và API contract;
- kiểm tra tính nhất quán dữ liệu sau các thao tác hậu battle, quest claim, tutorial sync và upgrade;
- xác định rủi ro vận hành còn lại và biện pháp giảm thiểu.

9.2 Bao phủ yêu cầu chức năng

FR	Phân hệ	Đánh giá trạng thái
FR-01, FR-02	Auth và vòng đời phiên	Register/login/refresh/logout đã chạy ổn định theo contract
FR-03	Đồng bộ player state	`/me` trả các trường chính của profile, statistics, tutorial, currency, inventory, formation, quests
FR-04	Quest sync và claim reward	Daily/weekly/achievement và trạng thái claim được cập nhật theo backend source-of-truth
FR-05, FR-06	Battle session start/finish	`battles/start` + `battles/{id}/finish` trả reward, currency, statistics, tutorialProgress, questUpdates
FR-07	Tutorial progress sync	`PATCH /me/tutorial-progress` và payload battle finish giữ tutorial flags nhất quán
FR-08, FR-09	Hero/skill progression	Luồng nâng cấp đã tích hợp API và phản ánh về UI Units theo response backend
FR-10	Castle progression	Nâng cấp castle đi qua endpoint backend riêng, không phụ thuộc local-only
FR-11	Shop catalog/purchase	`/shop/catalog` và `/shop/offers/{offerId}/purchase` chạy ổn định; reward popup hiển thị theo payload backend

Bảng 18: Đối chiếu mức bao phủ FR theo trạng thái triển khai hiện tại

9.3 Đánh giá nhất quán dữ liệu

9.3.1 Điểm đã đạt

- Battle finish trả gói hậu trận ở mức cần thiết, giảm sai lệch giữa battle scene và lobby state.

- Quest progress/reward claim hiển thị theo dữ liệu backend, không chốt theo suy diễn local.
- Tutorial flags được đồng bộ theo cả luồng patch riêng và payload hậu battle.
- Castle upgrade đã có backend source-of-truth nên ổn định hơn khi đổi scene hoặc reload.
- Shop catalog/purchase đồng bộ currency và reward theo response backend; popup nhận thưởng lấy trực tiếp từ payload purchase.

9.3.2 Điểm cần kiểm soát

- Cần tránh áp dụng response cũ đè lên state mới trong các request bất đồng bộ.
- Các thao tác retry phải duy trì idempotency key thống nhất giữa client và backend.
- Thứ tự cập nhật UI hậu battle phải luôn đi theo dữ liệu mới nhất từ backend.

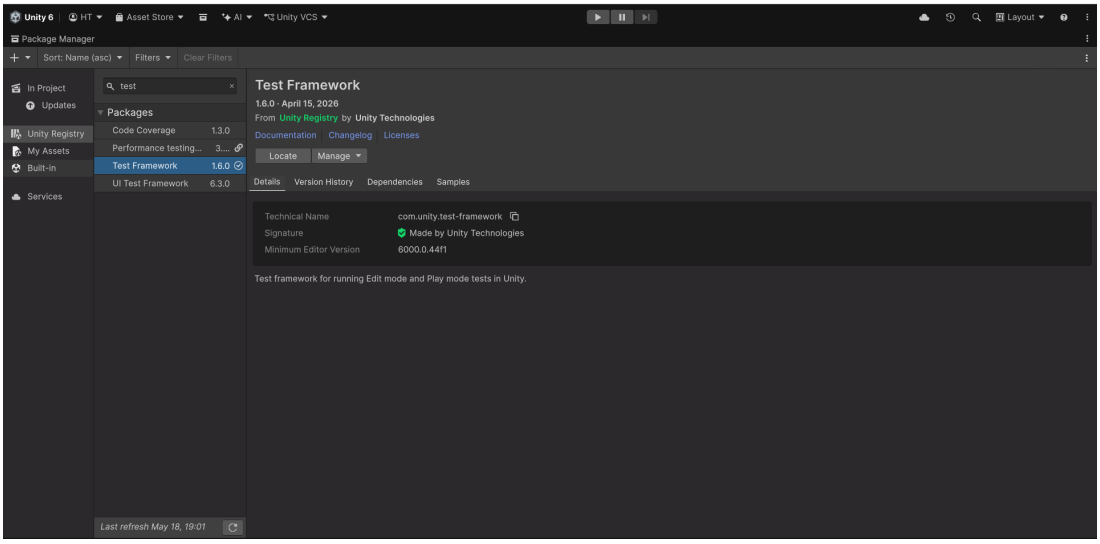
9.4 Đánh giá phi chức năng

- **NFR-01, NFR-02**: vòng đời token và ranh giới server-authoritative đang bám đúng baseline.
- **NFR-03, NFR-04**: đã có idempotency contract cho thao tác nhạy cảm (battle/claim/shop/upgrade); còn cần siết thêm xử lý stale response ở một số nhánh UI.
- **NFR-05**: log runtime hiện đủ cho trace luồng chính; nên tăng metric dashboard trước production scale.
- **NFR-06, NFR-07**: cấu trúc APIManager + UI layer giúp tách luồng rõ; trạng thái loading/error đã được thể hiện trong các màn chính.
- **NFR-08**: configVersion đã đi cùng luồng battle/config manifest để giảm lệch cấu hình.

9.5 Automation Testing với Unity Test Framework

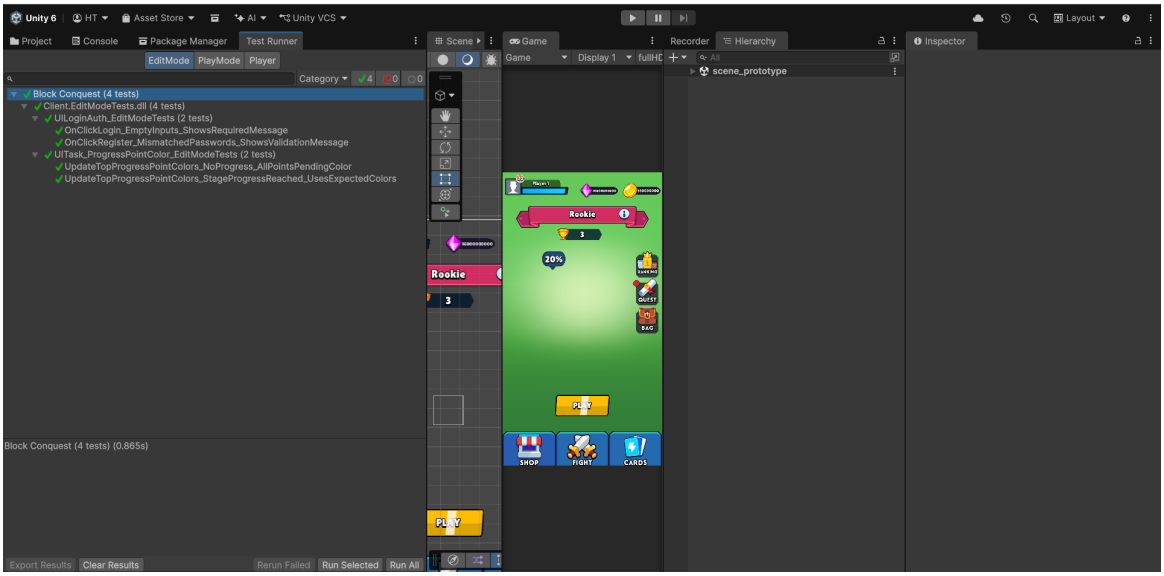
Nhóm sử dụng Unity Test Framework để tự động hóa kiểm thử hồi quy cho các luồng UI và logic cốt lõi đã triển khai. Mục tiêu là phát hiện sớm lỗi sau khi thay đổi code, đặc biệt ở các luồng xác thực, đồng bộ trạng thái và hiển thị UI theo dữ liệu backend.

9.5.1 Cài đặt package kiểm thử



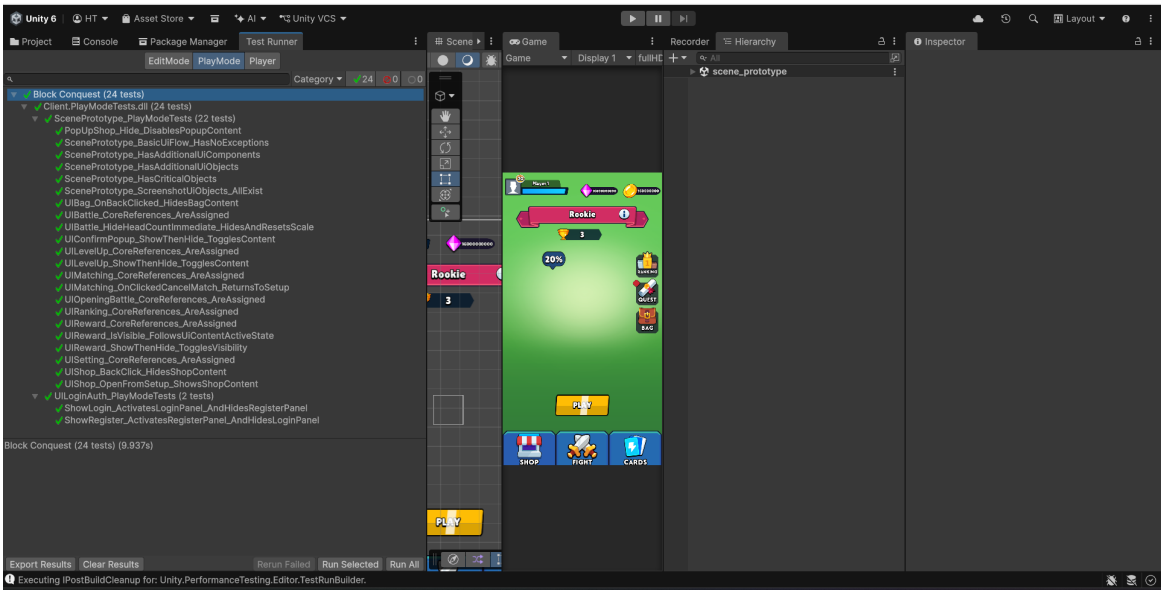
Hình 27: Unity Package Manager với Test Framework và UI Test Framework được cài trong dự án

9.5.2 Bộ EditMode tests



Hình 28: Kết quả chạy EditMode tests cho các ca kiểm thử logic UI cục bộ

9.5.3 Bộ PlayMode tests



Hình 29: Kết quả chạy PlayMode tests trên scene runtime để kiểm tra tích hợp thành phần

9.5.4 Kết quả chạy test trên player/test runner



Hình 30: Kết quả test runner ở chế độ player dùng để đối chiếu hành vi kiểm thử ngoài editor

Các ảnh trên thể hiện quy trình test đã được tích hợp ở mức khả dụng cho regression cơ bản. Bộ test hiện tại chưa đại diện cho toàn bộ bề mặt hệ thống, nhưng đủ làm bằng chứng tự động hóa cho các flow chính đã vận hành trong baseline.

9.6 Rủi ro còn lại và hướng giảm thiểu

Rủi ro	Mức	Mô tả	Giảm thiểu
Stale state hậu battle	Trung bình	UI nhận response chậm và có thể hiển thị lại state cũ	So sánh timestamp/ version trước khi apply state
Retry gây thao tác lặp	Cao	Request finish/claim/upgrade bị gửi lại do mạng	Bắt buộc idempotency key và kiểm tra duplicate phía backend
Drift tài liệu so với runtime	Trung bình	Tài liệu thay đổi chậm hơn code/API	Duy trì bảng đối chiếu FR định kỳ theo mỗi release
Thiếu benchmark tải	Trung bình	Chưa có bộ số liệu tải định lượng khi scale production	Bổ sung kịch bản load test và theo dõi metric nền

Bảng 19: Rủi ro hiện tại và biện pháp giảm thiểu

9.7 Tổng kết chương

Kết quả đánh giá cho thấy hệ thống đã đáp ứng phần lớn baseline implemented-only của Chương 5, đặc biệt ở các luồng auth, player state, quest, battle, tutorial sync, upgrade, castle progression và shop purchase. Các rủi ro còn lại chủ yếu thuộc nhóm đồng bộ bất đồng bộ và vận hành khi mở rộng tải.

10 Kết luận và hướng phát triển

10.1 Tổng kết kết quả hiện tại

Đề tài *Fortress of the Fallen* đã đi từ mức thiết kế sang trạng thái **đã tích hợp vận hành** cho các luồng cốt lõi giữa Unity client và backend. Kết quả chính:

- Hoàn chỉnh luồng auth: register/login/refresh/logout.
- Đồng bộ player state tập trung qua `/me`.
- Vận hành quest sync và claim theo backend.
- Vận hành battle session start/finish với reward và cập nhật progression.
- Đồng bộ tutorial progress liên phiên.
- Tích hợp upgrade hero/skill và castle upgrade theo API backend.
- Vận hành shop catalog/purchase và popup reward theo payload backend.

10.2 Đóng góp

- Thiết lập baseline kiến trúc client–server có khả năng mở rộng, nhưng vẫn bám sát luồng game thực tế.
- Chuẩn hoá nguyên tắc backend source-of-truth cho progression (exp/level/quest/reward/tutorial/castle/shop currency).
- Đồng bộ tài liệu kỹ thuật theo hướng claim có evidence, giảm drift giữa báo cáo và runtime.

10.3 Hạn chế hiện tại

- Benchmark tải vận hành chưa hoàn tất.
- Một số phân hệ mở rộng (live-ops quy mô lớn, cơ chế giảm gian lận, dashboard observability cho nhiều kịch bản) vẫn ở roadmap.
- Cần tiếp tục siết quy ước version/timestamp để chống stale response ở mọi màn UI.

10.4 Hướng phát triển tiếp theo

- Hoàn thiện benchmark tải, metric và cảnh báo vận hành.
- Tiếp tục chuẩn hoá contract và idempotency cho toàn bộ endpoint nhạy cảm.
- Mở rộng content/economy theo roadmap live-ops.

10.5 Kết luận

Hệ thống hiện không còn ở phạm vi "chỉ thiết kế". Các luồng gameplay/lobby cốt lõi đã được triển khai và đồng bộ theo backend source-of-truth. Nền tảng này đủ vững để chuyển trọng tâm sang tối ưu vận hành và hoàn thiện các phân hệ nâng cao.

Phân công nhiệm vụ và đóng góp của các thành viên

Phần này tổng hợp phạm vi công việc và mức đóng góp của từng thành viên trong suốt quá trình thực hiện đồ án. Việc phân công được xây dựng dựa trên thế mạnh của mỗi thành viên và được phối hợp xuyên suốt theo từng mốc triển khai, nhằm đảm bảo tính nhất quán giữa trải nghiệm người dùng (client/UI) và xử lý nghiệp vụ (server/DB/giao tiếp).

Hạng mục	**Mô tả công việc**	**Thành viên chính**	**Phối hợp**
Lên ý tưởng đề tài	Thống nhất định hướng gameplay, scope hệ thống, các phân hệ chính (auth, player state, inventory/equipment, skills, quest, battle, castle, shop).	Toàn + Khang	Cả hai
Tìm game tham khảo và chốt hướng UI	Tìm các tựa game/reference phù hợp, tổng hợp luồng UI, bố cục màn hình và quy ước tương tác để chốt phong cách giao diện và trải nghiệm.	Hà Thái Toàn	Trần Minh Khang
Client và UI/UX	Thiết kế luồng màn hình (login/register, profile, create character, main hub, inventory), định nghĩa UI state (loading/error/disabled), chuẩn hoá dữ liệu hiển thị theo response từ API.	Hà Thái Toàn	Trần Minh Khang
Đề xuất tính năng và đánh giá khả thi	Đề xuất các tính năng trọng tâm theo phạm vi đồ án; đánh giá rủi ro/độ phức tạp; chốt các chức năng phải server-authoritative và các điểm cần audit/log.	Trần Minh Khang	Hà Thái Toàn
Hiện thực hệ thống server	Thiết kế/hiện thực backend theo mô-đun; tổ chức REST API cho các luồng cốt lõi; xác thực, xử lý lỗi; chuẩn hoá payload; quản lý session runtime và luồng nghiệp vụ chính.	Trần Minh Khang	Hà Thái Toàn
Đồng bộ client–server	Thống nhất contract dữ liệu giữa UI và API (endpoint, request/response), rà soát mismatch, điều chỉnh luồng để đảm bảo trải nghiệm mượt và dữ liệu nhất quán.	Cả hai	Cả hai

Bảng 20: Phân công nhiệm vụ và đóng góp của các thành viên

Roadmap giai đoạn tiếp theo

Phần này mô tả roadmap sau baseline đã triển khai, tập trung vào các hạng mục còn thiếu để nâng mức sẵn sàng vận hành. Mốc thời gian dưới đây được chốt theo kế hoạch tại thời điểm **tháng 05/2026**.

Ưu tiên	Thời gian	Hạng mục	Kết quả mong đợi
P1	05–06/2026	Chuẩn hoá observability (metrics/ tracing/alert) cho auth, quest, battle, tutorial, upgrade, castle, shop	Có dashboard vận hành và ngưỡng cảnh báo
P1	05–06/2026	Hoàn thiện bộ test hồi quy cho các luồng đã chạy và kịch bản retry idempotency	Giảm lỗi drift state và lỗi cấp trùng
P2	07–08/2026	Mở rộng mode đối kháng theo kiến trúc backend-authoritative (room scaling, reconnect policy)	Luồng đối kháng ổn định hơn theo tiêu chí vận hành
P2	07–08/2026	Benchmark tải và tối ưu hiệu năng backend/client theo số liệu thực nghiệm	Báo cáo hiệu năng và kế hoạch scale
P3	09/2026 trở đi	Mở rộng live-ops: event loop, economy tuning, content pipeline sâu hơn	Nâng mức vận hành nội dung dài hạn

Bảng 21: Roadmap sau baseline triển khai (mốc thời gian tính từ 05/2026)

Nguyên tắc triển khai roadmap:

- Không phá vỡ baseline đã ổn định (auth/me/quest/battle/tutorial/progression/castle/shop).
- Mọi tính năng mới phải có claim $\leftrightarrow$ evidence trước khi ghi vào báo cáo là "đã triển khai".

Tài liệu

[1] Newzoo. *Global Games Market Report*. Online report. Báo cáo thống kê thị trường game toàn cầu. 2023.

[2] Michael Moriarty. "Networked Physics and Latency Compensation in Online Games". **in***Proceedings of the Game Developers Conference*: Trình bày các kỹ thuật client-side prediction, interpolation, reconciliation. 2014.

[3] Jason Gregory. *Game Engine Architecture*. 3 **edition**. Kiến trúc game engine, kiến trúc nhiều lớp và real-time system. CRC Press, 2018.

[4] Ernest Adams. *Fundamentals of Game Design*. 3 **edition**. Tài liệu cơ bản về thiết kế game, core loop, progression, economy. New Riders, 2014.

[5] Jesse Schell. *The Art of Game Design: A Book of Lenses*. 3 **edition**. Các nguyên lý thiết kế trải nghiệm, level design, UX trong game. CRC Press, 2019.

[6] Unity Technologies. *Unity User Manual*. `https://docs.unity3d.com/Manual/index.html`. Tài liệu chính thức hướng dẫn sử dụng Unity. 2023.

[7] Kamil Myśliwiec **and** NestJS Team. *NestJS Documentation*. `https://docs.nestjs.com`. Tài liệu chính thức của NestJS. 2023.

[8] Kristina Chodorow. *MongoDB: The Definitive Guide*. 2 **edition**. Giới thiệu và hướng dẫn sử dụng MongoDB. O'Reilly Media, 2013.

[9] Josiah L. Carlson. *Redis in Action*. Ứng dụng Redis cho cache, pub/sub, lưu trữ session. Manning Publications, 2013.

[10] Glenn Fiedler. *Gaffer on Games: Networking for Game Programmers*. Tài liệu tham khảo về server authoritative, prediction, interpolation. 2010.

[11] Microsoft. *C# Programming Guide*. `https://learn.microsoft.com/dotnet/csharp/`. Tài liệu chính thức về ngôn ngữ C#. 2023.

[12] OpenJS Foundation. *Node.js Documentation*. `https://nodejs.org/en/docs`. Tài liệu chính thức của Node.js. 2023.

[13] GitHub. *GitHub Documentation*. `https://docs.github.com`. Hướng dẫn sử dụng GitHub và Git cơ bản. 2023.

[14] GitHub. *GitHub Projects Documentation*. `https://docs.github.com/issues/planning-and-tracking-with-projects`. Hướng dẫn sử dụng GitHub Projects để quản lý tác vụ. 2023.

[15] Thomas Allweyer. *BPMN 2.0: Introduction to the Standard for Business Process Modeling*. Giới thiệu BPMN 2.0 dùng cho mô hình hoá quy trình. Books on Demand, 2016.

[16] Frank Mittelbach **andothers**. *The LaTeX Companion*. 2 **edition**. Tài liệu tham khảo về soạn thảo tài liệu bằng LaTeX. Addison-Wesley, 2004.

[17] Reki Kawahara. *Sword Art Online, Vol. 1: Aincrad*. Nguồn tham khảo ý tưởng leo tầng và boss tầng. Yen Press, 2014.

[18] Kugane Maruyama. *Overlord, Vol. 1: The Undead King*. Nguồn tham khảo cách xây dựng NPC và thế giới vận hành. Yen Press, 2016.

[19] Pick Me Up! Infinite Gacha. *Pick Me Up! Infinite Gacha (Webtoon/Manhwa)*. Nguồn tham khảo cơ chế gacha và meta-progression. 2022.

[20] Arcane Odyssey. *Arcane Odyssey (Game)*. Nguồn tham khảo trải nghiệm online RPG và cảm giác điều khiển. 2023.

[21] ChillyRoom. *Soul Knight Prequel (Game)*. Nguồn tham khảo vòng lặp phiên chơi ngắn và reward theo phiên. 2023.

[22] Supercell. *Clash of Clans (Game)*. Nguồn tham khảo hệ thống căn cứ/công trình và quản trị tài nguyên. 2012.